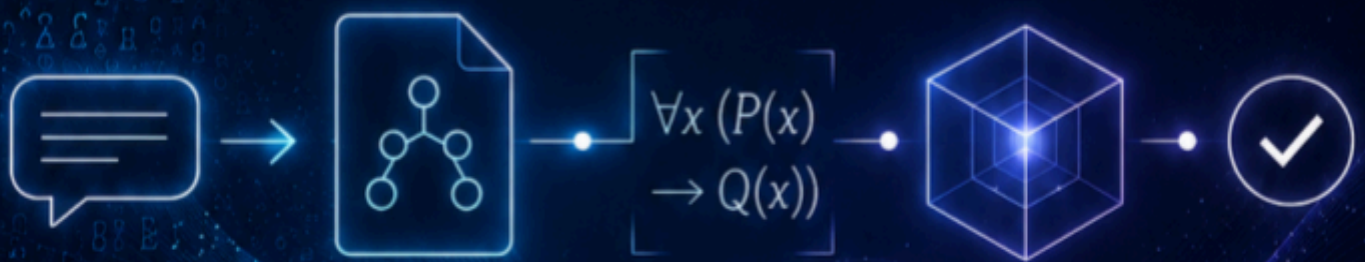


LINGUAGGIO NATURALE ESEGUIBILE

Un'Indagine Critica su Semantica Formale,
Interpretazione Astratta, Esecuzione Simbolica e
Architetture LLM Simbolicamente Fondate



LINGUAGGIO NATURALE ESEGUIBILE

Un'Indagine Critica su Semantica Formale, Interpretazione
Astratta, Esecuzione Simbolica e Architetture LLM
Simbolicamente Fondate

*Verso un Runtime Neuro-Simbolico per Documenti Lunghi, Regole e Sistemi
Agentici*



Copyright © [2026] Axiologic Research

Tutti i diritti riservati.

Diritti di pubblicazione KDP detenuti da Outfinity SRL (Iași, Romania).

Disponibile gratuitamente sul sito Axiologic (www.axiologic.net).

Nota dell'Autore

Questo lavoro è stato creato sotto l'egida di Axiologic Research come parte di due programmi di ricerca interconnessi, uno dedicato all'Intelligenza Artificiale e l'altro all'Outfinitismo, entrambi guidati da Sînică Alboaie. Per esplorare i dettagli di come questi due programmi si intersecano e si rafforzano reciprocamente, invitiamo i lettori a visitare la pagina di Outfinity Venture Studio all'indirizzo www.outfinity.ch. Sebbene

modelli linguistici di Intelligenza Artificiale siano stati usati per assistere la generazione del testo, la filosofia fondamentale, la struttura tematica e l'approccio analitico derivano esclusivamente da decenni di studio dedicato dell'autore sui fenomeni sociali, sulle tecnologie sociali e su autentiche tecnologie dure risultanti da vari progetti di ricerca europei e autofinanziati. Di recente, nell'ambito del progetto di ricerca Achilles, abbiamo studiato in profondità la letteratura generata dall'IA e la sua verifica automatizzata mediante approcci neuro-simbolici; tutti i libri gratuiti messi a disposizione del pubblico fanno parte dell'insieme di opere che utilizziamo per testare le nostre tecnologie più recenti.

Indice

Sintesi	5
Prefazione: Ambito, Metodo e Posizione Critica	8
1. Dal Linguaggio Fluente alla Semantica Eseguitibile	11
1.1 Che cosa Richiede l'Esecuzione e Dove Appartiene il Confine di Fiducia	11
1.2 Il Divario di Formalizzazione: Perché un Risolutore Corretto Può Ancora Produrre la Risposta Sbagliata	15
2. La Genealogia Intellettuale del Linguaggio Eseguitibile	18
2.1 Linguaggi Controllati, Semantica Formale, Discorso e Logica Naturale	18
2.2 Parsing Semantico, Query Eseguitibili, Induzione di Programmi e Primi Sistemi Neuro-Simbolici	21
3. Interpretazione Astratta ed Esecuzione Simbolica per il Linguaggio	24
3.1 Interpretazione Astratta come Semantica dell'Incertezza Limitata	24
3.2 Esecuzione Simbolica, Risoluzione dei Vincoli, Verifica di Modelli e Raffinamento Semantico	28
4. Che cosa ha Realmente Ottenuto l'Era degli LLM	32
4.1 Autoformalizzazione, Ragionamento Assistito da Programmi, Logica, Specifiche Temporali e Pianificazione	32
4.2 Analisi di Programmi Assistita da LLM, Esplorazione Simbolica, Trasformatori Astratti e NLP Certificato	36
5. Perché i Sistemi di Linguaggio Eseguitibile Falliscono	40
5.1 Ancoraggio, Identità, Formalizzazioni Spurie, Ontologie e Conoscenza del Mondo	40
5.2 Pragmatica, Non Monotonicità, Scala, Sicurezza e Fallimenti della Valutazione	44
6. Famiglie Architetture per il Linguaggio Naturale Eseguitibile	48
6.1 Progetti Concorrenti: Rappresentazioni Universali, Portafogli di Formalismi, Compilazione Controllata e Sistemi Orientati alla Query	48
6.2 Una Pipeline di Riferimento dal Testo ai Simboli, all'Esecuzione e a Risposte in Linguaggio Naturale Vincolato	53
7. LongTextJS, CircuitJS e nllAgent: Un Caso di Studio Critico	56
7.1 Che cosa Implementa l'Architettura Pubblica e Perché i suoi Confini Contano	57
7.2 Valutazione Indipendente: Punti di Forza, Problemi Seri Attuali ed Evoluzione Necessaria	60
8. Costruire e Validare un Runtime Neuro-Simbolico	64
8.1 Strategia di Implementazione: un Sistema di Linguaggio Eseguitibile Minimo ma Scientificamente Onesto	64
8.2 Domande di Ricerca, Benchmark e un Programma di Pubblicazione	70
Conclusione: dalla Forma Probabilistica alla Conseguenza Simbolica	76
Riferimenti	79

Sintesi

I modelli di linguaggio di grandi dimensioni hanno reso il linguaggio naturale un'interfaccia efficace per il calcolo, ma non hanno reso il linguaggio naturale eseguibile nel senso forte in cui un programma, una query di database, un sistema di transizione o una teoria logica è eseguibile. Un modello può generare una spiegazione convincente mentre cambia la portata di un quantificatore, omettendo un'eccezione, inventando una premessa o applicando la stessa regola in modo diverso in due punti. Gli strumenti esterni migliorano l'affidabilità solo quando il modello ha tradotto correttamente la richiesta. Un dimostratore di teoremi può dimostrare perfettamente la formalizzazione errata; un interprete Python può eseguire un programma che risponde accidentalmente all'esempio travisando la domanda; un esecutore simbolico può esporre un vero percorso attraverso il software anche se la regola del linguaggio naturale utilizzata per costruire la condizione del percorso è stata erroneamente tradotta. Il problema difficile non è quindi semplicemente aggiungere un risolutore dopo un LLM. È costruire una catena controllabile dal linguaggio di origine ai simboli fondati, dai simboli fondati agli oggetti formali eseguibili e dai risultati simbolici al linguaggio naturale senza consentire al modello di linguaggio finale di alterare ciò che è stato stabilito.

Questo libro esamina le tradizioni rilevanti come un problema di ricerca collegato. I primi fondamenti includono linguaggi naturali controllati, semantica teorico-modellistica e del discorso, logica naturale, analisi semantica, programmazione logica, sistemi text-to-query, sintesi del programma da esempi e linguaggio, motori di regole, tabelle decisionali, logiche temporali, pianificazione automatizzata e sistemi neuro-simbolici. I fondamenti dell'analisi del programma includono l'interpretazione astratta, l'esecuzione simbolica e concolica, l'analisi del flusso di dati, il controllo del modello, la risoluzione SAT/SMT/CSP, Datalog e la valutazione di punto fisso, automi e verifica con produzione di prove. La recente letteratura sugli LLM aggiunge il ragionamento assistito del programma, la sollecitazione del prompting dichiarativo con solutori, l'autoformalizzazione, la traduzione logica di primo ordine, la traduzione-logica temporale, la pianificazione formale, l'auto-riparazione guidata dal solutore, l'analisi del programma assistita da LLM, l'esplorazione simbolica guidata da LLM e la sintesi di trasformatori astratti verificati. In tutti questi corpi di lavoro, i risultati più forti appaiono costantemente quando il formalismo target è abbastanza stretto da avere una semantica esplicita e l'output accettato è controllato da un componente indipendente da LLM.

L'indagine pone particolare attenzione alla distinzione tra due usi dell'interpretazione astratta. Il primo, già stabilito in NLP, verifica un modello neurale rispetto a una famiglia formalmente definita di perturbazioni testuali. I domini astratti interrutture e relazionali possono certificare che la previsione di un classificatore non cambia sotto ogni sostituzione consentita da un insieme contraddittorio specificato. Questa è una vera garanzia formale, ma riguarda la stabilità del modello sotto una relazione di perturbazione costruita; non certifica che le parafrasi umane arbitrarie conservano il significato, né che la risposta originale del modello sia vera. Il secondo uso è più speculativo e più rilevante per il linguaggio naturale eseguibile: trattando l'insieme di interpretazioni ammissibili di un documento come semantica di raccolta e informatica conservative può, non può, e conclusioni sconosciute su quel insieme. Ciò richiede domini espliciti per identità, tempo, modalità, quantità, stato di origine, forza normativa e copertura. Richiede anche una garanzia condizionata onesta: l'analisi è valida solo rispetto

all'impostazione dell'interpretazione, all'ontologia, alla politica di fondazione e alle regole di trasferimento fornite.

L'esecuzione simbolica è complementare. L'interpretazione astratta può dimostrare a buon mercato che una violazione è impossibile sotto l'astrazione corrente o che una violazione rimane possibile. L'esecuzione simbolica può quindi tentare di costruire un'interpretazione e un percorso concreti che la realizzano. Se tale percorso non esiste, il testimone fallito può rivelare quale astrazione o scelta semantica fosse troppo grossolana. Ciò suggerisce un modello di ricerca chiamato perfezionamento semantico controesterno guidato: il nucleo controesemplare e insoddisfatto di un solutore, il nucleo insoddisfatto o la prova fallita viene mappata agli esatti passaggi di origine e agli impegni semantici che lo hanno generato; allo LLM o a un essere umano viene chiesto di rivedere solo quegli impegni; l'analisi simbolica viene rigiocata. Lo LLM viene utilizzato per cercare lo spazio delle formalizzazioni, mentre il kernel simbolico determina ciò che deriva da ogni formalizzazione accettata.

I sistemi recenti mostrano frammenti importanti di questa visione. PAL delega il calcolo procedurale a Python. SatLM genera vincoli dichiarativi per un motore di soddisfacibilità. Logic-LM e LINC traducono i compiti di ragionamento del linguaggio naturale in logica formale e si basano su risolutori simbolici o prover del teorema per la deduzione. NL2TL e nl2spec traducono i requisiti in specifiche temporali, con nl2spec che supporta esplicitamente la correzione interattiva a livello di subformula. Il lavoro di pianificazione formale ha mostrato grandi guadagni quando LLMs costruisce istanze di pianificazione che vengono quindi risolte e controllate da strumenti sonori. SymGPT traduce le regole ERC in una grammatica ristretta e utilizza l'esecuzione simbolica per trovare violazioni concrete degli smart contract, segnalando migliaia di violazioni delle regole e percorsi di attacco espliciti. Nella direzione opposta, AutoBug decompone l'analisi del programma in percorsi simbolici gestiti da LLM, ma rimane approssimativa; SAIL utilizza LLMs per cercare trasformatori astratti mentre i controlli semantici e sintattici duri preservano la solidità globale. Questi risultati sono la prova che LLMs sono efficaci meccanismi di proposta e di ricerca intorno ai nuclei formali, non la prova che la generazione probabilistica stessa è diventata un ragionamento formale.

Una recente revisione sistematica dello NLP neuro-simbolica sostiene che le architetture federative, in cui i moduli neurali e simbolici rimangono autonome e comunicano attraverso interfacce strutturate, spesso mostrano guadagni più grandi rispetto alle architetture che semplicemente iniettano pregiudizi simbolici nelle reti neurali. I suoi autori avvertono anche che il vantaggio apparente non è ancora conclusivo perché i sistemi federativi e iniettivi sono di solito valutati su diversi tipi di compiti. Questo libro adotta la stessa posizione disciplinata. Il caso architettonico per la separazione è forte perché chiarisce il confine di fiducia, consente la riproduzione indipendente e mantiene la componente simbolica insopzionabile; l'affermazione empirica che una famiglia supera universalmente un altro rimane non dimostrato.

Il libro fornisce anche un caso di studio critico del pubblico NaturalLanguageLinterAgent, o nllAgent, architettura e dei suoi componenti LongTextJS e CircuitJS. L'implementazione separa un programma immutabile sul lato documentale da un programma lato teorico riutilizzabile. Conserva con ancora di origine esatte, tipi di osservazione delle versioni, identità dei produttori, copertura, lacune e stato epistemico. La compilazione del circuito deriva le osservazioni effettivamente richieste da una regola, verifica la pubblicazione se esistono produttori compatibili, test di compatibilità del runtime se il documento concreto ha prodotto osservazioni e copertura sufficienti e un verificatore indipendente controlla se la scoperta di un candidato è

pubblicata. L'apprendimento avviene in uno spazio di lavoro usa e getta e non può essere pubblicato automaticamente. Si tratta di decisioni di governance insolitamente gravi per un sistema di analisi documentale assistita da LLM. Il repository pubblico registra anche limitazioni irrisolte piuttosto che nasconderle. Il suo attuale registro dei problemi gravi identifica l'ecceduralizzazione dello scheduler delimitato, la propagazione incompleta delle alternative di interpretazione attraverso ogni percorso di esecuzione, l'analisi conservativa della compatibilità per il comportamento procedurale opaco, la pianificazione minima delle query e l'indicizzazione temporale, la generazione controllata deliberatamente ristretta e il supporto di mutazione che rimane specifico per scenario. Si tratta di limitazioni di ricerca credibili: riguardano il substrato semantico e di esecuzione, non solo mancando la lucidatura dell'interfaccia utente.

Nessuna valutazione peer-reviewed indipendente o revisione tecnica di terze parti di nllAgent è stata trovata nelle ricerche condotte per questa edizione. Di conseguenza, la valutazione del caso-studio è esplicitamente la revisione architettonica dell'autore, fondata sul archivio pubblico, la documentazione, l'ambito eseguibile e le questioni serie auto-dichiarate. Il giudizio è favorevole sulla separazione delle prove, della teoria, della verifica e della pubblicazione, ma cauti sulla completezza semantica. Le più forti affermazioni di attuazione relative ai controlli strutturali e controllati-inglese limitati. La risoluzione generale dell'identità, la semantica ricca degli eventi, la conoscenza del mondo attuale, l'ampio trasferimento di domini e le rivendicazioni di qualità supportate statisticamente rimangono un lavoro di ricerca piuttosto che capacità consolidata.

La proposta centrale del libro è un runtime di linguaggio naturale eseguibile multi-formalismo. Un LLM propone per la prima volta un piccolo insieme di strutture semantiche candidate, ognuna legata agli span di origine esatti e tipizzata contro un'ontologia selezionata. Un kernel simbolico che mette a nudo solo gli oggetti la cui identità, provenienza, portata, tempo, modalità e prove soddisfano contratti espliciti. Un router seleziona uno o più bersagli formali in base alla domanda: valutazione relazionale o Datalog per la chiusura e le query sui documenti; tabelle decisionali per i criteri; SAT / SMT / CSP per vincoli; automi per modelli ordinati; controllo del modello per il comportamento temporale; esecuzione simbolica per i testimoni del percorso; e assistenti di prova della prova della prova del teorema per affermazioni deduttive dedusse. L'esecuzione formale produce un risultato canonico contenente fatti accettati, candidati respinti, incertezze, testimoni, oggetti di prova, limiti di copertura e ipotesi. Un ultimo LLM può spiegare tale risultato, ma ogni frase nella spiegazione deve essere autorizzata dalla struttura canonica e riconducibile a prove o derivazione verificata.

Il linguaggio naturale eseguibile non dovrebbe pertanto essere pubblicizzato come esecuzione di prosa senza restrizioni. Dovrebbe essere definito come un processo di compilazione, messa a base, esecuzione e realizzazione controllato su uno o più candidati semantici espliciti. La garanzia è condizionata ma utile: data la revisione della fonte dichiarata, l'ontologia, la base, l'insieme di interpretazione, la semantica formale e i limiti di esecuzione, il risultato riportato segue e può essere riprodotto. Il principale problema di ricerca aperta non è il risolutore. È la costruzione, il confronto, il perfezionamento e la convalida delle formalizzazioni su cui opera il solutore.

Prefazione: Ambito, Metodo e Posizione Critica

L'ambizione del linguaggio naturale eseguibile è facile da esprimere e facile da sopravvalutare. Un utente scrive una regola, una politica, un'ipotesi scientifica, una procedura o una questione nel linguaggio ordinario; un sistema lo comprende; il calcolo segue; il risultato viene restituito come linguaggio ordinario. I modelli di linguaggio contemporaneo fanno apparire l'inizio e la fine di questa pipeline. Possono leggere prosa senza restrizioni, generare codice e formule, invocare strumenti e spiegare fluentemente gli output. La pericolosa illusione è che il mezzo è scomparso. Infatti, il mezzo contiene quasi ogni problema difficile: decidere quale significato conta, identificare gli oggetti attraverso i passaggi, distinguere le prove esplicite dalle ipotesi di fondo, preservare modalità ed eccezioni, selezionare un formalismo appropriato, dimostrare che il manufatto generato è ben formato e impedire che la spiegazione finale di dire più dell'esecuzione stabilita.

Un'indagine utile non può essere un catalogo di piccoli sistemi presentati in brevi sottosezioni isolate. Il campo è frammentato proprio perché le stesse parole sono usate per operazioni diverse. "Esecuzione simbolica" può significare l'esplorazione del percorso in stile King rispetto agli input simbolici del programma, l'esecuzione di una forma logica generata su una base di conoscenza o semplicemente l'uso di un interprete esterno. La "verifica formale" può riferirsi alla correttezza di un risultato risolutore, alla correttezza di un programma compilato, alla fedeltà della traduzione in linguaggio naturale o alla robustezza di una rete neurale sotto una famiglia di perturbazioni predefinita. "Grounding" può significare collegare le parole alle colonne del database, alle entità per i canali, alle proposizioni alla conoscenza del mondo o istruzioni del robot agli oggetti percettivi. Un trattamento serio deve mostrare come questi strati si riferiscono e dove le garanzie si fermano.

Il metodo dell'indagine segue quindi meccanismi piuttosto che categorie di marketing. Per ogni linea di lavoro, l'analisi pone cinque domande. Quale artefatto viene prodotto dal linguaggio? Che semantica ha questo artefatto indipendentemente dallo LLM? Come viene fondato il manufatto nella fonte o nell'ambiente? Quale componente determina effettivamente il risultato? Cosa è e non è garantito quando il sistema ha successo? I documenti sono trattati come prova dei loro compiti segnalati, non come prova di un'architettura generale. I risultati numerici forti sono inclusi quando chiariscono il meccanismo, ma non vengono trasferiti a domini con una diversa struttura semantica. L'orizzonte di indagine è il 2 agosto 2026 e comprende letteratura peer-reviewed, atti ufficiali, documenti di ricerca primari e il repository e la documentazione pubblica nllAgent.

Il termine "revisione" è usato con attenzione. Esistono diverse indagini generali, ma ognuna copre solo una parte del problema end-to-end. Il sondaggio controllato-naturale di Kuhn organizza le lingue in base alla precisione, all'espressività, alla naturalezza e alla semplicità [Kuhn 2014]. Hamilton e colleghi valutano se NLP neuro-simbolica ha effettivamente fornito la generalizzazione di ragionamento, trasferimento, interpretabilità, efficienza dei dati e generalizzazione fuori distribuzione, mentre i sondaggi più recenti classificano i sistemi neuro-simbolici orientati ai compiti e basati sui dati e alla conoscenza [Hamilton et al. 2024; Delvecchio et al. 2025; Wang et al. 2025b]. I sondaggi sull'esecuzione simbolica e l'analisi del programma assistita da LLM descrivono i limiti ingegneristici dell'esplorazione del percorso e il ruolo crescente dei componenti appresi [Baldoni et al. 2018; Wang et al. 2025a]. Una revisione dedicata studia LLMs come formalizzatori per la pianificazione automatizzata [Tantakoun et al.

2025] e l'indagine di autoformalizzazione si concentra sulla traduzione della matematica informale in forme selezionabili [Weng et al. 2025]. Una roadmap bidirezionale per l'ingegneria dei requisiti esamina sia l'uso di LLMs per rendere accessibili i metodi formali sia l'uso di metodi formali per vincolare LLM-produce [Ferrari and Spoletini 2025]. Un'indagine mirata su trentacinque articoli mappa la formalizzazione assistita da LLM dei requisiti e delle specifiche del software, incluso il lavoro rivolto a Dafny, C e Java; il suo ambito conferma che il supporto alla traduzione sta crescendo più rapidamente della garanzia semantica end-to-end [Beg et al. 2025]. La revisione sistematica delle frontiere del 2026 sviluppa una nuova tassonomia per NLP neuro-simbolica e sostiene una maggiore attenzione ai sistemi federativi [Chatzikyriakidis and Lappin 2026].

Nessuna revisione esaminata unifica il linguaggio controllato, la base semantica, l'interpretazione astratta, l'esecuzione simbolica, la selezione del formalismo, i risultati portatori di prove e la rigenerazione limitata come un'architettura di fiducia. Questa conclusione è una sintesi della letteratura piuttosto che una rivendicazione di un'inerenza esaustiva: la terminologia differisce tra i campi e i documenti pertinenti sono distribuiti in linguistica computazionale, linguaggi di programmazione, database, metodi formali, ingegneria del software, pianificazione e robotica. Laddove non esista una revisione indipendente, in particolare per nllAgent, il testo etichetta i suoi giudizi come analisi originali piuttosto che implicare l'approvazione esterna.

La posizione del libro è supponeta ma conservatrice. Rifiuta l'idea che un grande modello dovrebbe essere considerato il ragionatore finale solo perché la sua prosa assomiglia a una prova. Rifiuta anche l'idea opposta che solo un parser semantico scritto a mano possa supportare un'utile esecuzione formale. Nei domini aperti, un LLM è spesso il miglior meccanismo disponibile per proporre eventi, relazioni, schemi formali e mappature dei candidati. La questione architettonica è come limitare tale potenza. La risposta più difendibile è un sistema federato con espliciti artefatti intermedi, interfacce digitate e versioni, esecuzione simbolica indipendente, stati sconosciuti conservatori e prove rigiocabili.

Questa posizione ha una conseguenza importante. Lo scopo del sistema non è sempre quello di produrre una formalizzazione definitiva. A volte il manufatto corretto è un insieme di alternative. Un pronome può riferirsi a due possibili attori; una frase legale può ammettere scopi concorrenti; un requisito può essere incompleto; due fonti possono non essere d'accordo; un documento potrebbe non dire se si applica un'eccezione. Un runtime simbolico che sceglie arbitrariamente una interpretazione è meno rigoroso di un LLM che afferma candidamente l'ambiguità. L'esecuzione diventa preziosa quando può calcolare ciò che segue in ogni lettura ammissibile, quanto segue almeno uno, quali conclusioni dipendono da una scelta contestata e quali prove aggiuntive risolverebbero la scelta.

L'architettura proposta tratta quindi l'incertezza strutturalmente. I punteggi di fiducia possono classificare i candidati, ma non trasformano un'interpretazione proposta in un fatto certificato. Lo stato di un elemento registra come è diventato noto: estratto direttamente da un parser deterministico, proposto da un modello, assunto per uno scenario, derivato da una regola, controllato da un verificatore, o confermato da una persona. La copertura registra se un dominio è stato completamente esaminato, campionato, recuperato o non elaborato. Una conclusione negativa è consentita solo quando la ricerca pertinente è chiusa in base a una politica dichiarata. Questa è la differenza tra un sistema simbolico affidabile e un classificatore fluente con un output a forma di JSON.

La discussione rimane accessibile piuttosto che pesantemente. L'interpretazione astratta, l'esecuzione simbolica, il controllo dei modelli e la semantica logica sono spiegati attraverso la loro essenza operativa e gli obblighi che creano. La notazione formale appare solo dove chiarisce una distinzione che la prosa potrebbe confondere. L'obiettivo non è quello di presentare la matematica decorativa. È quello di dare ai progettisti di sistemi, ai revisori e ai ricercatori abbastanza precisione concettuale per decidere dove una garanzia proposta è valida, dove è condizionata e dove è semplicemente aspirazionale.

La tesi risultante può essere dichiarata chiaramente. LLMs dovrebbe scoprire forme formali plausibili; componenti deterministici e formalmente vincolati dovrebbero controllare la fondazione, la compilazione, l'esecuzione e l'accettazione; LLMs può quindi realizzare il risultato canonico come linguaggio, ma solo in base a un contratto che impedisce aggiunte non supportate. Non è una soluzione completa al significato. È un percorso credibile dall'elaborazione del linguaggio probabilistico a un ragionamento limitato, eseguibile e verificabile.

1. Dal Linguaggio Fluente alla Semantica Esecuibile

1.1 Che cosa Richiede l'Esecuzione e Dove Appartiene il Confine di Fiducia

La parola “esecuzione” porta un impegno più forte di “interpretazione”, “risposta” o “ragione”. Un programma è eseguibile perché le sue operazioni hanno un effetto definito su uno stato. Una query di database è eseguibile perché gli operatori relazionali determinano quali tuple sono selezionate, unite, raggruppate o escluse. Un problema di pianificazione è eseguibile perché le azioni hanno precondizioni ed effetti, e un pianificatore cerca una sequenza che raggiunge un obiettivo. Una teoria logica è eseguibile in un senso più ampio perché una procedura di inferenza determina se una conclusione segue, se esiste un modello o se i vincoli sono soddisfatti. In ogni caso, l'operazione decisiva è esterna alla retorica usata per descrivere il compito. Un risultato può essere riprodotto da un manufatto e semantica.

Una risposta LLM è diversa. Il modello mappa un contesto per una distribuzione rispetto alle continuazioni. Può interiorizzare modelli che assomigliano ad algoritmi e possono risolvere correttamente molte attività, ma la regola applicata in un particolare passaggio non è ispezionabile separatamente o garantita per essere utilizzata in modo coerente. La catena di pensiero che richiede può migliorare le prestazioni, ma una catena di frasi non è un oggetto di prova solo perché è organizzata come passi. L'autocoerenza campiona diverse continuazioni, non diverse esecuzioni formalmente definite. L'uso dello strumento può rendere un valore esatto lasciando l'interpretazione che ha prodotto la chiamata dello strumento non verificata. La distinzione cruciale è tra un modello che propone un artefatto eseguibile e un modello la cui generazione è trattata come esecuzione.

Il linguaggio naturale eseguibile inizia quindi con la materializzazione. Il sistema deve creare un oggetto con una grammatica dichiarata, un sistema di tipo, una semantica e una mappa di origine. Quell'oggetto potrebbe essere una query relazionale, una teoria logica, una macchina dello stato, un insieme di vincoli temporali, una tabella delle decisioni, un programma Datalog, un automa finito o un grafico digitato con gli operatori registrati. La scelta particolare dipende dal compito. Ciò che conta è che il comportamento dell'oggetto non sia più determinato dalla distribuzione next-token del modello linguistico. Può essere rifiutato, eseguito, testato, confrontato e rigiocato in modo indipendente.

Questo requisito è più forte dell'output strutturato ordinario. Un modello può emettere JSON valido sintatticamente che contiene entità inventate o campi reciprocamente incoerenti. Uno schema convalida la forma, non la verità. Allo stesso modo, un modello può generare SQL legale che si unisce alle colonne sbagliate, una logica valida di primo ordine che inverte un quantificatore o una formula temporale che esprime la relazione di precedenza opposta. Un utile confine di fiducia non è quindi semplicemente “dopo la generazione JSON”. Si trova dopo la fondazione delle fonti, il controllo dei tipi, la compatibilità semantica e la compilazione formale. Ogni simbolo che riguarda la risposta deve avere un'origine ammissibile e un ruolo definito.

L'architettura separa naturalmente quattro autorità. L'autorità di origine determina quali testi, dati o osservazioni sono stati effettivamente forniti. L'autorità semantica determina quali tipi, relazioni, modalità e politiche identitarie sono consentiti. L'autorità della regola determina quali

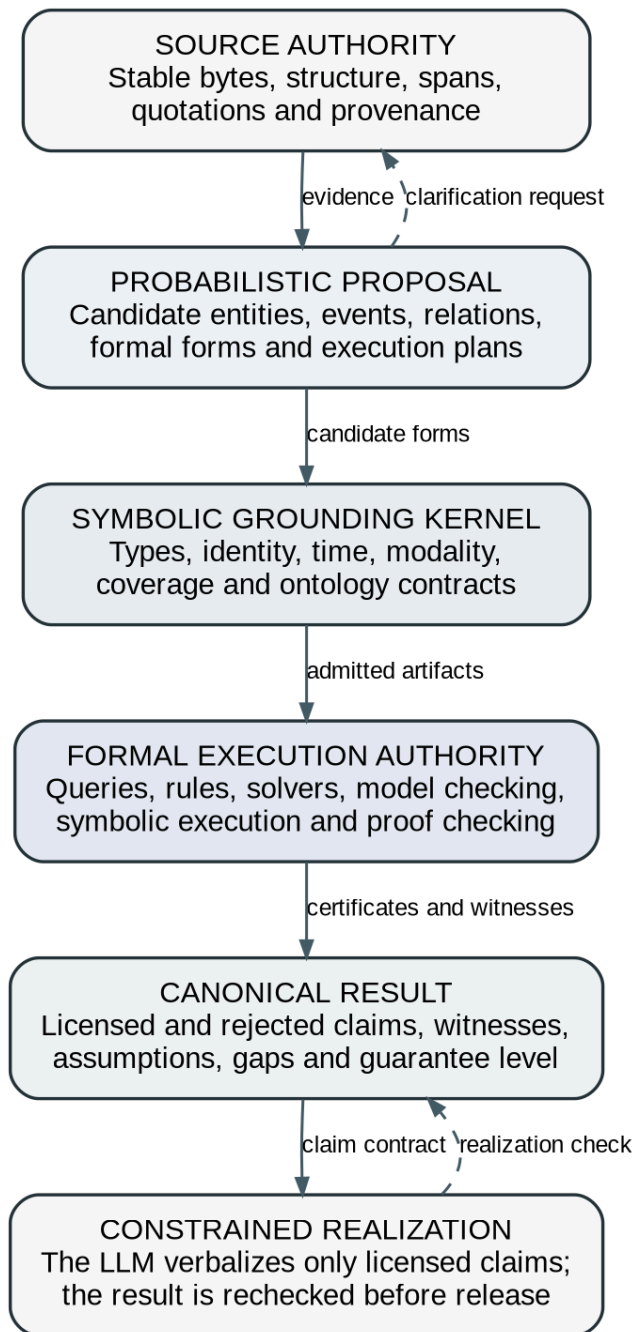
vincoli o trasformazioni costituiscono la teoria che viene eseguita. L'autorità di esecuzione determina se un risultato candidato segue sotto questa teoria. Un LLM può aiutare tutti e quattro, ma non dovrebbe fonderli silenziosamente. Quando lo stesso modello inventa una premessa, seleziona un'ontologia, applica una regola e scrive il verdetto, non esiste una separazione significativa tra l'evidenza e la conclusione.

L'autorità della fonte inizia con byte stabili e intervalli. Un sistema di analisi dei documenti dovrebbe preservare l'esatta revisione, la struttura e l'ubicazione di ogni frammento citato. Dovrebbe distinguere una citazione da una sintesi e una sintesi da un'inferenza. Quando un modello propone che una frase esprima un obbligo, l'osservazione deve indicare la frase e mantenere il modello, la sollecitazione, lo schema e le alternative che hanno prodotto la classificazione. Se la citazione non può essere trovata nella fonte, l'osservazione è respinta piuttosto che accettata come una ricostruzione plausibile. Sembra un'ingegneria di provenienza mondana, ma è una preconditione per ogni garanzia successiva.

L'autorità semantica è di solito distribuita in un'ontologia superiore e pacchetti di dominio. Un'ontologia universale è attraente perché promette l'interoperabilità, ma tende a diventare troppo astratta per decidere questioni utili o troppo grandi e controverse da governare. Un design migliore definisce un piccolo livello comune per intervalli di sorgenti, menzioni, entità, eventi, tempo, mondi, stato, provenienza, alternative, copertura e lacune. I pacchetti di dominio definiscono quindi concetti come l'intervento clinico, l'obbligo normativo, l'evento di oggetti narrativi, la chiamata API del software o la transazione finanziaria. Ogni pacchetto deve indicare non solo i nomi dei campi, ma anche gli impegni semantici e i test. Un tipo chiamato "approvazione" non è significativo a meno che il sistema non specifichi se denota un documento, un atto, uno stato, una decisione di autorità o una combinazione.

L'autorità dell'autore della regola dovrebbe essere versionata separatamente dall'interpretazione dei documenti. Una politica, una teoria linterca o un modello scientifico cambia a un ritmo diverso dalla fonte che analizza. Se le regole sono incorporate in modo invisibile in modo intuitivo, diventa difficile sapere se due esecuzioni hanno applicato la stessa teoria. Gli artefatti delle regole Versioned consentono la pubblicazione di riferimento, i diff semantici e il rollback. Permettono anche più teorie sulla stessa rappresentazione del documento. Un manoscritto può essere controllato per la coerenza terminologica, la coerenza causale e il supporto della citazione senza ricompilarlo in un diverso impulso opaco per ogni regola.

L'autorità di esecuzione dovrebbe essere piccola e meccanica quanto pratica. Per le query relazionali, può essere un valutatore deterministico. Per vincoli, un solutore SMT o CSP. Per il comportamento temporale, un modello checker. Per le proprietà del programma per percorso, un esecutore simbolico. Per gli obblighi di prova, un prover del teorema più un piccolo controllo a prova. Per un linguaggio di circuito personalizzato, può essere un interprete limitato con operatori registrati e verificatori indipendenti. Il componente di accettazione non dovrebbe chiedere a un LLM se il risultato "sembra corretto". Dovrebbe controllare un testimone contro la fonte e la teoria.



La figura 1. Confidamento di fiducia per il linguaggio naturale eseguibile. Lo LLM propone forme semantiche, mentre la base della fonte, l'esecuzione formale, l'accettazione e la licenza di reclamo finale rimangono verificabili in modo indipendente.

Un modo utile per classificare i sistemi è la posizione dell'oracolo finale. In un sistema puramente neurale, lo LLM è l'oracolo. In un sistema avanzato di strumenti, uno strumento calcola un valore, ma lo LLM può rimanere l'oracolo per se il richiamo dello strumento rappresenta la domanda. In un sistema sovrruldatore, il risolutore è l'oracolo per le conseguenze di una formalizzazione generata. In un sistema basato su evidenza, il solutore e il kernel di fondazione determinano congiuntamente il risultato accettato. In un sistema di correzione di prove, anche l'output del solutore può essere controllato da un kernel più piccolo. Ogni passo restringe lo spazio in cui l'errore probabilistico può cambiare silenziosamente il verdetto.

Ciò non implica che ogni applicazione necessiti di una garanzia a livello di teorema. Un assistente di scrittura può utilizzare circuiti deterministici leggeri, mentre un sistema di regolamentazione o di sicurezza può richiedere artefatti a prova formale. Il principio importante è che la garanzia corrisponda al meccanismo. Un controllo deterministico regex può certificare la presenza di un modello letterale, non l'assenza di un concetto. Un estrattore di eventi assistiti dal modello può supportare un suggerimento di avvertimento di continuità, non un resoconto meccanico di identità narrativa. Un risolutore può stabilire insoddisfabilità dei vincoli, non la completezza della traduzione in linguaggio naturale. Il rigore consiste in gran parte nel rifiutare di trasferire fiducia oltre questi confini.

L'architettura più forte è quindi federativa. La componente neurale non viene dissolta in una rete simbolica e la componente simbolica non si riduce a un regolarizzatore differenziabile. Rimangono sistemi separati con un'interfaccia digitata. Lo LLM è responsabile della ampiezza, della generazione di ipotesi, della normalizzazione semantica e del recupero da un linguaggio insolito. Il nucleo formale è responsabile per l'ammissibilità, l'esecuzione, gli invarianti, i testimoni e la riproducibilità. La revisione del 2026 da Chatzikyriakidis e Lappin chiama i sistemi correlati federativi e osserva che spesso mostrano forti guadagni su compiti in cui la percezione può essere separata dall'inferenza. Gli autori sottolineano anche che i confronti rimangono confusi dalle differenze di attività. L'argomento architettonico per la federazione è tuttavia indipendente dalla superiorità del benchmark: i moduli separati rendono le affermazioni verificabili e consentono a un componente di rifiutare un altro [Chatzikyriakidis and Lappin 2026].

L'output finale del linguaggio naturale richiede un proprio limite. Una volta che un solverante restituisce un risultato, è allettante fornire la traccia a un LLM e richiedere una risposta lucida. Senza vincoli, il modello può aggiungere spiegazioni di fondo, dedurre motivazioni o semplificare l'incertezza in certezza. Un risultato canonico dovrebbe quindi enumerare le affermazioni che possono essere verbalizzate, il loro status, le prove, le ipotesi e la portata. Il renderizzatore può scegliere la formulazione e l'organizzazione, ma dovrebbe essere confrontato con la struttura canonica. Un metodo pratico è la realizzazione a livello di affermazione: il modello genera frasi associate agli identificatori di risultato; un verificatore garantisce che ogni frase sia implicata o parafrasa un reclamo consentito; il materiale non supportato viene rimosso o etichettato come commento. Nelle impostazioni ad alta garanzia, il linguaggio controllato con modelli può essere preferibile alla prosa senza restrizioni.

Il confine di fiducia proposto in questo libro può essere riassunto senza codice. LLMs può proporre forme semantiche, mappature onlogiche, scelte di formalismo, riparazioni e spiegazioni. Non stabiliscono prove, chiudono una ricerca open-world, certificano l'identità, decidono la soddisfacibilità o autorizzano un reclamo finale. Queste funzioni appartengono a contratti simbolici espliciti. Questa separazione preserva il valore unico dei modelli linguistici, impedendo alla loro fluidità di essere scambiati per prova.

1.2 Il Divario di Formalizzazione: Perché un Risolutore Corretto Può Ancora Produrre la Risposta Sbagliata

La debolezza centrale della maggior parte dei sistemi di ragionamento neuro-simbolici non è il motore di inferenza. I solutori SAT, i solutori SMT, i motori di database, i verificatori del teorema, i controllori dei modelli e gli esecutori simbolici sono spesso molto più affidabili del modello linguistico che li precede. La debolezza sta nel divario di formalizzazione: la differenza tra ciò che la fonte significa nel contesto e l'oggetto formale presentato per l'esecuzione. Un sistema può essere perfettamente suonato da input formali a output formali mentre è malsano dal linguaggio naturale alla risposta.

Si consideri una semplice frase politica: “Un revisore può approvare un comunicato solo dopo che il rapporto di sicurezza è completo, a meno che il comandante dell'incidente non autorizzi un'eccezione di emergenza”. Un formalizzatore deve preservare almeno l'attore, l'azione, l'oggetto, la condizione temporale, lo stato di completamento, l'eccezione, l'autorità e la portata di “solo dopo”. Deve distinguere l'autorizzazione dall'obbligo. Deve decidere se l'eccezione rimuove solo la condizione temporale o autorizza l'approvazione in generale. Deve collegare “il rapporto di sicurezza” al manufatto corretto e al “comandante dell'incidente” a un'identità o un ruolo. Se una di queste scelte è errata, un risolutore può applicare correttamente la politica sbagliata.

L'ambito di applicazione di Quantifier è una classica fonte di errore. “Ogni ricercatore ha esaminato un articolo” può significare che ogni ricercatore ha esaminato almeno un documento, non necessariamente lo stesso. “Un articolo è stato rivisto da ogni ricercatore” ha una struttura diversa. La negazione crea problemi simili: “non tutti i campioni passati” non significa che nessun campione sia passato. Condizionati, comparazioni, modalità, relazioni temporali, anafora e presupposto creano tutti distinzioni che la traduzione a livello di superficie può cancellare. Le tradizioni semantiche formali hanno sviluppato sofisticati meccanismi per questi fenomeni, ma il linguaggio open-domain aggiunge la conoscenza del mondo, le convenzioni di genere e gli obiettivi impliciti che nessuna singola rappresentazione cattura completamente.

Il divario appare anche nell'analisi semantica dalle denotazioni. Un sistema addestrato solo su una domanda e una risposta finale possono trovare un programma che si imbatte in grado di produrre la risposta corretta sull'ambiente di formazione. Tali programmi spuri sono un problema noto nell'analisi semantica debolmente supervisionata [Pasupat and Liang 2016; Guu et al. 2017; Goldman et al. 2018]. L'esecuzione non distingue il programma previsto da uno accidentale se entrambi producono la stessa denotazione. I sistemi Text-to-SQL affrontano lo stesso rischio: una query può restituire le righe giuste a causa dei valori di dati correnti mentre contengono l'unità o il filtro errati. La decodifica guidata dall'esecuzione rimuove i programmi che bloccano o contraddicono i vincoli intermedi, ma non può dimostrare la fedeltà semantica [Wang et al. 2018].

Il divario di formalizzazione è meglio scomposta in obblighi separati. La radicazione lessicale chiede quali predicati o campi corrispondono a parole e frasi. La fondazione referenziale chiede quali menzioni denotano la stessa entità. La base strutturale chiede quali argomenti, ambiti e dipendenze appartengono insieme. La fondazione temporale collega frasi a istanti, intervalli, calendari e ordinazione. La fondazione moda distingue fatti, possibilità, obblighi, autorizzazioni, intenzioni, reclami segnalati e controfattuale. La fondazione ontologica determina se un concetto appartiene al modello di dominio selezionato. La fondazione evidenzia collega ogni

premessa a una lunghezza di sorgente, a un sensore, a una riga di database o a un'ipotesi dichiarata. Ogni obbligo può fallire in modo indipendente.

Il sistema deve anche decidere quanto significato formalizzare. Una rappresentazione semantica totale della prosa senza restrizioni non è né pratica né necessaria per la maggior parte dei compiti. La regola che viene eseguita crea la domanda. Un circuito terminologico può richiedere solo intervalli esatti e termini normalizzati. Una regola di conformità temporale può richiedere eventi, attori, date ed eccezioni. Un'analisi causale può richiedere affermazioni, interventi, esiti e incertezza. La formalizzazione basata sulla domanda riduce i costi e limita il numero di impegni non supportati. Crea anche un problema di copertura esplicita: se una regola richiede un concetto che i produttori disponibili non possono materializzarsi, la corsa deve fermarsi o tornare sconosciuta piuttosto che silenziosamente approssimare.

L'ambiguità dovrebbe essere rappresentata come alternative, non immediatamente crollata. Supponiamo che "Alex abbia detto a Jordan che dovrebbero andarsene" appare in una procedura. Il pronome può riferirsi ad Alex, Jordan o a un gruppo, a seconda del contesto e delle convenzioni linguistiche. Un modello può classificare i candidati, ma un'elevata fiducia non è una prova. Se il verdetto della regola a valle è lo stesso sotto tutti i legami plausibili, l'ambiguità è irrilevante e l'esecuzione può procedere. Se il verdetto cambia, il risultato dovrebbe esporre la dipendenza e richiedere chiarimenti o un contesto aggiuntivo. Questo è un luogo in cui l'interpretazione astratta diventa concettualmente utile: può calcolare le conclusioni su un insieme di interpretazioni senza enumerare ogni combinazione in dettaglio.

Una mappatura completa è spesso impossibile perché il linguaggio naturale è open-world. Un documento può omettere fatti che sono veri, assumere conoscenze comuni o fare affidamento sulla pratica istituzionale. Il ragionamento del mondo chiuso tratta i fatti non registrati come falsi; il ragionamento del mondo aperto li tratta come sconosciuti. Entrambe le politiche sono appropriate in contesti diversi. Un inventario di database può essere chiuso per una tabella e un tempo specificati. Una raccolta di e-mail recuperate non è generalmente la prova completa che non è stata inviata alcuna notifica. Lo strato di fondazione deve quindi collegare la copertura alle osservazioni e alle ricerche. Le conclusioni negative richiedono un token di copertura che indichi il dominio, la revisione della fonte, l'ambito, il metodo, le esclusioni e la politica di completezza.

La conoscenza esterna introduce ulteriori complicazioni. Un LLM generale contiene informazioni estese ma non modificate e potenzialmente stantie. Un sistema simbolico non può trattare la memoria modello come un fatto senza tempo. La conoscenza del mondo dovrebbe entrare attraverso pacchetti di provenienza, datati, specifici della giurisdizione e indirizzati ai contenuti. Un brano può affermare che una regola legale era efficace in una particolare giurisdizione durante un particolare intervallo o che una costante scientifica è stata prelevata da una fonte nominata. I conflitti tra i pacchetti dovrebbero rimanere visibili. I mondi fittizi o ipotetici devono essere selezionabili senza contaminazione da fatti del mondo attuale. La documentazione nllAgent tratta correttamente tali pacchetti come un confine piuttosto che fingere che la memoria di un modello di base fornisca un'autorità formale.

La fedeltà della formalizzazione può essere messa alla prova, anche se non universalmente provata. Un metodo è il controllo bidirezionale: tradurre l'oggetto formale in un linguaggio controllato e confrontarlo con la fonte. Un altro è il test contrastivo: generare minime modifiche testuali che dovrebbero cambiare un elemento formale e preservare il resto. Un terzo è un confronto estensionale: chiedere formalizzazioni concorrenti per generare esempi distintivi o

risultati di query. Un quarto è la stabilità parafrasi: le formulazioni semanticamente equivalenti dovrebbero essere compilate a manufatti equivalenti o dimostrabilmente correlati. Un quinto è una spiegazione controesempio: quando il risolutore produce un testimone, mappa ogni dipendenza per la fonte si estende e chiede se lo scenario sia linguisticamente ammissibile. Un sesto è la revisione umana focalizzata sulla piccola serie di scelte che influenzano il risultato piuttosto che l'intero programma formale.

Sistemi interattivi come nl2spec illustrano il valore della riparazione locale. Invece di chiedere all'utente di comprendere o riscrivere un'intera formula temporale, l'interfaccia associa le sottoforme a frammenti di origine, consentendo di correggere un componente sbagliato [Cosler et al. 2023]. Il feedback del rilievo può svolgere un ruolo simile. Logic-LM utilizza errori di sintassi e di esecuzione per richiedere la riformalizzazione [Pan et al. 2023]. I sistemi più avanzati possono utilizzare core o controesempi non soddisfabili piuttosto che messaggi di errore generici. L'intuizione principale è che il feedback formale dovrebbe restringere la questione linguistica.

Il divario di formalizzazione pone anche dei limiti alle affermazioni di "sonità" end-to-end. Un sistema può provare un teorema della forma: se ogni osservazione accettata rappresenta accuratamente la sua fonte e l'ontologia e le regole sono corrette, allora il verdetto segue. E' una cosa preziosa. Non dovrebbe essere riscritto come "il verdetto è dimostrato dal documento" a meno che non siano stati espletati anche gli obblighi di osservazione e di fondazione. La garanzia dovrebbe nominare le sue ipotesi e le sue incognite. Un involucro di solidità è più onesto di un'etichetta di fiducia binaria.

Questo libro tratta quindi la formalizzazione come un processo di compilazione con più artefatti intermedi piuttosto che un singolo prompt. La sorgente è segmentata e ancorata. Le osservazioni dei candidati sono prodotte in schemi. Sono rappresentate le alternative di identità e tempo. La compatibilità dell'ontologia è verificata. Viene selezionato un formalismo target. Il compilatore produce un oggetto eseguibile e una mappa sorgente. L'esecutore produce un testimone, una prova, un modello o un fallimento. Il risultato viene rigiocato e reso. In ogni fase, un errore ha una categoria specifica e una possibile riparazione. L'architettura non elimina l'incertezza semantica; impedisce l'incertezza di essere nascosta all'interno dell'output fluente.

Segue la più importante conclusione della ricerca. I sistemi LLM supportati dal sistema di semplificazione del riso sono valutati separatamente sulla fedeltà e sull'esecuzione formale. Un alto punteggio finale-risposta può nascondere programmi spuri. Un punteggio basso può essere causato da una buona formalizzazione con un risolutore inadeguato, o da una cattiva formalizzazione con un perfetto solutore. I benchmark hanno bisogno di oro o strutture formali ammissibili, allineamenti di prova e coppie minime semantiche, non solo risposte. Fino a quando questa separazione non diventerà standard, molte affermazioni sul "ragionamento formale con LLMs" rimarranno difficili da interpretare.

2. La Genealogia Intellettuale del Linguaggio Eseguitibile

2.1 Linguaggi Controllati, Semantica Formale, Discorso e Logica Naturale

L'idea che il linguaggio leggibile dall'uomo possa supportare l'inferenza meccanica precede di decenni i grandi modelli linguistici. Il lavoro precedente si è avvicinato al problema dalla direzione opposta: piuttosto che chiedere a un modello statistico di interpretare la prosa senza restrizioni, i ricercatori hanno progettato linguaggi o teorie semantiche che limitavano abbastanza ambiguità da supportare la traduzione in logica. Queste tradizioni sono essenziali perché rivelano ciò che un sistema in lingua eseguibile deve preservare anche quando un LLM esegue l'analisi front-end.

I linguaggi naturali controllati sono il precedente più chiaro. Limitano il vocabolario, la grammatica o l'interpretazione mantenendo una superficie che assomiglia al linguaggio ordinario. Attempto Controlled English è l'esempio più noto. Le frasi si traducono in strutture di rappresentazione del discorso e quindi in logica formale, consentendo query e inferenza pur rimanendo leggibili ai non-logicisti [Fuchs and Schwitter 1996; Fuchs et al. 2008]. I linguaggi controllati sono stati utilizzati per la rappresentazione della conoscenza, le specifiche tecniche, le wiki semantiche, le regole di business e i requisiti. L'indagine di Kuhn mostra che la famiglia è diversa: alcuni linguaggi sono progettati per la chiarezza umana piuttosto che per la precisione formale, mentre altri commettono la naturalezza per la semantica deterministica [Kuhn 2014].

La lezione duratura non è che la lingua senza restrizioni debba essere sostituita da un dialetto controllato. È che un sistema eseguibile ha bisogno di un livello di normalizzazione la cui semantica è più rigorosa della prosa ordinaria. Un LLM può servire come traduttore dalla lingua aperta in una forma semantica controllata, ma la forma controllata deve avere una grammatica e un'interpretazione indipendenti. Senza quel confine, il "linguaggio naturale controllato" diventa semplicemente uno stile di scrittura preferito e non fornisce alcuna garanzia computazionale.

Le lingue controllate espongono anche l'importanza del feedback degli autori. Se una frase è al di fuori del frammento controllato, un sistema tradizionale la rifiuta o produce un'implogità esplicita. Le moderne interfacce LLM tendono a fare il contrario: selezionano una lettura plausibile e procedono. Il rifiuto è spesso trattato come un difetto di usabilità, ma in un sistema ad alta assicurazione è una forma di integrità epistemica. Un'interfaccia in lingua eseguibile di prossima generazione dovrebbe combinare la flessibilità della parafrasi LLM con l'onestà di un compilatore in linguaggio controllato. Può proporre diverse letture normalizzate, spiegare le distinzioni e chiedere solo quando la distinzione cambia il risultato.

La semantica formale fornisce un lignaggio teorico più profondo. La semantica del modello-teorico rappresenta un significato in termini di condizioni in cui le espressioni sono vere in un modello. La compositività collega l'interpretazione di un'espressione complessa ai significati delle sue parti e alla loro combinazione sintattica. La semantica in stile Montague, calcoli lambda digitati, quantificatori generalizzati e framework correlati offrono resoconti precisi di portata, struttura dell'argomento e operatori logici. Dimostrano che il significato del linguaggio naturale non è riducibile a una lista piatta di triple. Quantifiers, negazione, verbi intensivi,

modalità e contesto possono cambiare il comportamento logico di frasi apparentemente semplici.

Per il linguaggio naturale eseguibile, la semantica formale contribuisce a due requisiti. In primo luogo, la rappresentanza intermedia deve preservare la struttura dell'operatore piuttosto che solo le entità e le relazioni. “Tutti”, “alcuni”, “non”, “solo”, “a meno che non si tratti”, “dare” e “prima” non siano annotazioni intorno a una proposizione; determinano lo spazio dei modelli. In secondo luogo, la costruzione semantica dovrebbe essere consapevole. Un intervallo temporale, una persona, un'organizzazione, una proposizione e un obbligo non possono essere liberamente sostituiti solo perché le loro etichette superficiali si assomigliano. Il controllo dei tipi è uno dei modi più economici per rifiutare le formalizzazioni incoerenti prima di invocare un risolutore.

La teoria della rappresentazione del discorso e i relativi approcci dinamici-semantici affrontano i fenomeni che la logica frase per frase è scarsa. Un discorso introduce riferimenti, li porta attraverso frasi, aggiorna il contesto e crea vincoli di accessibilità per pronomi e descrizioni. I parser di Boxer e DRS hanno reso operativi queste idee per l'analisi semantica ad ampio raggio [Bos 2008; Liu et al. 2018]. Le attività di analisi di Parallel Meaning e le attività di analisi condivise di DRS hanno permesso di valutare le rappresentazioni di significato tra le lingue [Abzianidze et al. 2019]. Questi sforzi mostrano sia il progresso che la difficoltà: il legame variabile, l'assegnazione dei sensi, la portata e le relazioni di discorso sono sostanzialmente più difficili dell'estrazione superficiale.

Un sistema eseguibile a livello di documento dovrebbe imparare dalla semantica dinamica senza richiedere uno DRS monolitico per ogni fonte. Ha bisogno di oggetti discorsistici persistenti, ipotesi di entità, eventi, mondi e relazioni di aggiornamento. Deve distinguere una menzione da un'entità e un collegamento di identità candidato da uno stabilito. Deve conservare i contesti citati, ipotetici, negati e fittizi. Un grafico di conoscenza piatta che unisce le stringhe uguali in un nodo commettono errori sistematici in documenti lunghi. I nomi si ripetono, si verificano alias, i ruoli cambiano e i documenti descrivono mondi reciprocamente incoerenti.

Il costo pratico dell'analisi semantica formale completa spiega perché molti sistemi si sono spostati verso rappresentazioni meno profonde. L'etichettatura del ruolo semantico identifica la struttura predicato-argomentazione. La rappresentazione di significato astratta rappresenta concetti e relazioni in un grafico. L'estrazione di informazioni identifica entità, eventi e relazioni. Queste rappresentazioni sono utili, ma la loro semantica è spesso incompleta per l'esecuzione. Possono omettere la portata del quantificatore, la modalità, l'ancoraggio temporale e la differenza tra il contenuto affermato e quello riportato. Un runtime in lingua eseguibile dovrebbe trattare tali strutture come osservazioni portanti di prova piuttosto che come una forma logica completa. Un grafico può supportare molte regole locali lasciando espliciti aspetti irrilevanti.

La logica naturale offre un altro importante compromesso. Invece di tradurre le frasi in un linguaggio formale senza restrizioni, i sistemi naturali-logici ragionano attraverso trasformazioni strutturate vicino alla forma superficiale. MacCartney e Manning svilupparono relazioni di inferenza basate sulla monotonia, l'attrattiva lessicale e la produttività [MacCartney and Manning 2007, 2009]. NaturalLI in seguito ha utilizzato operatori-logici naturali per inferenza efficiente su testo [Angeli and Manning 2014]. LangPro combina una grammatica formale, una conoscenza lessicale e un teorema che prova per l'inferenza in lingua naturale [Abzianidze 2017].

La logica naturale è attraente perché molte questioni documentali riguardano relazioni di coinvolgimento che possono essere catturate senza costruire un grande modello mondiale. Sostituire il “cane” con “animale” è valido in un contesto di esclusione verso l’alto, ma non necessariamente sotto negazione o restrizioni universali. Questi calcoli sono più trasparenti di un punteggio di similarità incorporante. Possono sostenere i controlli di contraddizione, il ragionamento tassonomico e la deduzione controllata. Tuttavia, la logica naturale non è un esecutore universale. Gestisce bene certe monotocanità e relazioni lessicali, ma non esprime naturalmente procedure arbitrarie, aritmetiche, pianificazione o comportamenti temporali complessi. In un’architettura multiformalista, dovrebbe essere un motore tra i tanti.

Le tradizioni semantiche chiariscono anche perché la “fondazione” ha diversi livelli. La semantica formale può specificare come un’espressione contribuisce alle condizioni della verità, ma non determina automaticamente quale oggetto del mondo reale un nome denota o se una fonte è credibile. DRS può rappresentare un referente del discorso, ma la risoluzione dell’identità rimane un compito empirico. La logica naturale può propagare l’attraversamento lessicale, ma la relazione lessicale può essere dipendente dal dominio. L’esecuzione richiede quindi una partnership tra la struttura linguistica, l’ontologia del dominio e la gestione delle prove.

Un’architettura utile può distinguere almeno tre prodotti semantici. Il primo è un modello sorgente che contiene struttura esatta, intervalli e citazioni. Il secondo è un livello di interpretazione contenente menzioni, eventi, relazioni, modalità e alternative. La terza è una teoria eseguibile che utilizza interpretazioni selezionate per rispondere a una particolare domanda. Le tecniche semantiche formali informano principalmente il secondo livello. I linguaggi controllati e i prover del teorema collegano il secondo e il terzo. Provenienza e copertura si collegano entrambi al primo.

La letteratura precedente mette anche in guardia dal supporre che una rappresentazione possa servire a ogni compito. DRS è espressivo per il discorso, ma costoso da produrre e ragionare. La logica di primo ordine è ben compresa ma imbarazzante per i fenomeni temporali, probabilistici o defeasibili. Le logiche offrono un ragionamento di ontologia decidabile ma limita l’espressività. La logica naturale rimane vicina al linguaggio ma è specializzata. L’inglese controllato migliora la prevedibilità limitando l’input. La conclusione corretta non è quella di scegliere un vincitore, ma di conservare abbastanza informazioni semantiche per ridurre un compito nel formalismo che si adatta ad esso.

LLMs cambia l’economia di questo disegno. Possono generare analisi dei candidati, riscrivere un linguaggio incontrollato in forme controllate, suggerire mappature di ontologia e spiegare alternative formali agli utenti. Rendono pratico per tentare l’interpretazione semantica su documenti che le grammatiche scritte a mano scaricherebbero. Non eliminano la necessità delle distinzioni semantiche sviluppate dal lavoro precedente. Al contrario, tali distinzioni diventano i contratti con cui vengono giudicate le proposte LLM.

Un errore ricorrente nei sistemi contemporanei è quello di trattare un triplo negozio generato da LLM come una rappresentazione semantica completa. Le triple sono convenienti per il recupero e le query grafiche, ma non codificano da sole la portata, la provenienza, la modalità, il tempo, la negazione, le alternative di identità o la copertura. Uno strato semantico eseguibile dovrebbe essere più ricco ma ancora finito e operativo. Può memorizzare un’affermazione come un oggetto digitato con partecipanti, contesto temporale, polarità, modalità, mondo, prove, produttore e status. Può rappresentare le obbligazioni alternative senza sceglierne uno. Può

dichiarare quali dimensioni sono assenti. Questo è meno elegante di una singola forma logica, ma è più fedele al modo in cui i sistemi reali acquisiscono significato.

Il contributo più profondo del lignaggio pre-LLM è quindi la disciplina concettuale. La leggibilità umana non implica determinazioni semantiche. La determinazione formale è ottenuta sottotenendo la restrizione, la digitazione, il contesto esplicito e il rifiuto delle costruzioni non supportate. Il significato del documento è dinamico e ambito. L'inferenza vicino al linguaggio superficiale è possibile ma specializzata. Molteplici rappresentazioni formali sono legittime perché rispondono a domande diverse. Queste lezioni definiscono l'architettura che un compilatore basato su LLM deve recuperare piuttosto che bypassare.

2.2 Parsing Semantico, Query Eseguibili, Induzione di Programmi e Primi Sistemi Neuro-Simbolici

L'analisi semantica converte le espressioni del linguaggio naturale in oggetti formali che possono essere eseguiti con un database, una base di conoscenza, un ambiente o un interprete. È il predecessore storico più vicino al gasdotto in lingua eseguibile proposto qui. Il campo include la risposta delle domande su tabelle e grafici della conoscenza, text-to-SQL, istruzioni successive, sintesi di espressione regolare, interpretazione del comando robotico e generazione del programma. I suoi successi dimostrano il potere dell'esecuzione esterna; i suoi fallimenti rivelano perché l'esecuzione da sola non stabilisca la comprensione.

I primi parser semantici si affidavano a grammatiche, lessici e forme logiche supervisionate. Potevano produrre rappresentazioni trasparenti, ma richiedevano costose annotazioni e spesso trasferite male attraverso i domini. La supervisione debole ha ridotto l'onere di annotazione allenandosi dalle denotazioni: una domanda è abbinata a una risposta e il sistema cerca un programma che lo produce. Questo approccio ha permesso set di dati più ampi, ma ha creato il problema del programma spurio. Molti programmi possono restituire accidentalmente la stessa risposta, specialmente nelle piccole tabelle o basi di conoscenza. Il segnale di allenamento premia quindi il comportamento senza identificare la semantica prevista [Pasupat and Liang 2016].

Il problema è strutturale, non solo statistico. Supponiamo che la risposta a “Quale città abbia ospitato l'evento 2012?” è Londra. È previsto un programma che seleziona la riga con l'anno 2012. Un programma che seleziona la prima fila può anche tornare Londra nel tavolo di formazione. Ulteriori dati possono ridurre le correlazioni accidentali, ma non possono garantire che il programma appreso rifletta la lingua. Esempi astratti, modelli di allineamento, vincoli di tipo e filtraggio basato sull'esecuzione sono stati sviluppati per sopprimere i candidati spuri [Goldman et al. 2018; Lee et al. 2023]. Queste tecniche sono importanti anche per la formalizzazione LLM. Un risultato del solutore deve essere trattato come un test di una formalizzazione, non come certificato semantico.

Le macchine simboliche neurali combinano la generazione di programmi neurali con un interprete simbolico Lisp [Liang et al. 2017]. La componente neurale propone programmi, mentre l'esecutore rifiuta i candidati non validi e restituisce le denotazioni. Mou e colleghi hanno distribuito esplicitamente l'esecuzione neurale con l'esecuzione simbolica per le query in linguaggio naturale, riportando miglioramenti in termini di accuratezza, efficienza e interpretabilità [Mou et al. 2017]. Questi sistemi hanno stabilito un modello durevole: i modelli neurali gestiscono la variabilità del linguaggio; gli interpreti simbolici rafforzano il significato operativo.

Text-to-SQL ha reso il modello commercialmente importante. Una domanda dell'utente viene compilata in una query la cui esecuzione restituisce i dati. La decodifica guidata dall'esecuzione può eseguire candidati SQL parziali o completi e scartare quelli che producono errori o risultati impossibili [Wang et al. 2018]. Schema che collega i nomi e le relazioni. Il moderno LLMs migliora sostanzialmente la generazione, ma la generalizzazione cross-domain, le domande ambigue, le convenzioni specifiche del database e le richieste senza risposta rimangono difficili. La lezione chiave per il linguaggio naturale eseguibile è che l'ambiente formale stesso fornisce vincoli utili. I tipi di colonna, le chiavi estiche e le query esterne forniscono un feedback che manca di modellazione linguistica pura.

Tuttavia, text-to-SQL mostra anche i limiti di una singola metrica di precisione finale. La corrispondenza delle stringhe esatte può rifiutare query equivalenti. L'accuratezza dell'esecuzione può accettare query semanticamente errate che restituiscono lo stesso risultato. Una valutazione rigorosa richiede l'equivalenza in varie istanze di database o controesempi accuratamente costruiti. Questo è direttamente analogo alla valutazione della formalizzazione delle regole: un programma formale generato dovrebbe essere testato su scenari distintivi, non solo su una risposta osservata.

La sintesi del programma dal linguaggio naturale generalizza l'idea. I sistemi generano espressioni regolari, snippet di codice, trasformazioni dei dati o piccoli programmi specifici del dominio. Gli esempi possono limitare lo spazio candidato e i sistemi di tipo possono eliminare i programmi senza senso. La generazione neurale di espressioni regolari ha mostrato che la mappatura language-to-program può funzionare anche con una conoscenza limitata del dominio, illustrando anche la fragilità quando le descrizioni sono sottospecificate [Locascio et al. 2018]. Il programma assistito LLMs in seguito ha spostato il calcolo al di fuori del modello, ma il problema di sintesi fondamentale è rimasto: il modello deve scegliere un programma la cui semantica corrisponde alla richiesta.

Un solido compilatore dovrebbe quindi cercare molteplici forme di prova. Le descrizioni in lingua naturale limitano l'intento. Gli esempi di input-output limitano il comportamento estensuale. Tipi limitano le operazioni ammissibili. Gli schemi di dominio limitano il vocabolario. Test casi di bordo della sonda. Un programma generato che soddisfa tutti questi vincoli è più credibile di uno che si limita a eseguire. Questo principio di multi-prova può essere applicato alle regole dei documenti: autorità testuale, esempi positivi e negativi, risultati attesi e mutazioni semantiche dovrebbero tutti modellare un circuito prima della pubblicazione.

La programmazione logica fornisce un altro obiettivo eseguibile. Datalog regola le conseguenze di calcolo per un punto di correzione e supporta la valutazione incrementale. Prolog e linguaggi correlati rappresentano relazioni e ricerca. L'interpretazione astratta è stata a lungo utilizzata per analizzare i programmi logici, mostrando un ponte concettuale tra semantica dichiarativa e analisi del programma. Per le applicazioni in linguaggio naturale, la programmazione logica è adatta per chiusure monotoni: relazioni transitive, tassonomie, propagazione della dipendenza e regole politiche senza inadempienze complesse. È meno adatto quando il ragionamento dipende fortemente dall'incertezza, da fonti incoerenti o da eccezioni non monotoniche, a meno che non siano estesi con meccanismi espliciti.

Le tabelle decisionali e i sistemi di stabilimento sono spesso trascurati nei sondaggi accademici, ma sono molto rilevanti. Molte politiche non sono profonde teorie logiche; sono combinazioni finite di condizioni e risultati. Una tabella decisionale rende visibili la copertura, i conflitti e le combinazioni mancanti. Può essere compilato in predicati eseguibili e testato sistematicamente. Il linguaggio naturale può essere usato per scrivere o spiegare la tabella, mentre la tabella rimane la semantica autorevole. Per l'affittamento di documenti aziendali, le tabelle decisionali possono essere più sicure e più facili da convalidare rispetto alla logica arbitraria di primo ordine.

Le istruzioni temporali e procedurali richiedono obiettivi più ricchi. Macchine dello stato finito, alberi del comportamento, lingue di pianificazione e logiche temporali rappresentano azioni e condizioni ordinate. La robotica ha usato a lungo i parser linguistici che compongono primitivi eseguibili. I recenti sistemi di manipolazione neuro-simbolica continuano questo modello: un parser del linguaggio mappa un'istruzione a un programma i cui componenti possono includere

la percezione neurale e il conteggio simbolico, il ragionamento spaziale o il controllo dell'azione. Il successo di tali sistemi dipende da una libreria di competenze ristrette e da un ambiente a terra, non da una copertura semantica senza restrizioni [Tziafas and Kasaei 2026].

La programmazione logica probabilistica e i sistemi neuro-simbolici differenziabili tentano un'integrazione più stretta. DeepProbLog, Logic Tensor Networks e i sistemi correlati combinano i predicati appresi con la struttura logica [Manhaeve et al. 2018; Badreddine et al. 2022]. Sono preziosi quando l'incertezza e l'apprendimento sono intrinseci e possono iniettare vincoli nella formazione. Tuttavia, la differenziabilità non è equivalente alla garanzia formale. I valori della verità fuzzy e le sanzioni pezzanti possono guidare i modelli senza produrre obblighi di prova discreta. Per il linguaggio naturale eseguibile, tali sistemi possono stimare o classificare i candidati semantici, ma un confine di accettazione ad alta sicurezza beneficia ancora di un controllo simbolico separato.

La revisione neuro-simbolica 2026 NLP distingue i sistemi di iniezione, che compilano informazioni simboliche in modelli neurali, dai sistemi federativi, che conservano componenti autonome. Riferisce che i sistemi di iniezione dominano la letteratura, ma spesso mostrano modesti guadagni su ampi compiti NLU, mentre i sistemi federativi mostrano maggiori guadagni sui compiti di ragionamento. Gli autori avvertono esplicitamente che il confronto è confuso da diversi benchmark e che la generalizzazione fuori dal dominio rimane un grave problema [Chatzikyriakidis and Lappin 2026]. L'interpretazione equilibrata è che la federazione è architetticamente attraente per l'auditability e la decomposizione dei compiti, ma non ancora empiricamente provata come una strategia di ridimensionamento universale.

Il lignaggio semantico-parsing chiarisce anche il ruolo di un linguaggio intermedio. Una rappresentazione deve essere abbastanza espressiva da catturare il significato rilevante per compiti, ma abbastanza vincolato da compilare in modo affidabile. Se è troppo vicino al testo di superficie, l'esecuzione rimane implicita. Se è troppo generale, la traduzione e la complessità del risolutore diventano ingestibili. Beiser e Penz descrivono questo come un problema di linguaggio intermedio nei sistemi neuro-simbolici contemporanei: la scelta di un formalismo può muoversi piuttosto che risolvere la difficoltà [Beiser and Penz 2025]. Un'architettura multiformalista riduce la pressione su ogni lingua, ma introduce un nuovo requisito per il routing digitato e la coerenza cross-formalismo.

Un utile perno semantico non dovrebbe quindi fingere di essere la logica finale. Dovrebbe preservare le prove e le distinzioni neutrali del compito - entità, menzioni, eventi, relazioni, tempo, modalità, quantità, mondi, provenienza, alternative e copertura - pur consentendo l'abbassamento di esecutori specializzati. Alcune query possono essere risolte direttamente sul pivot utilizzando operazioni relazionali. Altri richiedono una formula temporale compilata, un sistema di vincolo o una macchina dello stato. Il perno è prezioso perché separa la comprensione del linguaggio da qualsiasi particolare solutore mantenendo le informazioni sufficienti per l'esecuzione a base di origine.

Il programma di ricerca pre-LLM ha raggiunto un successo sostanziale ma limitato nel dominio. Ha dimostrato che il linguaggio naturale può essere compilato in programmi eseguibili, che gli interpreti simbolici migliorano la sistematicità, che i tipi e gli schemi riducono gli errori e che la supervisione debole crea scorciatoie semantiche. Ha anche dimostrato che un'ampia copertura e una semantica esatta sono in tensione. LLMs migliora sostanzialmente la copertura e la generazione dei candidati, ma ereditano ogni problema strutturale identificato dall'analisi semantica. La sintesi corretta non è quella di scartare la macchina precedente. È quello di

posizionare LLMs davanti e intorno ad esso, con interfacce formali che rendono le vecchie lezioni applicabili su nuova scala.

3. Interpretazione Astratta ed Esecuzione Simbolica per il Linguaggio

3.1 Interpretazione Astratta come Semantica dell'Incertezza Limitata

L'interpretazione astratta è stata sviluppata come una teoria generale per l'elaborazione di approssimazioni sicure del comportamento del programma [Cousot and Cousot 1977]. Un programma può avere infinitamente molti input e stati, rendendo impossibile l'esecuzione esaustiva. Invece di enumerare ogni stato, un'analisi rappresenta insiemi di stati attraverso proprietà astratte come intervalli, segni, possibili tipi di oggetti, raggiungendo definizioni o relazioni alias. Le operazioni astratte sono progettate in modo che ogni comportamento concreto rilevante per la proprietà rimanga rappresentato. L'analisi può includere comportamenti impossibili e quindi segnalare falsi allarmi, ma un'eccessiva approssimazione del suono non omette silenziosamente un comportamento che la semantica concreta permette.

L'idea operativa è semplice. Un'esecuzione concreta tiene traccia dei valori esatti. Un'analisi a intervalli potrebbe tracciare che una variabile si trova tra zero e dieci. Quando il programma ne aggiunge cinque, l'operazione di trasferimento astratto produce un intervallo tra i cinque e i quindici. In un ramo, gli stati sono raffinati in base alla condizione. In una fusione, le informazioni sono unite. Per i loop e i processi ricorsivi, l'analisi cerca un punto di correzione. L'ampliamento può forzare la convergenza sacrificando la precisione, mentre il restringimento può recuperare alcuni dettagli in seguito. Il quadro matematico fornisce un vocabolario disciplinato per la precisione, la solidità, la convergenza e la raffinatezza.

L'applicazione del framework al linguaggio richiede una semantica concreta. L'oggetto approssimato non può essere "significato" in un senso indefinito. Per un compito limitato, il dominio concreto può essere l'insieme di strutture semantiche ammissibili in una grammatica specifica, ontologia, politica di fondazione, revisione delle fonti e teoria di fondo. Ogni struttura contiene possibili entità, assegnazioni di eventi, relazioni temporali, modalità e link sorgente. L'ambiguità crea diverse strutture. Le informazioni mancanti lasciano le dimensioni senza vincoli. Le fonti contrastanti possono generare mondi alternativi piuttosto che un'unione incoerente.

Uno stato semantico astratto riassume il set. Può registrare che un pronome può riferirsi a una delle due persone, che un evento si verifica entro un intervallo di tempo, che un obbligo è presente sotto un'interpretazione e assente sotto un altro, o che una quantità si trova all'interno di un intervallo. Un fatto deve valere in ogni interpretazione rappresentata. Può essere presente in almeno uno. Non può essere in grado di non ceppo. Ignoti indica che il sistema manca della rappresentazione, della copertura o delle risorse computazionali necessarie per decidere. Questa distinzione a quattro vie è più informativa di una probabilità da sola e più onesta che costringere ogni domanda a diventare vera o falsa.

Considerare un documento di criterio che afferma: "L'operatore invia l'avviso dopo la convalida. Se la modalità di emergenza è attiva, il supervisore può rinunciare al periodo di attesa. Supponiamo che non sia chiaro se "il periodo di attesa" si riferisca al periodo prima della convalida o di un periodo successivo. Una singola analisi rende una scelta fragile. Uno stato astratto può mantenere due modelli temporali. Se entrambi implicano che l'avviso deve seguire la convalida, tale conclusione è necessario. Se un solo permesso immediatamente, l'invio

immediato può essere effettuato. Una regola di conformità può identificare quale verdetto dipende dalla frase ambigua e richiedere chiarimenti solo quando necessario.

Il vantaggio principale non è che l'interpretazione astratta comprenda magicamente il linguaggio. È che dà una semantica di esecuzione di principio una volta che è stato costruito un insieme limitato di letture. Dice al sistema come aggregare le alternative senza confondere la possibilità con la necessità. Supporta anche domini modulari. L'identità, il tempo, la quantità, la modalità, la provenienza e lo stato normativo possono avere la propria astrazione, quindi essere combinati attraverso prodotti ridotti che si scambiano informazioni dove utili.

Un dominio di identità potrebbe rappresentare classi di equivalenza dei candidati con relazioni esplicite, diverse e irrisolte. Non deve unire le entità solo perché i nomi corrispondono. Un dominio temporale potrebbe utilizzare ordini parziali, intervalli o vincoli di differenza. Un dominio di modalità potrebbe distinguere il contenuto affermato, riportato, destinato, consentito, richiesto, proibito, ipotetico e controfattuale. Un dominio di quantità può utilizzare valori, intervalli, unità e compatibilità dimensionale. Un dominio di provenienza potrebbe ordinare stati di prova come estratti, proposti, derivati, verificati e confermati dall'uomo senza consentire la fiducia di cancellare la loro origine. Un dominio di copertura potrebbe tenere traccia se una ricerca è abbastanza completa per supportare le richieste di assenza.

I domini devono essere scelti dalla proprietà da controllare. Una terminologia linter non ha bisogno di un ricco reticolo diontico. Una regola temporale non necessita di semantica lessicale completa se gli eventi e le date sono già materializzati. Questa selezione basata sulle attività riduce la complessità e rispecchia l'analisi statica basata sulla domanda. In un lungo tempo di esecuzione, circuiti o regole dichiarano i tipi di osservazione e le dimensioni astratte di cui hanno bisogno. Il compilatore materializza solo quelle dimensioni e rifiuta la corsa quando un dominio critico non può essere fornito.

Le funzioni di trasferimento sono la parte più difficile. Nell'analisi ordinaria del programma, la semantica di un'assegnazione o di un'operazione aritmetica è fissata dal linguaggio di programmazione. Nell'analisi del linguaggio naturale, l'operazione che aggiorna lo stato semantico può essere proposta da un LLM. Una frase potrebbe introdurre un evento, modificare uno stato precedente, stabilire una relazione temporale, creare un'eccezione o segnalare la convinzione di qualcun altro. Se un traduttore istruito diventa direttamente la funzione di trasferimento, la solidità classica viene persa.

Un design difendibile separa la generazione dei candidati dal trasferimento ammesso. Lo LLM propone un set finito di aggiornamenti semantici digitati con intervalli di sorgenti e alternative. Un materializzatore deterministico convalida che l'evidenza citata esiste, i campi soddisfano uno schema, unità e tipi sono compatibili, i riferimenti puntano agli oggetti ammissibili e nessuno stato viene aggiornato silenziosamente. L'interprete astratto applica quindi le funzioni di trasferimento registrate ai registri ammessi. L'analisi risultante è solida rispetto al set di candidati ammessi, non necessariamente relativo ad ogni interpretazione che un essere umano potrebbe riconoscere. Questa condizionalità deve essere esplicitata.

Ci sono diversi modi per rafforzare l'ipotesi del candidato. Molteplici prompt o modelli indipendenti possono proporre alternative. I generatori basati su grammatica o ontologia possono garantire la copertura per i frammenti controllati. Le varianti di parafrasi possono verificare la stabilità. Esempi contrastivi possono rivelare interpretazioni mancanti. La revisione umana può concentrarsi sulle ambiguità rilevanti per i risultati. Un compilatore può deliberatamente generare eccessivamente all'interno di una grammatica delimitata e lasciare

che i vincoli successivi eliminino i candidati impossibili. Ogni metodo scambia i costi per la fiducia che l'insieme contiene tutte le letture materialmente plausibili.

L'interpretazione astratta è anche rilevante per la propagazione delle regole su scala documentale. Molti problemi di linting e compliance assomigliano all'analisi del flusso di dati. I fatti vengono generati in posizioni di origine, propagati attraverso gli ambiti, uniti tra le sezioni e invalidati da dichiarazioni successive. Una definizione può essere disponibile dopo la sua dichiarazione, ma non prima. Un oggetto può cambiare lo stato attraverso una narrazione. Un requisito può essere applicato entro una giurisdizione o versione. Un reclamo può acquisire prove in seguito. Un motore a punti di correzione incrementale può calcolare le conseguenze e aggiornare le regioni interessate solo quando il documento cambia.

Questa prospettiva evita di eseguire nuovamente una LLM generale su un intero documento dopo ogni modifica. Gli artefatti strutturali e semantici stabili possono essere memorizzati nella cache dal contenuto digest. Un paragrafo modificato invalida le osservazioni locali e gli stati astratti a valle che dipendono da esse. Il grafico di analisi identifica quali circuiti hanno bisogno di ripronorsi. Questo è analogo ai framework incrementali di compilazione e statico-analisi. Rende anche precise le spiegazioni: il sistema può mostrare quale cambiamento di fonte alterato quale invariante astratto e perché.

L'ampliamento e la perdita di precisione hanno analoghi semantici. Una lunga narrazione può contenere centinaia di possibili identità di oggetti o relazioni temporali. Mantenere ogni combinazione è poco pratico. Il sistema può riassumere “uno di questi oggetti”, “qualche tempo all'interno di questo intervallo” o “una di queste giurisdizioni”. Tale allargamento può introdurre avvertimenti spuri. Il restringimento o la raffinatezza locale può recuperare precisione intorno a una sospetta violazione. L'obiettivo progettuale non è quello di evitare l'astrazione ma di rendere visibile e reversibile la sua perdita di precisione.

La consolidata letteratura NLP sulla robustezza certificata utilizza idee di interpretazione astratta in modo diverso. Jia e colleghi hanno utilizzato la propagazione a intervalli per addestrare modelli le cui previsioni sono certificate contro ogni sostituzione di parole in una famiglia predefinita, riportando l'accuratezza avversaria del 75% su IMDB e SNLI sotto il loro modello di minaccia [Jia et al. 2019]. Il lavoro successivo ha utilizzato soluzioni di levigatura randomizzate, incorporazioni consapevoli della robustezza e domini di Transformer specializzati per migliorare i limiti certificati [Ye et al. 2020; Wang et al. 2023; Huang et al. 2026]. Si tratta di garanzie autentiche, ma il dominio concreto è un insieme di input di modelli perturbati, non un insieme di interpretazioni semantiche. La famiglia di trasformazione è definita esternamente e si presume che sia preservata l'etichetta.

Questa distinzione è importante. Certificare che la previsione di un classificatore è invariata sotto tutte le sostituzioni in un elenco sinonimico non prova che ogni sostituzione preserva il significato della frase nel contesto. Non prova che il classificatore sia corretto sulla frase originale. Certifica la robustezza rispetto a un set di perturbazioni formali. Questa letteratura è ancora molto rilevante perché dimostra come i domini astratti possono scalare gli input del linguaggio combinatorio e come la precisione della verifica dipende dalla rappresentazione delle correlazioni. Fornisce anche un avvertimento metodologico: la più forte affermazione formale è sempre relativa al set concreto effettivamente definito.

Una direzione promettente della ricerca è quella di collegare i due usi. L'analisi neurale certificata potrebbe limitare il comportamento del modello della proposta semantica sotto parafrasi controllate, mentre l'interpretazione astratta semantica potrebbe limitare le

conseguenze di interpretazioni alternative fondate. Il primo chiede se l'estrazione di un modello cambia in base alle trasformazioni di superficie consentite. Quest'ultimo chiede se una conclusione a livello di documento cambia in tutte le strutture semantiche ammissibili. Insieme potrebbero identificare se l'incertezza ha origine nell'instabilità del modello o nell'autentica ambiguità semantica.

LLMs può anche aiutare a costruire interpreti astratti, ma solo sotto accettazione meccanica. SAIL dimostra questo principio utilizzando LLMs per cercare trasformatori astratti, imponendo vincoli sintattici e semantici e ottimizzando un costo che misura la malinconia. I trasformatori accettati sono solidi a livello globale e il sistema può sintetizzare trasformatori precisi per gli operatori non lineari che non erano precedentemente disponibili [Gu et al. 2026]. La lezione più ampia è quella architettonica: i modelli generativi possono esplorare un immenso spazio di progettazione se un piccolo validatore formale controlla l'ammissione.

Chiedere direttamente a un LLM di eseguire l'interpretazione astratta è meno affidabile. Esperimenti sui benchmark di analisi dei programmi mostrano che i modelli possono seguire gli stili di interpretazione astratta, ma commettono errori logici su strutture complesse e stati allucinati [Mitchell et al. 2025]. Questo risultato dovrebbe temperare le proposte in cui un modello narra un invariante e la narrazione è accettata come analisi. Il modello è prezioso per suggerire invarianti, partizioni di dominio o predicati di perfezionamento. I controlli di calcolo e di solidità del punto di correzione dovrebbero rimanere simbolici.

Per il linguaggio naturale eseguibile, l'interpretazione astratta non è quindi né una metafora né una soluzione completa. È una teoria per l'approssimazione disciplinata dopo che il sistema ha definito ciò che conta come possibile interpretazione. Supporta la semantica di maggio / deve, la propagazione incrementale, la composizione del dominio, il ragionamento di assenza conservativa e la raffinatezza mirata. La garanzia è subordinata alla copertura semantica. Questa limitazione non è fatale; è il punto esatto in cui l'interpretazione assistita da modello deve essere valutata e governata.

3.2 Esecuzione Simbolica, Risoluzione dei Vincoli, Verifica di Modelli e Raffinamento Semantico

L'esecuzione simbolica classica esegue un programma con input simbolici piuttosto che valori specifici [King 1976]. Uno stato simbolico contiene una posizione di controllo, espressioni simboliche per le variabili di programma e una condizione di percorso che descrive quali input raggiungono quello stato. In un ramo, forchette di esecuzione. Un percorso aggiunge la condizione del ramo; l'altro aggiunge la sua negazione. Un solvitore di vincoli determina se le condizioni sono soddisfatti e possono produrre input concreti che realizzano un percorso fattibile. Il metodo è potente perché produce testimoni: non solo un avvertimento che un errore può esistere, ma un input e un percorso che lo dimostrano.

La difficoltà è l'esplosione del percorso. I rami si moltiplicano, i loop e la ricorsi creano percorsi incollati, teorie complesse risolutori di sfida e librerie esterne sono difficili da modellare. I sondaggi di esecuzione simbolica descrivono un ampio spazio di progettazione delle strategie di ricerca, fusione di stato, riassunti compositivi, esecuzione concolica, modelli ambientali e ottimizzazioni dei solutori [Baldoni et al. 2018]. Nessun singolo motore esplora ogni comportamento di software reale. Le affermazioni di esecuzione simbolica includono quindi limiti, ipotesi di modellazione e limitazioni di copertura.

Il linguaggio naturale si collega all'esecuzione simbolica in diversi modi distinti. Il più semplice è il linguaggio-a-codice: un LLM genera un programma e un esecutore simbolico convenzionale analizza quel programma. Un altro è il linguaggio-to-specification: il modello genera una proprietà o un vincolo, che viene controllato rispetto al codice esistente. Un terzo è l'esecuzione semantico-programma: un documento viene compilato in una macchina statale o in un programma di regole le cui interpretazioni alternative diventano scelte simboliche. Una quarta è l'esplorazione assistita da LLM: un modello dà la priorità ai percorsi, propone riassunti o motivi di vincoli che i solutori convenzionali non possono facilmente esprimere.

La terminologia deve rimanere precisa. L'esecuzione di SQL, Python o una forma logica generata da una domanda è l'esecuzione formale, ma non è necessariamente un'esecuzione simbolica nel senso sensibile al percorso. La differenza è importante perché l'esecuzione simbolica offre condizioni di percorso e testimoni concreti, mentre l'esecuzione ordinaria fornisce un risultato per uno stato concreto. Entrambi sono utili. Un sondaggio che li raggruppa senza distinzione fa apparire le garanzie più forti di quanto non siano.

SymGPT fornisce uno degli esempi più chiari di linguaggio-specification. Il sistema studia 132 regole in lingua naturale da tre standard ERC, li traduce con un LLM in una grammatica ristretta, sintetizza i vincoli che descrivono le violazioni e utilizza l'esecuzione simbolica per cercare contratti Ethereum. Gli autori riportano 5.783 violazioni su 4.000 contratti, di cui 1.375 con percorsi di attacco espliciti per il furto, e riportano prestazioni più forti rispetto a sei tecniche automatizzate e un servizio di auditing esperto di sicurezza [Xia et al. 2026]. La forza del disegno risiede nella grammatica stretta, nella sintesi sistematica dei vincoli e nei percorsi concreti del programma. Il suo rischio semantico rimanente sta nel fatto che ogni regola ERC sia stata tradotta fedelmente.

Questo modello si generalizza. Una politica può essere compilata in precondizioni, transizioni proibite ed eccezioni. Una procedura può essere compilata in stati e azioni. Una regola di continuità narrativa può modellare oggetti ed eventi, quindi cercare un percorso in cui un oggetto viene utilizzato dopo essere stato lasciato indietro senza recupero. Un protocollo

clinico può modellare l'ammissibilità e le sequenze di intervento. Un flusso di lavoro scientifico può modellare le trasformazioni dei dati e i requisiti di provenienza. In ogni caso, l'esecutore simbolico dovrebbe operare su un programma digitato piuttosto che su prosa grezza.

L'esecuzione simbolica è particolarmente utile per le controesempi. L'analisi astratta può indicare che una violazione è possibile perché ha perso una correlazione. Una ricerca di percorso può tentare di scegliere identità, tempi, condizioni ed eccezioni concrete che realizzano la violazione. Se il percorso è soddisfacibile, il testimone può essere reso come scenario basato sulla fonte. Se è insoddisfacibile, il sistema impara che l'avvertimento astratto era spurio sotto la semantica attuale.

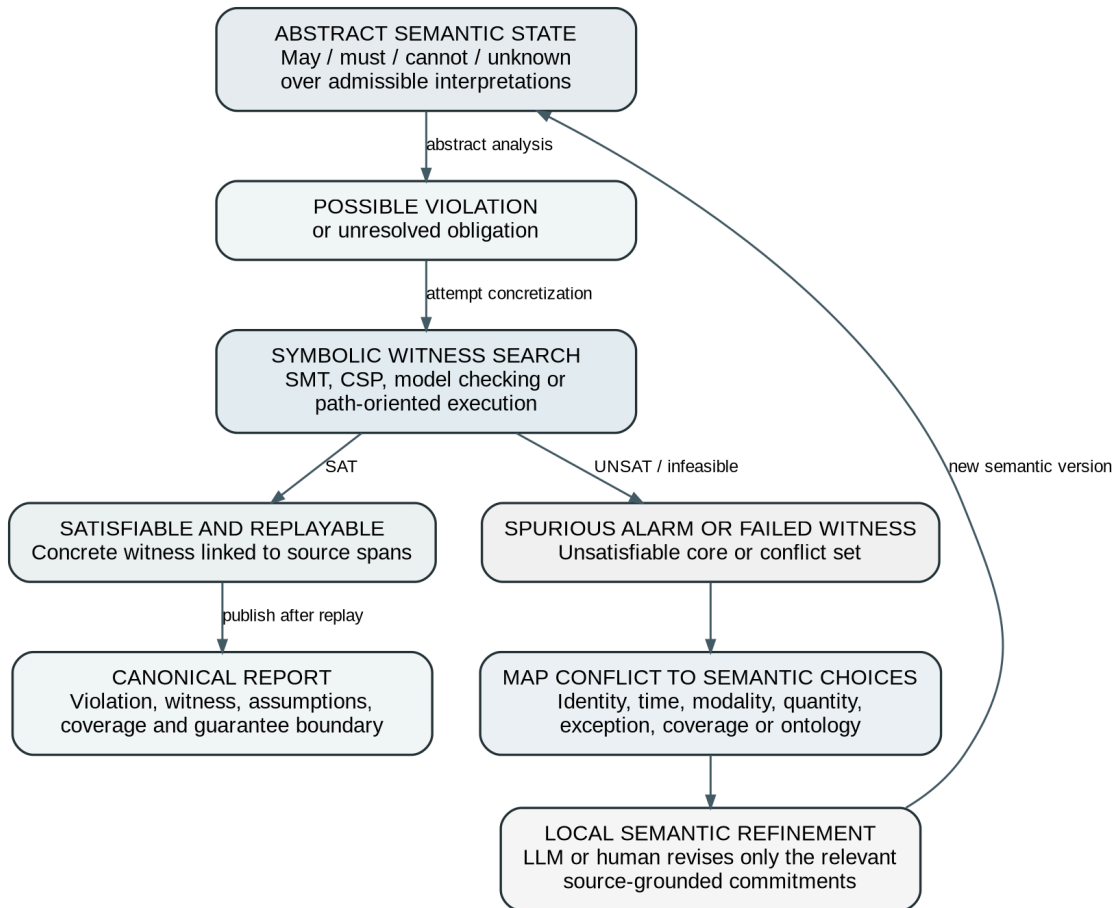


Figura 2. Raffinazione semantica guidata dal conteoexample. L'analisi astratta identifica una possibile violazione; la ricerca simbolica dei testimoni conferma un caso rigiocabile o espone le distinzioni semantiche che devono essere perfezionate.

Questo porta a un perfezionamento semantico controesterno guidato. CEGAR tradizionale inizia con un programma grossolano di astrazione. Il controllo del modello trova un controesempio astratto. Un controllo di fattibilità determina se il controesempio corrisponde ad un'esecuzione concreta. In caso contrario, i predicati vengono aggiunti per perfezionare l'astrazione e l'analisi si ripete. Nell'impostazione del linguaggio, l'astrazione include scelte semantiche. Un controesempio spurio può sorgere perché due menzioni sono state fuse, un'eccezione è stata ignorata, un intervallo temporale è stato ampliato, o una dichiarazione modale è stata trattata come un fatto.

Un ciclo di perfezionamento semantico dovrebbe preservare la spiegazione simbolica del fallimento. Supponiamo che un avviso di conformità dipenda dal presupposto che “la tavola” in due sezioni denota lo stesso corpo. La riproduzione simbolica mostra che il percorso diventa

impossibile se le entità sono distinte. Il sistema mappa il predicato dell'identità alle menzioni pertinenti e pone una domanda mirata: sono questi riferimenti destinati a indicare la stessa scheda? Un LLM può riesaminare il contesto locale e proporre alternative; un essere umano può giudicare se necessario. Il resto del modello del documento rimane invariato. Questo è più affidabile ed efficiente che chiedere a un modello di rileggere tutto e “provare di nuovo”.

I nuclei insoddisfatti forniscono un meccanismo correlato. Quando un insieme di vincoli è incoerente, un risolutore può spesso restituire un sottoinsieme responsabile del conflitto. La mappatura del nucleo di back to source span identifica l'insieme minimo di dichiarazioni e interpretazioni che non possono coesistere. In una revisione politica, ciò può rivelare obblighi contraddittori. Nell'analisi semantica, può rivelare che un quantificatore, un collegamento di identità e un'ipotesi temporale creano congiuntamente l'impossibilità. L'utente riceve una spiegazione fondata sul conflitto formale piuttosto che una critica generica del modello.

Non tutti i problemi sono rappresentati al meglio come esecuzione del percorso. Il controllo del modello analizza i sistemi di transizione contro le proprietà temporali. La logica lineare può esprimere che qualcosa alla fine accade, tiene sempre o si verifica prima di un altro evento. Le logiche di calcolo e probabilistica esprimono il ramo o un comportamento incerto. I sistemi-linguamente-temporali-logici come NL2TL e nl2spec mostrano che LLMs può costruire specifiche temporali quando il vocabolario e il dominio sono controllati [Chen et al. 2023; Cosler et al. 2023]. L'esecutore è quindi un modello checker o pianificatore, non un LLM.

La risoluzione dei vincoli copre un'ampia via di mezzo. SAT rappresenta le scelte booleane. SMT aggiunge teorie come aritmetica, array, stringhe e funzioni non interpretate. CSP e CP-SAT gestiscono domini finiti, pianificazione e ottimizzazione. MaxSAT e vincoli morbidi esprimono preferenze. Molte attività in linguaggio naturale - pianificazione, allocazione delle risorse, controllo della coerenza, relazioni familiari, puzzle e combinazioni di politiche - sono meglio compilati in vincoli che in codice procedurale. SatLM ha dimostrato che le specifiche dichiarative più un solutore possono sovraperformare gli approcci assistiti dal programma su compiti aritmetici e logici rigidi [Ye et al. 2023]. Il risultato supporta una scelta architettonica centrale: lo LLM dovrebbe descrivere il problema, mentre il risolutore esegue la ricerca.

La pianificazione formale fornisce la semantica dell'azione e la ricerca orientata agli obiettivi. Il lavoro recente posiziona LLMs come formalizzatori di pianificazione piuttosto che pianificatori. Hao e colleghi traducono i vincoli di pianificazione del mondo reale in strumenti di verifica formali, utilizzano core insoddisfatti per la riparazione e riportano il 93,9% di successo su TravelPlanner nel loro ambiente, rispetto alle linee di base di secondo piano molto più basse [Hao et al. 2025]. Un sondaggio 2025 conclude che l'opportunità principale non è la sostituzione dei pianificatori, ma l'utilizzo di LLMs per costruire e perfezionare i modelli di pianificazione [Tantakoun et al. 2025]. Questa è proprio la divisione qui sostenuta.

I motori Dataflow e Datalog forniscono l'esecuzione di punti di correzione anziché la ricerca di percorso. Sono utili per le conseguenze monotoniche e la propagazione a livello documentale. Automi e motori di modello gestiscono sequenze ordinate di eventi. Le tabelle decisionali gestiscono politiche limitate. L'architettura dovrebbe instradare ogni problema al formalismo più semplice che la esprime. L'esecuzione simbolica non deve diventare un'etichetta universale per ogni componente simbolica.

LLMs può aiutare l'esecuzione simbolica tradizionale in modi più sicuri e più rischiosi. Un ruolo più sicuro è la priorità del percorso. Un modello può identificare le regioni di codice che possono contenere vulnerabilità o percorsi che potrebbero raggiungere un obiettivo, mentre

KLEE o un altro motore mantiene la decisione di fattibilità finale. Recenti lavori come KLEECopilot riportano miglioramenti sostanziali della copertura della ricerca guidata da LLM, sebbene lo stato di pre stampa e la sensibilità del modello richiedano un'interpretazione prudente [Chen et al. 2026]. Il principio è il suono: l'euristica appresa decide dove cercare, non se un percorso è valido.

Un ruolo più rischioso è la sostituzione del risolutore. AutoBug, presentato dai suoi autori come esecuzione simbolica a modello in grande lingua, decompone il codice in compiti di ragionamento basati sul percorso e chiede a un modello di ragionare direttamente su di loro senza tradurre tutto in SMT. L'approccio è leggero e linguistico agnostico e gli esperimenti segnalati migliorano l'analisi del programma LLM [Li et al. 2025]. Gli autori lo caratterizzano come approssimativo. Può analizzare costrutti che sono difficili per i solutori convenzionali, ma non fornisce la solidità classica. Per un uso ad alta garanzia, tale modello è meglio trattato come un motore in un portafoglio i cui risultati richiedono test, rigiocazioni o conferme indipendenti.

LLMs può anche generare test, riassunti o modelli di route-sacloving per API esterne. Gli studi riportano una copertura utile su programmi che gli strumenti tradizionali lottano per gestire, ma le prestazioni diminuiscono con percorsi lunghi, API complesse, ricorsione e classificazione in modo impossibile. Questi metodi sono promettenti perché gli ambienti software contengono semantica che sono costosi da assigillare. Il loro stato di risultato dovrebbe rimanere "candidato" fino a quando l'esecuzione concreta non confermi il percorso o un solutore dimostri la condizione.

La sicurezza è una preoccupazione importante. Le regole del linguaggio naturale e i documenti di origine sono input non attendibili. Un documento dannoso può tentare l'iniezione rapida, fabbricare riferimenti o sfruttare un traduttore che genera codice. Gli artefatti formali dovrebbero essere dati, non un codice eseguibile senza restrizioni. Quando sono necessari algoritmi personalizzati, dovrebbero essere rivisti le estensioni host con digest bloccati, ingressi e uscite digitati, limiti di risorse e nessuna autorità ambientale. Le invocazioni del risolutore hanno bisogno di budget di tempo e memoria. Un testimone dovrebbe includere la versione esatta della teoria, del compilatore e del motore che l'ha prodotta.

Il modello di analisi combinata è ormai chiaro. Un'analisi semantica e del flusso di dati grossolano identifica i possibili problemi in un lungo documento. I motori formali specializzati testano solo le parti pertinenti. L'esecuzione simbolica costruisce testimoni per le rivendicazioni dipendenti dal percorso. Il controllo del modello gestisce i comportamenti temporali. I risolutori di vincoli gestiscono le scelte combinatorie. Testimoni e nuclei di solutori falliti guidano la raffinatezza semantica locale. Ogni conclusione accettata viene restituita con uno stato e una provenienza che riflettono l'anello più debole della catena.

Questa architettura non promette la verifica totale del linguaggio senza restrizioni. Promette qualcosa di più operativo: ogni rivendicazione formale ha un significato eseguibile; ogni simbolo ha un'ipotesi di origine o dichiarata; ogni controesempio può essere rigiocato; ogni pretesa di assenza è legata alla copertura; e ogni ambiguità rimanente è rappresentata piuttosto che nascosta. L'interpretazione astratta fornisce scala e aggregazione conservativa. L'esecuzione simbolica fornisce precisione e testimoni. La loro combinazione è una delle basi più plausibili per il linguaggio naturale eseguibile.

4. Che cosa ha Realmente Ottenuto l'Era degli LLM

4.1 Autoformalizzazione, Ragionamento Assistito da Programmi, Logica, Specifiche Temporal e Pianificazione

L'arrivo di grandi modelli linguistici ha cambiato il confine pratico della compilazione semantica. I sistemi precedenti richiedevano grammatiche specifiche per attività, forme logiche supervisionate o lessici attentamente progettati. LLMs può proporre artefatti formali da descrizioni aperte con poca o nessuna formazione specifica per attività. Questo espande sostanzialmente la copertura, ma crea anche la tentazione di dedurre più dai guadagni di benchmark rispetto ai meccanismi giustificati. La domanda pertinente non è semplicemente se un metodo migliora l'accuratezza della risposta. È quali errori sono stati trasferiti a una componente simbolica, che gli errori rimangono nella formalizzazione, e quali prove indipendenti esiste che l'artefatto formale rappresenta la fonte.

I modelli di lingua con aiuto al programma, o PAL, forniscono un punto di partenza pulito. Lo LLM legge un problema, genera un programma Python e delega il calcolo all'interprete [Gao et al. 2022]. Su GSM8K, il sistema riportato è migliorato sostanzialmente rispetto al prompting della catena del pensiero. Il meccanismo è convincente perché i modelli linguistici sono spesso più bravi a scomporre un problema che a eseguire esattamente una lunga aritmetica. Una volta che la decomposizione è corretta, Python fornisce operazioni e stato deterministici.

La limitazione di PAL è altrettanto istruttiva. L'interprete certifica solo che ha eseguito il programma generato. Non certifica che il programma modelli fedelmente la questione del linguaggio naturale. Una variabile errata, una condizione omessa o una formula sbagliata possono essere eseguite senza errori. Il successo del sistema dipende quindi da una distribuzione in cui i programmi sono brevi, esempi fortemente limitano la struttura e gli errori di esecuzione rivelano molti errori. Per il ragionamento documentale, l'analisi delle politiche e l'interpretazione scientifica, la mappatura è meno limitata e l'interprete fornisce un feedback più debole.

SatLM sostituisce il codice procedurale con vincoli dichiarativi [Ye et al. 2023]. Il modello indica variabili e relazioni e un motore di soddisfabilità cerca una soluzione. Questo può essere più facile che generare un algoritmo perché il modello non deve decidere l'ordine delle operazioni. Le rappresentazioni dichiarative consentono anche più modelli, prove di infeasibilità e debug a livello di vincolo. I guadagni segnalati sulle attività di ragionamento duro supportano l'idea che la scelta del solutore dovrebbe seguire la struttura dei problemi. Un programma è naturalmente un problema di vincoli; un processo dinamico può essere una macchina dello stato; un problema di parola aritmetica può essere un programma o vincoli.

Logic-LM formalizza i problemi logici del linguaggio naturale e invoca i risolutori simbolici [Pan et al. 2023]. Utilizza anche gli errori del risolutore per richiedere l'auto-raffinamento. In diversi set di dati, gli autori riportano grandi miglioramenti medi rispetto a quelli standard e a catena del pensiero. L'architettura dimostra due principi utili. In primo luogo, il motore simbolico rimuove l'incoerenza deduttiva una volta che esiste una valida formalizzazione. In secondo luogo, i messaggi di errore formali forniscono un segnale di riparazione migliore rispetto a una richiesta generica da riconsiderare.

Tuttavia, la riparazione della sintassi non è una riparazione semantica. Una formula può essere ben formata, soddisfacendo e sbagliata. La pipeline di Logic-LM può correggere i predicati malformati o i fallimenti del risolutore, ma potrebbe non rilevare che “alcuni” sono diventati “tutti” o che è stata omessa un’eccezione. Il lavoro successivo come Logic-LM++ aggiunge perfezionamento in più fasi e LTRAG recupera esempi di formalizzazione guidata dal pensiero, riportando miglioramenti su Logic-LM e LINC su FOLIO e AR-LSAT [Kirtania et al. 2024; Hu et al. 2025]. Questi guadagni suggeriscono che la formalizzazione stessa beneficia di esempi e critiche strutturate. Non eliminano la necessità di valutare direttamente la fedeltà semantica.

LINC rende esplicita la modularità: un LLM funge da parser semantico, la logica di primo ordine rappresenta il problema e un dimostratore di teoremi esegue la deduzione [Olausson et al. 2023]. Su ProofWriter, la combinazione riportata ha prodotto grandi guadagni rispetto ai sistemi di catena del pensiero, compresi i modelli sostanzialmente più grandi del parser. L’analisi degli errori ha mostrato modalità di fallimento complementari: il sistema simbolico evita molti errori di ragionamento, mentre i guasti di formalizzazione rimangono. Questa è esattamente l’evidenza che ci si aspetta da un’architettura federata. Il sistema è più forte quando il compito in linguaggio naturale corrisponde già a una teoria compatta del primo ordine.

FOLIO fornisce un utile benchmark perché contiene problemi di ragionamento in linguaggio naturale creato dall’uomo accoppiati con la logica di primo ordine verificata [Han et al. 2024]. Valuta sia il ragionamento che la traduzione NL-to-FOL. Il set di dati rimane impegnativo anche per i modelli forti, il che riduce l’idea che la formalizzazione del primo ordine sia un passo di pre-elaborazione risolto. FOLIO illustra anche l’importanza della complessità creata dall’uomo. I dataset sintetici delle regole possono rendere il linguaggio insolitamente regolare e consentire ai modelli di sfruttare i modelli. Gli esempi umani introducono variazioni lessicali, conoscenza del mondo e scelte di ambito più realistiche.

FoVer adotta un altro approccio verificando il ragionamento del linguaggio naturale attraverso la logica di primo ordine [Pei et al. 2025]. Invece di trattare il testo di ragionamento fluente come auto-autenticante, costruisce obblighi formali e li controlla. Questo è un modello promettente per la generazione con procedure: un LLM può proporre un argomento, ma un verificatore accetta solo passaggi inferenziali che corrispondono a conseguenze formali valide. Il problema rimanente è l’allineamento tra ogni frase e i predicati formali usati per controllarla.

La ricerca sull’autoformalizzazione tratta sempre più la traduzione come una disciplina distinta piuttosto che un trucco immediato. Sondaggi pubblicati nel 2025 la traduzione del catalogo in proverbi del teorema, lingue logiche e specifiche formali, sottolineando la necessità di cicli di feedback, recupero, rappresentazioni intermedie e benchmark che separano la sintassi dalla semantica [Weng et al. 2025]. Il lavoro diretto da NL a FOL riporta un progresso significativo, ma anche errori sistematici nelle conoscenze implicite, nell’ambito di applicazione più quantificabile e nella progettazione di ontologia [Yang et al. 2024; Lalwani et al. 2025]. L’aggiunta di conoscenze implicite in background può migliorare le prestazioni delle attività aumentando al contempo il rischio che il sistema formale dimostri una premessa non concessa in licenza dalla fonte.

Il trattamento delle conoscenze di base è una scelta architettonica decisiva. In un benchmark di puzzle, le precondizioni di buon senso possono essere implicitamente attese. Nella verifica legale o scientifica, l’introduzione di locali non dichiarati può invalidare il risultato. Un sistema rigoroso dovrebbe distinguere i locali della fonte, i locali di imballaggio delle autorità,

gli assiomi di dominio e le ipotesi suggerite dai modelli. Il risolutore può ragionare su tutti loro, ma la spiegazione finale deve mostrare quali conclusioni dipendono da quale autorità. Se un'ipotesi è essenziale e non verificata, il risultato dovrebbe essere condizionato.

La traduzione tempo-logica comporta diverse sfide. NL2TL utilizza rappresentazioni revocate per apprendere la struttura temporale tra i domini e riporta un forte adattamento con relativamente pochi dati di dominio del bersaglio [Chen et al. 2023]. La strategia di sollevamento nasconde proposizioni atomiche concrete mentre si imparano modelli logici riutilizzabili. Questa è una preziosa intuizione progettuale: separa la struttura di “sempre prima”, “alla fine del dopo” o “fino” dalla base specifica del dominio degli eventi.

nl2spec enfatizza l'interazione [Cosler et al. 2023]. Il sistema mappa parti del requisito del linguaggio naturale per parti della formula temporale generata, consentendo agli utenti di riparare errazioni errate locali. Questa interfaccia riconosce che l'ambiguità non può sempre essere risolta automaticamente. Suggerisce anche come il perfezionamento semantico basato su LLM dovrebbe funzionare: l'utente dovrebbe vedere la decisione linguistica, non solo una serie di operatori temporali.

Il terreno rimane il punto fragile. Una formula temporale può avere la forma corretta mentre le sue proposizioni atomiche si riferiscono agli eventi o agli oggetti sbagliati. Lavorare su rapporti di equivalenza logica fondata che i meccanismi di base espliciti migliorano la correttezza, rafforzando la distinzione tra la sintassi della formula e l'attaccamento semantico. Per un runtime in lingua eseguibile, gli atomi temporali dovrebbero essere riferimenti a osservazioni digitate con intervalli e identità, non etichette a forma libera inventata dal traduttore.

La pianificazione automatizzata fornisce un backend simbolico particolarmente maturo. I linguaggi di pianificazione definiscono i tipi, i predicati, le azioni, le condizioni preliminari, gli effetti, gli stati iniziali e gli obiettivi. I pianificatori cercano quindi una sequenza di azione valida. LLMs sono pianificatori di longozon quando devono mantenere lo stato e i vincoli internamente, ma possono essere efficaci nel formalizzare un problema per un pianificatore. Un'indagine 2025 identifica la costruzione e il perfezionamento dei modelli come il ruolo centrale di LLMs in quest'area [Tantakoun et al. 2025].

Hao e colleghi dimostrano una versione rigorosa del modello sulla pianificazione reale [Hao et al. 2025]. Il sistema formalizza i vincoli, invoca strumenti di verifica e utilizza nuclei insoddisfatti per riparare i problemi. Gli autori riportano il 93,9% di successo su TravelPlanner nel loro ambiente sperimentale, molto al di sopra delle linee di base dirette con cui si confrontano. Il risultato è importante perché mostra che gli strumenti formali possono trasformare l'affidabilità della pianificazione quando il problema è rappresentabile. Ciò non significa che lo LLM sia diventato un pianificatore del suono. Significa che il pianificatore e il verificatore trasportano lo stato a lungo-orizzonte mentre lo LLM costruisce e ripara l'istanza formale.

La pianificazione espone anche i vincoli nascosti. Le richieste in linguaggio naturale includono aspettative di buon senso, preferenze e condizioni di fattibilità che non possono essere indicate come requisiti formali. Sistemi come LRPlan tentano la collaborazione tra linguaggio e modelli di ragionamento per vincoli espliciti e impliciti, mentre benchmark come PlanningArena mostrano che anche i modelli forti lottano con la selezione degli strumenti, la memoria di contesto e la coerenza logica [Karthikeyan et al. 2025; Zheng et al. 2025]. Un

sistema ad alta sicurezza deve formalizzare i vincoli impliciti da una fonte attendibile o etichettarli come ipotesi. “Commonsense” non è una categoria di provenienza sufficiente.

RuleArena e i relativi benchmark testano LLMs contro i sistemi di regole realistici e trovano che l'aumento degli strumenti aiuta ma non rende banale la formalizzazione. Le attività di SemEval 2026 sul contenuto e il ragionamento formale mostrano inoltre che i modelli sono influenzati dal contenuto semantico anche quando la forma logica dovrebbe determinare la risposta. Le voci neuro-simboliche modulari utilizzano FOL e Z3 per ridurre il bias dei contenuti, ma la traduzione formale e il rumore multilingue rimangono difficili [Grimaldi 2026; Kartáč et al. 2026; Wang et al. 2026]. Questi risultati rafforzano la necessità di districare l'interpretazione linguistica dall'inferenza formale.

Una delle osservazioni più consequenziali in questa letteratura è che i sistemi di successo utilizzano linguaggi intermedi specifici per l'obiettivo. PAL usa Python. SatLM utilizza vincoli dichiarativi. Logic-LM e LINC usano la logica. NL2TL usa la logica temporale. I sistemi di pianificazione utilizzano modelli PDDL-like. SymGPT utilizza una grammatica su misura per le regole ERC. Il campo non ha scoperto una rappresentazione formale universale per il linguaggio naturale. Ha scoperto che LLMs può instradare la lingua in più ecosistemi formali quando sono disponibili esempi, schemi e feedback.

Questo supporta un'architettura di portafoglio piuttosto che un “codice del linguaggio” universale. Un perno semantico può preservare prove, entità, eventi, portata e modalità, mentre un selettore di formalismo sceglie il bersaglio dell'esecuzione. Il selettore stesso può essere assistito da un LLM, ma la sua decisione dovrebbe essere convalidata rispetto ai contratti di capacità. Un formalismo è compatibile solo se può esprimere le distinzioni richieste e le osservazioni fondate necessarie. Un pianificatore non dovrebbe essere selezionato per una query puramente tassonomica; la logica di primo ordine non dovrebbe approssimare silenziosamente una dinamica temporale incerta; un semplice motore relazionale non dovrebbe pretendere di decidere eccezioni legali defeasibili.

L'evidenza empirica sostiene quindi un'affermazione più ristretta ma sostanziale. LLMs può migliorare notevolmente la costruzione di oggetti formali eseguibili. I sistemi simbolici esterni possono migliorare la correttezza, la coerenza e l'interpretabilità. Il feedback del sovvertimento può guidare la riparazione. I risultati più forti appaiono in domini limitati con linguaggi espliciti, schemi chiari e risposte valutabili. La sfida irrisolta è la compilazione semantica: garantire che l'oggetto formale preservi il significato rilevante della fonte e che ogni premessa sia fondata e autorizzata.

4.2 Analisi di Programmi Assistita da LLM, Esplorazione Simbolica, Trasformatori Astratti e NLP Certificato

L'interazione tra LLMs e l'analisi del programma viene eseguita in entrambe le direzioni. LLMs può aiutare gli strumenti formali a gestire le specifiche, i modelli di codice e gli ambienti difficili da modellare. Gli strumenti formali possono controllare il codice generato da LLMs, verificare le reti neurali e limitare i componenti appresi. Questa letteratura è particolarmente preziosa per il linguaggio naturale eseguibile perché testa la stessa domanda architettonica sotto semantica più chiara: dove può contribuire un modello probabilistico senza diventare l'oracolo finale?

L'analisi statica tradizionale dipende da astrazioni progettate a mano, funzioni di trasferimento, riassunti e modelli ambientali. L'esecuzione simbolica dipende dalla ricerca del percorso e dalle codifiche risoltrici. Questi strumenti offrono garanzie di principio ma lottano con le caratteristiche dinamiche del linguaggio, il codice della libreria, la manipolazione delle stringhe complesse, i servizi esterni e la scalabilità. LLMs è in grado di riconoscere l'intento del codice, proporre invarianti, riassumere le funzioni, identificare le posizioni dei bug e generare test. Un sondaggio 2025 dell'analisi del programma assistito da LLM classifica il lavoro in metodi statici, dinamici e ibridi e sottolinea sia l'opportunità che i rischi degli output probabilistici [Wang et al. 2025].

Un ruolo architettonico è la guida euristica. Un esecutore simbolico può mantenere la sua semantica e utilizzare un LLM per classificare i percorsi, identificare le regioni vulnerabili o selezionare i limiti di loop. Se l'euristica è sbagliato, la copertura può soffrire, ma i percorsi accettati rimangono controllati. Questo è analogo all'uso di un modello neurale per l'ordinazione di ricerca nel teorema di prova. La distinzione tra orientamento e accettazione è fondamentale. Un punteggio appreso può determinare dove vengono spese le risorse; non dovrebbe determinare se una condizione di percorso è soddisfacibile.

KLEECopilot rappresenta questa direzione. La pre stampa 2026 utilizza la guida LLM per indirizzare KLEE verso le probabili vulnerabilità e introduce euristiche per la fuga del ciclo, riportando miglioramenti sostanziali nella copertura di codici di base e di linea e dozzine di violazioni uniche [Chen et al. 2026]. Come recente pre stampa, i risultati richiedono una replica indipendente e la sensibilità alle questioni familiari di modello. Tuttavia, il modello di progettazione è plausibile: LLMs contribuisce alla prioritizzazione semantica che manca di ricerca classica, mentre KLEE mantiene la semantica dell'esecuzione.

Un altro ruolo è la generazione di test che soddisfano i percorsi. Un modello può leggere il codice e proporre input che dovrebbero attraversare un percorso di destinazione. L'esecuzione concreta quindi convalida se il percorso è stato raggiunto. Questo approccio è in grado di gestire le convenzioni API o i formati di dati che sono gravosi per SMT. Gli studi riportano tassi di successo utili ma anche fallimenti su percorsi lunghi, ricorsi, rami impestibili e librerie complesse. L'interpretazione sicura è che lo LLM è un generatore di test. Una corsa concreta, una traccia di strumentazione o un risolutore rimane il validatore.

AutoBug adotta un approccio più radicale utilizzando LLMs per eseguire il ragionamento tracciato direttamente [Li et al. 2025]. Invece di tradurre tutti i vincoli in una teoria del solutore, presenta le condizioni del percorso a livello di codice al modello. Questo può essere agnostico del linguaggio e può analizzare la semantica difficile da codificare. Gli autori riportano miglioramenti su diversi benchmark di analisi dei programmi, in particolare per i modelli più

piccoli. Essi caratterizzano anche l'analisi come approssimativa. Il sistema non offre una solidità classica e le sue conclusioni possono variare con il modello.

AutoBug è prezioso perché mostra dove i metodi simbolici falliscono operativamente. Molti programmi reali chiamano librerie opache, manipolano oggetti complessi o dipendono da contratti informali. Un modello di linguaggio può dedurre il probabile comportamento da nomi, commenti e modelli di codice appresi. Ma questa flessibilità è la stessa fonte di incertezza. In un portafoglio orientato all'assicurazione, l'analisi LLM di approssimazione dovrebbe generare candidati, riepiloghi o controlli prioritari. I risultati dovrebbero essere confermati da test, contratti, monitoraggio del runtime o motore formale ogni volta che viene presentata una richiesta di sicurezza.

LLMs può proporre invarianti per l'interpretazione astratta. Un candidato invariante può accelerare la prova se un controllore verifica che detiene inizialmente, è preservato dalle transizioni e implica la proprietà. Questa è una potente divisione del lavoro: la scoperta invariante è creativa e combinatoria, mentre il controllo invariante è meccanico. Modelli simili appaiono nel teorema che prova e nella sintesi del programma. Il modello di lingua cerca; il verificatore accetta.

SAIL estende questo principio alla sintesi dei trasformatori astratti [Gu et al. 2026]. La costruzione di un trasformatore che sia sano e preciso per un operatore non lineare è difficile. SAIL chiede a LLMs di generare candidati, ma impone vincoli sintattici e semantici e utilizza un costo matematicamente fondato per la non sana. Il sistema riportato sintetizza i trasformatori di suono a livello globale in diversi domini, abbina i progetti manuali e trova trasformatori precisi non precedentemente in letteratura. Il significato va oltre la verifica della rete neurale. È la prova che LLMs può automatizzare la costruzione del metodo formale quando ogni artefatto accettato è soggetto a obblighi di prova indipendenti.

Il contrasto con la richiesta di un LLM di “agire come un interprete astratto” è acuto. Mitchell e colleghi hanno spinto i modelli a ragionare attraverso l'interpretazione astratta sui programmi SV-COMP [Mitchell et al. 2025]. I modelli potevano seguire gli stili compositivi o di punto di correzione in alcuni casi, ma hanno commesso errori logici e allucinati su strutture complesse. Il risultato è coerente con un modello generale: i modelli possono imitare i processi formali e generare candidati utili, ma l'affidabilità diminuisce quando il processo richiede uno stato esico persistente. Un vero interprete astratto dovrebbe eseguire le operazioni reticolari; il modello può suggerire domini, predicati o spiegazioni.

La verifica formale dei modelli NLP neurali è un altro ramo importante. Il lavoro di robustezza certificata chiede se la previsione di un modello rimane invariata su tutti gli input in un insieme di perturbazioni definite. Jia e colleghi hanno utilizzato la propagazione a intervalli per le sostituzioni di parole e hanno riportato una precisione contraddittoria del 75% su IMDB e SNLI sotto le famiglie di sostituzione costruite [Jia et al. 2019]. SAFER ha utilizzato la levigatura randomizzata e potrebbe applicarsi ai modelli black-box tra cui BERT [Ye et al. 2020]. Gli involucri di robustezza-consapevoli rafforzano i limiti di certificazione, migliorando la robusta precisione certificata su IMDB da 76,73 a 84,78 percento nell'impostazione riportata [Wang et al. 2023].

La verifica del trasformatore è difficile perché l'auto-attenzione contiene prodotti, operazioni softmax e correlazioni tra le posizioni. I domini specializzati migliorano i limiti. L'interpretazione astratta parametrizzata per la verifica dei trasformatori ha introdotto domini affine parametrizzati per i prodotti interni e ha riportato limiti più stretti e la verifica delle istanze che i metodi

precedenti non potevano dimostrare [Huang et al. 2026]. Questa linea di lavoro dimostra che l'analisi formale può scalare in architetture NLP non banali quando la proprietà e l'insieme di input sono definiti con precisione.

L'ambito di applicazione della garanzia non deve essere offuscato. Un classificatore certificato può essere robusto per ogni sostituzione prelevata da un set specificato e comunque sbagliarsi. Un elenco di sostituzione può includere “sinonimi” contestualmente inappropriati o omettere parafrasi significative. La certificazione afferma che il modello è invariante sulla famiglia delle perturbazioni formali. Non stabilisce l'equivalenza semantica nel linguaggio umano. Questa distinzione dovrebbe essere esplicita in qualsiasi sistema di linguaggio eseguibile che utilizza modelli di estrazione certificati.

Un'utile integrazione aiuterebbe un certificato di robustezza a un'osservazione semantica. Supponiamo che un estrattore di eventi identifichi che una frase indica un obbligo. Un modello certificato potrebbe dimostrare che questa classificazione è stabile sotto una famiglia controllata di sostituzioni lessicali. L'osservazione è ancora proposta, ma ha un'ulteriore proprietà di stabilità. L'analisi simbolica a valle può registrare questa evidenza senza aggiornare l'osservazione ai fatti. Tale stato stratificato è più informativo di un punteggio di confidenza.

I metodi formali verificano anche il codice generato da LLMs. La generazione di test, l'analisi statica, il controllo del tipo e gli assistenti di prova possono rilevare molti errori. I sistemi di sintesi del programma utilizzano sempre più il compilatore e il feedback di prova per la riparazione iterativa. Lo stesso modello dovrebbe governare i circuiti semantici generati e il codice di formalizzazione. Un agente di codifica può autorizzare un'implementazione delle regole, ma la pubblicazione dovrebbe richiedere la compilazione, i test di mutazione, i casi di benchmark del linguaggio naturale, il replay e i verificatori indipendenti. A un LLM non dovrebbe essere consentito di creare un comportamento eseguibile nascosto all'interno di un artefatto di teoria dichiarativa.

La letteratura sulla modellazione del protocollo dal linguaggio naturale offre un risultato sobrio. I lavori presentati a ICSE 2025 hanno studiato la generazione di modelli Tamarin o ProVerif dalla documentazione del protocollo. L'uso di Naive LLM era inefficace; la pipeline di successo richiedeva la decomposizione semantica, la disambiguazione umana limitata e le trasformazioni intermedie sonore. Ha generato modelli corretti in scala moderata per 10 dei 18 protocolli reali [Mao et al. 2025]. Il risultato è significativo perché i protocolli contengono ruoli, messaggi, ipotesi crittografiche, stato e ordine temporale. Mostra anche che anche una forte documentazione strutturata non si compila automaticamente in modelli di sicurezza formali.

I casi di guasto sono istruttivi. La documentazione omette le ipotesi che gli esperti deducono. Lo stesso termine può indicare un tipo di messaggio, uno stato o un'istanza. Le proprietà di sicurezza possono essere dichiarate in modo informale o distribuito tra le sezioni. Un modello di protocollo formale deve rendere esplicite le capacità avversarie. Si tratta di analoghi diretti del ragionamento documentale. La soluzione non era un prompt più grande, ma un'architettura che separa l'analisi semantica, la disambiguazione e la trasformazione verificata.

Le revisioni esterne del campo più ampio convergono su una conclusione simile. La revisione neuro-simbolica 2026 NLP favorisce una più forte indagine sulle architetture federative, ma avverte che i confronti attuali non sono controllati e che non è stato stabilito [Chatzikyriakidis and Lappin 2026] un sistema federativo su scala fondamentale. Il sondaggio di pianificazione-formalizer tratta LLMs come costruttori e raffinatori di modelli formali, non sostituzioni per i pianificatori [Tantakoun et al. 2025]. Il sondaggio di analisi del programma

enfattizza i sistemi ibridi e la necessità di gestire allucinazioni e riproducibilità [Wang et al. 2025]. Queste sono linee di prova indipendenti per la stessa divisione del lavoro.

Il campo ha quindi raggiunto più di un modello superficiale “LLM plus solver”. Ha dimostrato la riparazione guidata dal solutore, le formalizzazioni verificate, la pianificazione formale, la guida del percorso, la proposta invariante, la sintesi dei trasformatori e la robustezza certificata. Ha anche esposto un confine stabile. Quando lo LLM controlla l'accettazione finale, le garanzie diventano approssimative. Quando LLM propone artefatti che un sistema formale separato controlla, è possibile mantenere forti garanzie per la proprietà controllata. La domanda senza risposta è come rendere il confine naturale-linguistico-artefatto comparabilmente disciplinato.

L'architettura proposta più avanti in questo libro applica direttamente le lezioni di analisi del programma. Le trasformazioni semantiche candidate sono trattate come codice sintetizzato. Hanno contratti digitati, mappe di origine, limiti di risorse e test. I domini astratti sono registrati piuttosto che improvvisati durante una corsa. I predicati di raffinamento sono suggeriti da LLMs ma convalidati attraverso distinzioni concrete. I portafogli del riassortimento espongono oggetti o testimoni di prova. Il renditore del linguaggio consuma risultati canonici, non tracce grezze. In questo design, la flessibilità dello LLM è una risorsa proprio perché è circondata da cancelli di accettazione simbolica.

5. Perché i Sistemi di Linguaggio Eseguitabile Falliscono

5.1 Ancoraggio, Identità, Formalizzazioni Spurie, Ontologie e Conoscenza del Mondo

L'errore dominante nei sistemi in lingua eseguibile non è una deduzione errata da locali accettati. È l'ammissione di premesse sbagliate, oggetti o relazioni. Il maschinamento è il processo mediante il quale i simboli acquisiscono un riferimento operativo: un predicato è collegato a una frase, un'entità a una o più menzioni, un evento per l'evidenza, un'espressione di tempo a un intervallo, un campo di database a un concetto o un'azione per una capacità ambiente. Un programma formale senza fondazione affidabile è esatto nel senso meno utile. Esegue costantemente mentre descrive il mondo sbagliato.

Il terreno inizia con la posizione della fonte. Ogni osservazione semantica dovrebbe essere collegata a prove di origine esatte o contrassegnata esplicitamente come ipotesi esterna. Una citazione deve essere convalidata contro la revisione immutabile della fonte. Una sintesi documentale non può sostituire la fonte perché può omettere qualifiche. I risultati del recupero non possono supportare un reclamo in merito all'assenza a meno che il processo di recupero non sia completo per il dominio pertinente. Questa disciplina di origine impedisce un fallimento comune nei sistemi LLM: un modello produce un testo di supporto plausibile che non si verifica nel documento e l'output strutturato a valle dà all'invenzione un'apparenza di autorità.

La fondazione lessicale collega il linguaggio all'ontologia. L'espressione "approvazione" può indicare un'azione, uno stato, un manufatto firmato, un permesso o una decisione istituzionale. Uno schema di dominio deve scegliere e documentare l'interpretazione necessaria per una regola. LLMs è utile perché possono dedurre le mappature e proporre mappature, ma una mappatura dovrebbe essere testata rispetto a esempi e alternative. Una corrispondenza di stringa tra una frase e un nome di tipo non è la compatibilità semantica. La documentazione nllAgent mette esplicitamente in guardia dalla mappatura di due concetti solo perché le loro etichette corrispondono; richiede un adattatore o una sonda semantica quando i significati differiscono.

La base dell'identità è più difficile e più consequenziale. L'analisi dei documenti deve spesso sapere se due menzioni indicano la stessa persona, organizzazione, oggetto, set di dati, requisito o evento. Gli uguali nomi non implicano l'identità e nomi diversi non implicano la differenza. I titoli cambiano, si verificano alias, i pronomi sono ambigui, le organizzazioni hanno suddivisioni e le menzioni fittizie o citate potrebbero non appartenere al mondo di origine. Uno strato di identità sicuro separa le menzioni dalle ipotesi di entità e dai candidati all'identità. Registra il produttore, le prove, la portata, il mondo e le alternative per ogni collegamento.

La fusione dell'identità prematura crea errori a cascata. In una narrazione, due personaggi con lo stesso cognome possono essere fusi, producendo impossibili violazioni della continuità. In un documento normativo, "l'Autorità" può riferirsi a diversi organismi in diverse giurisdizioni. Nella documentazione software, "client" può indicare un utente, un componente applicativo o un endpoint HTTP. Una volta fuse, le regole successive trattano tutte le proprietà come appartenenti a un oggetto. L'errore diventa difficile da diagnosticare perché il solutore vede un identificatore coerente. Un sistema conservativo dovrebbe consentire un'identità irrisolta e tornare sconosciuto quando una regola richiede un collegamento che non è stato stabilito.

La fondazione temporale ha strati simili. Le date possono essere esplicite o relative. “Entro trenta giorni” richiede un evento di ancoraggio e una politica del calendario. “Prima del rilascio” può riferirsi a un rilascio pianificato o effettivo. “Dopo l’approvazione” può significare immediatamente dopo o in qualsiasi momento successivo. Le giurisdizioni definiscono i giorni lavorativi in modo diverso. Un solutore temporale può gestire gli intervalli e ordinare esattamente una volta che queste scelte sono fatte, ma non può dedurre l’ancoraggio previsto da un’etichetta arbitraria. Le osservazioni temporali dovrebbero pertanto includere l’espressione testuale, il valore normalizzato o il vincolo, l’ancora, il calendario, il fuso orario, la fiducia e le alternative.

La fondazione normativa è spesso maltrattata. “Deve”, “devono”, “può”, “può”, “può” e “si prevede di” avere una forza specifica per il dominio. Le convenzioni di redazione legale differiscono dalla lingua ordinaria. Un’eccezione può sconfiggere una regola, restringere il suo ambito di applicazione o creare un permesso separato. Le autorità in conflitto possono avere delle priorità. Una semplice rappresentazione booleana della conformità perde queste distinzioni. La logica deontica, le priorità delle regole, le tabelle delle decisioni o i sistemi defeasibili possono modellarle, ma la mappatura source-to-rulezione rimane un giudizio linguistico e istituzionale.

Le formazioni spurie sorgono quando il programma sbagliato produce il risultato atteso. L’analisi semantica fortemente supervisionata ha reso esplicito questo problema, ma è pervasivo nella valutazione LLM. Una formula generata può risolvere un benchmark perché gli esempi hanno modelli regolari. Una query text-to-SQL può restituire le righe correnti corrette nonostante un’iscrizione errata. Un modello di pianificazione può trovare un piano perché un vincolo omesso è irrilevante nell’istanza testata. Un circuito di regole può passare dieci esempi mentre codifica il concetto sbagliato. La precisione della risposta finale da sola non può distinguere questi casi.

La difesa appropriata è la mutazione semantica. Il valutatore cambia un aspetto della fonte o dell’ambiente preservandone altri, quindi controlla se la formalizzazione cambia nel modo previsto. Se “tutti” diventa “alcuni”, il quantificatore dovrebbe cambiare. Se viene rimossa un’eccezione, il percorso precedentemente esente dovrebbe essere vietato. Se due entità vengono rinominate nella stessa stringa, l’identità non deve unire automaticamente. Se l’istanza del database cambia, una query SQL corre corretta accidentalmente dovrebbe fallire. Il test di mutazione valuta il significato del programma piuttosto che solo il suo comportamento su un unico dispositivo.

La generazione del conteessato può confrontare le formalizzazioni concorrenti. Dato due programmi logici o eseguibili, un solutore può cercare un modello in cui le loro uscite differiscono. Lo scenario risultante viene tradotto in linguaggio naturale e presentato a un revisore. Questo trasforma un confronto di formula opaca in una questione concreta: “Sotto l’interpretazione A, un supervisore che non ha completato la formazione può approvare; in base all’interpretazione B, non possono. Quale è inteso?” Il metodo è particolarmente utile quando diversi LLMs producono forme plausibili ma non equivalenti.

Il design ontologico crea un’altra modalità di fallimento. Uno schema universale è attraente per il riutilizzo, ma può nascondere i disaccordi di dominio e forzare i concetti in categorie inappropriate. Uno schema estremamente ristretto, al contrario, produce molte lingue locali incompatibili. Un disegno a strati è preferibile. Il livello superiore definisce la fonte, le prove, le ipotesi di identità, i mondi, il tempo, la modalità, lo stato, la copertura e le lacune. I pacchetti di

dominio definiscono tipi e relazioni sostanziali. I pacchetti di mappatura dichiarano corrispondenze, perdite e ipotesi tra domini.

La compatibilità ontologica deve essere operativa, non retorica. Un circuito dichiara i tipi e garantisce che consuma. Un produttore dichiara quali tipi può materializzarsi, sotto il quale gli stati e le modalità di copertura. La pubblicazione verifica se esiste un produttore compatibile. Verificare il tempo di esecuzione se il documento concreto ha prodotto le osservazioni richieste. Se il circuito ha bisogno di identità confermate dall'uomo, ma esistono solo candidati di identità proposti da modelli, non può correre alla garanzia dichiarata. Se una regola temporale ha bisogno di un calendario giurisdizionale assente, il risultato è un divario operativo, non una data indovinata.

La conoscenza mondiale è particolarmente pericolosa. LLMs contiene una vasta conoscenza ma nessuna provenienza stabile, data effettiva o giurisdizione. Trattare la memoria del modello come base di fatti formale crea dipendenze invisibili e conclusioni stantie. Un sistema simbolico dovrebbe caricare pacchetti di conoscenza espliciti con identificatori di origine, digesti, intervalli effettivi, giurisdizioni, politiche di conflitto e livelli di garanzia. Il pacchetto può essere curato, recuperato o assistito da modello, ma le sue affermazioni accettate sono artefatti esterni. Un pacchetto legale può scadere. Un pacchetto scientifico può essere sostituito. Un pacchetto di mondi immaginario può deliberatamente contraddire la realtà attuale.

I conflitti tra le fonti non dovrebbero essere cancellati attraverso il voto a maggioranza a meno che il compito non definisca tale politica. Il sistema potrebbe aver bisogno di ragionamenti paraconsistenti, priorità di origine, versione temporale o mondi separati. Un'affermazione può essere sostenuta da una fonte e contraddetta da un'altra. Il risultato canonico dovrebbe preservare sia che quello che la politica, se del caso, li risolve. Ciò è essenziale per le indagini scientifiche e l'analisi normativa, in cui il disaccordo è l'informazione piuttosto che il rumore.

Le ipotesi di fondo devono essere visibili. I benchmark logici spesso presumono che i nomi denotino individui distinti o che le categorie siano disgiunti. I documenti reali raramente indicano tutti questi assiomi. Un LLM può inserirli perché rendono il puzzle risolvibile. In un sistema di ricerca o di conformità, tali ipotesi dovrebbero apparire in un livello di autorità separato. La risposta finale può quindi affermare che una conclusione segue solo se l'ipotesi regge. Ciò impedisce la conoscenza implicita del mondo di mascherarsi da prove documentali.

La copertura è la fondazione dell'assenza. Per concludere che un documento non contiene un'approvazione richiesta, il sistema deve sapere cosa è stato cercato, quali sezioni e canali sono stati inclusi, come è stato riconosciuto il concetto e se il metodo di ricerca è stato completato per il reclamo. Una scansione lessicale può essere completa per una frase esatta, ma non per parafrasi. Il recupero può essere di richiamo elevato, ma non il mondo chiuso. Un registro autorizzato dall'uomo può essere completo per un rilascio. La copertura dovrebbe essere un oggetto di prima classe consumato da regole negative.

L'architettura nllAgent contiene una versione importante di questo principio. LongTextJS registra la copertura e le lacune insieme alle osservazioni. La compatibilità del circuito verifica se il programma effettivo soddisfa lo stato e la copertura richiesti. La conoscenza mancante non è riportata come conformità. Questo è più forte del modello comune di chiedere a un LLM di restituire vero, falso o non trovato, perché non trovato viene interpretato in relazione a un dominio di ricerca documentato [AssistOS-AI 2026].

La difficoltà rimanente è come convalidare le osservazioni prodotte da modelli. La convalida dello schema è necessaria ma debole. L'ancoraggio delle prove impedisce le citazioni inventate ma non l'interpretazione errata. I punteggi di calibrazione aiutano a classificare i candidati, ma non stabiliscono il significato. L'approccio pratico più forte combina segnali indipendenti: parser di linguaggio controllato per frammenti trattabili; insiemi di modelli o stimoli diversi; vincoli di ontologia; coerenza incrociata; mutazioni semantiche; esempi negativi; revisione umana per distinzioni ad alto impatto; e controesempi a valle che espongono le conseguenze.

Anche la fondazione dovrebbe essere guidata dalle query. Un sistema non deve risolvere ogni entità o evento in un documento di 300 pagine. Dovrebbe derivare la richiesta semantica dalla regola in corso, recuperare gli scopi rilevanti e materializzare solo i tipi necessari. Tuttavia, l'elaborazione query-first non deve confondere la pertinenza con la completezza. Se una regola chiede se esiste un'eccezione in qualsiasi parte del testo dell'autorità, il recupero deve fornire un argomento di copertura o la risposta rimane sconosciuta. L'esecuzione basata sulla domanda riduce il lavoro; non giustifica la chiusura prematura.

La conclusione fondamentale è severa ma costruttiva. Non esiste un rimedio puramente simbolico per un simbolo attaccato al referente sbagliato. La fondazione non è un dettaglio preliminare NLP; fa parte del calcolo attendibile. Il sistema deve esporre la catena dalla fonte al simbolo, preservare alternative, tracciare la copertura e rifiutare l'esecuzione incompatibile. Una volta disciplinata la catena, gli strumenti formali maturi possono aggiungere una sostanziale affidabilità. Senza di essa, gli strumenti si limitano a certificare le conseguenze di una finzione plausibile.

5.2 Pragmatica, Non Monotonicità, Scala, Sicurezza e Fallimenti della Valutazione

Anche una rappresentazione ben fondata può fallire perché il linguaggio naturale fa più che dichiarare i fatti monotoni. I relatori implicano, presuppongono, riconsiderano, citono, scherzano, negoziano e si affidano al contesto istituzionale. I documenti contengono eccezioni, priorità, definizioni che cambiano in seguito e affermazioni la cui forza dipende dal genere. Il linguaggio naturale eseguibile deve decidere quale di questi fenomeni intende sostenere e deve tornare sconosciuto al di fuori di tale ambito.

La pragmatica è il primo confine. Una frase può comunicare più delle sue condizioni di verità letterale. “Il rapporto è quasi completo” può implicare che non è completo. “Alcuni test sono passati” possono suggerire pragmaticamente che non tutti lo hanno fatto, anche se la logica classica non lo comporta. L'ironia inverte il sentimento apparente. Le richieste indirette utilizzano le dichiarazioni come azioni. Un formalizzatore che tratta le inferenze pragmatiche come i fatti possono esagerare; uno che li ignora può perdere l'intento pratico dell'autore. Il trattamento appropriato dipende dall'applicazione. Un motore di conformità dovrebbe di solito privilegiare l'autorità esplicita ed etichettare l'interpretazione pragmatica come proposto. Un agente conversazionale può aver bisogno di modelli pragmatici più ricchi, ma non dovrebbe rappresentarli come prova concreta.

Il presupposto crea una sfida correlata. “Il comitato ha smesso di rivedere la proposta” presuppone che l'avesse riveduta. Le descrizioni definite spesso presuppongono l'esistenza o l'unicità. I documenti possono sfruttare tali costruzioni senza affermare esplicitamente lo sfondo. Un sistema formale deve decidere se i presupposti entrano come ipotesi, inferenze defeasibili o requisiti di chiarimento. Affermarli automaticamente può essere pericoloso; ignorarli può rendere il discorso incoerente. Il sistema di stato deve preservare la loro origine.

Le regole in lingua naturale sono spesso non monotone. Le nuove informazioni possono ritrattare una conclusione. “I dipendenti possono accedere al sistema” possono essere sconfitti da “a meno che la loro certificazione non sia scaduta”. La logica predefinita, la programmazione del set di risposte, le logiche defeasibili e le regole prioritarie forniscono meccanismi formali per le eccezioni. La logica classica del primo ordine può codificare esplicitamente le eccezioni, ma i grandi set di regole diventano difficili da gestire e le informazioni incoerenti possono banalizzare l'inferenza se si applica l'esplosione ordinaria.

Un runtime pratico dovrebbe supportare la semantica a più regole. Monotone Datalog è appropriato quando i fatti si accumulano solo. Le regole defeasibili sono appropriate per inadempienze ed eccezioni. Il ragionamento paraconsistente è appropriato quando le fonti sono in conflitto e il sistema deve evitare di derivare tutto. Le tabelle delle decisioni sono appropriate quando le combinazioni di politiche sono finite ed esplicite. La semantica scelta deve essere in modalità con il pacchetto di regole; non si può dedurre ad hoc dallo LLM durante l'esecuzione.

I sistemi normativi aggiungono autorità e priorità. Una regola successiva può prevalere su una precedente. Un regolamento di livello superiore può anticipare una procedura locale. La competenza e la data effettiva sono importanti. Permessi e obblighi possono essere in conflitto. I paradossi deontici mostrano che le ingenue codifiche logiche possono produrre risultati controintuitivi. Molti casi pratici sono meglio gestiti da priorità esplicite delle regole, eccezioni e condizioni di violazione che da un calcolo deontico universale. Il linguaggio naturale eseguibile dovrebbe favorire la chiarezza operativa rispetto alla grandezza teorica.

La scala crea diverse esplosioni interagenti. I documenti lunghi contengono molte lunghezze e osservazioni dei candidati. Menzioni ambigue creano incarichi di identità combinatoria. Le relazioni temporali creano alternative di ordine parziale. Le regole creano conseguenze derivate. L'esecuzione simbolica crea dei percorsi. Solvers affrontano sistemi di costrizione di grandi dimensioni. LLM finestre e costi del contesto impongono limiti separati. Un'architettura ingenua che formalizza ogni frase in ogni logica possibile è impossibile.

La materializzazione guidata dalla domanda è la prima risposta. I circuiti dichiarano ciò che consumano. Il runtime indicizza la struttura e le informazioni lessicali stabili, quindi produce semantica più ricca solo per scopi rilevanti. Il calcolo incrementale riutilizza gli artefatti invariati. L'interpretazione astratta riassume le alternative. Percorsi di controllo simbolico di fusione e confinato. I portafogli del riassolutore instradano semplici casi a motori economici. I bilanci duri producono risultati incompleti espliciti piuttosto che troncamento nascosto.

La seconda risposta è la modularità semantica. Un documento può essere analizzato attraverso le visualizzazioni: sezioni, finestre temporali, giurisdizioni, scene narrative, stadi sperimentali o livelli di autorità. Le conseguenze dell'identità e delle regole possono essere locali in una visione. Le relazioni trasversali sono introdotte solo quando richiesto. Ciò riduce la fusione globale accidentale e supporta il parallelismo. Il sistema dovrebbe ancora registrare quando una conclusione dipende dal presupposto che una vista sia completa.

La terza risposta è la precisione. Un passaggio strutturale o lessicale poco costoso elimina molti casi. Un pass di estrazione a modello che produce candidati per i restanti. L'analisi astratta identifica le potenziali violazioni. La raffinatezza simbolica gestisce solo le ambiguità rilevanti per il risultato. La revisione umana è riservata alle scelte irrisolte ad alto impatto. Questo assomiglia a condutture statico-analisi che utilizzano analisi economiche prima che dimostrino il teorema costoso.

Le minacce alla sicurezza sorgono in ogni fase. La rapida iniezione nei documenti di partenza può istruire LLM di ignorare gli schemi o fabbricare i risultati. Il traduttore dovrebbe ricevere il contenuto della fonte come dati sotto una richiesta specifica per il ruolo e il suo output dovrebbe essere convalidato in modo indipendente. La fonte non deve mai ottenere l'autorità per modificare il runtime, il pacchetto di regole o la politica di pubblicazione. Qualsiasi testo che afferma di essere un'istruzione di sistema è ancora contenuto di documenti.

Il codice generato è un rischio ancora maggiore. Una DSL che consente JavaScript arbitrario, Python, importa o accesso alla rete trasforma l'apprendimento semantico in esecuzione di codice. I circuiti dichiarativi dovrebbero normalizzare ai dati semplici e chiamare solo gli operatori registrati. Gli algoritmi personalizzati dovrebbero essere installati come estensioni rive con contratti digitati, effetti, limiti di risorse e digest bloccati. La sabbiatura rimane necessaria, ma la restrizione della capacità è più forte che fare affidamento solo su un limite di processo.

Gli attacchi del sovrintendenti includono la negazione del servizio attraverso vincoli patologici, oggetti a prova malformata e sfruttamento delle librerie native. Sono richiesti budget, restrizioni teoriche, controllo delle prove e pinning della versione. Un modello può deliberatamente o accidentalmente costruire una formula tecnicamente valida ma computazionalmente esplosiva. La selezione del formalismo dovrebbe stimare i costi e rifiutare i compiti al di fuori della politica.

L'avvelenamento da dati riguarda i produttori semantici appresi e le suite di riferimento. Se un agente di codifica genera sia un circuito che gli esempi che lo convalidano, gli artefatti possono condividere lo stesso malinteso. La generazione indipendente di test, le mutazioni semantiche, i casi di ancoraggio curati manualmente e i ruoli di revisione separati riducono questo rischio. L'architettura di apprendimento nllAgent impedisce correttamente a un agente di codifica di pubblicare il proprio candidato. La pubblicazione manuale è un controllo della governance, sebbene la sua efficacia dipendano dalla qualità dei parametri di riferimento e dai controlli indipendenti.

La realizzazione in linguaggio naturale può reintrodurre rivendicazioni non supportate. Un modello può convertire “possibile sotto un'unica interpretazione” in “il documento viola la regola”. Può omettere ipotesi o trasformare una ricerca incompleta in assenza. La realizzazione dovrebbe quindi essere limitata da uno schema di risultati canonici e controllata a livello di rivendicazione. Le uscite ad alto impatto possono utilizzare modelli controllati. I commenti esplicativi dovrebbero essere chiaramente separati da risultati verificati.

La valutazione è forse il fallimento più sistemico. Molti benchmark misurano solo l'accuratezza della risposta finale. Ciò confonde l'estrazione, la formalizzazione, il ragionamento e il rendering. Un modello può raggiungere la risposta corretta per la ragione sbagliata. Un sistema formale può essere penalizzato per la restituzione sconosciuta laddove il benchmark presuppone una convenzione non dichiarata. I set di dati sintetici possono premiare il riconoscimento del modello. Le metriche LLM-as-a-judge possono condividere i pregiudizi con i sistemi in fase di valutazione.

Un benchmark rigoroso dovrebbe contenere più livelli. Il livello di origine include testo e struttura esatti. Il livello semantico include una o più interpretazioni ammissibili con evidenze. Il livello formale include programmi target o test di equivalenza di estensione. Il livello di esecuzione include modelli attesi, prove, testimoni o nuclei insoddisfatti. Il livello di risultato include stati canonici e copertura. Il livello di realizzazione include reclami consentiti e esagerazioni proibite. Gli errori possono quindi essere attribuiti piuttosto che crollare in un unico punteggio.

Le coppie minime sono essenziali. Cambiare una parola o una relazione dovrebbe produrre una differenza formale prevedibile. Le famiglie di parafrasi testano l'invarianza. Le sezioni Distractor prova il recupero. Le collisioni di identità testano il test. Le prove mancanti testano un comportamento sconosciuto. Le fonti contrastanti testano la paraconsistenza o la politica di autorità. I casi di contorno temporale testano calendari e intervalli. Test di testo di testo contraddittori universali di isolamento di sicurezza. Le mutazioni semantiche testano se un circuito implementa effettivamente la sua regola.

La valutazione dovrebbe anche misurare l'astensione. Un sistema affidabile deve sapere quando manca di un'ontologia, di un produttore, di una copertura o di un budget. Sconosciuto non è un fallimento quando la domanda non è supportata. Le metriche dovrebbero distinguere l'astensione giustificata da quella evitabile. Le curve di qualità di copertura possono mostrare come la precisione e la falsa garanzia cambiano man mano che il sistema ammette più semantica proposta dal modello.

Questioni di attuazione indipendenti. Un compilatore di circuiti e il suo valutatore “ombra” non dovrebbero condividere ogni funzione critica se si afferma il test differenziale. I punteggi di mutazione dovrebbero provenire da mutazioni eseguibili, non da descrizioni in prosa. I profili del modello in tempo reale devono essere valutati separatamente dagli infissi deterministici. La

pianificazione e la realizzazione richiedono benchmark specifici per attività piuttosto che essere dedotti dalle prestazioni di audit. Questi sono proprio alcuni dei gravi problemi riconosciuti nel repository nllAgent [AssistOS-AI 2026].

Le recensioni esterne dovrebbero essere interpretate con una cura simile. L'indagine Frontiers 2026 è peer reviewed e utile, ma la sua affermazione che i sistemi federativi funzionano meglio è esplicitamente qualificata dalla confusione dei compiti. Le indagini di pianificazione e analisi dei programmi mappano le tendenze, ma non possono stabilire che un'architettura proposta lavori su documenti senza restrizioni. Non è stata trovata alcuna recensione indipendente di nllAgent. Un libro responsabile dovrebbe quindi utilizzare le revisioni per identificare il consenso e il disaccordo, quindi indicare i propri giudizi laddove le prove sono assenti.

Il principio di valutazione centrale è l'attribuzione causale. Quando un sistema migliora, i ricercatori dovrebbero determinare se il guadagno è venuto da una migliore analisi semantica, correzione del risolutore, recupero, campionamento del tempo di test o perdita di benchmark. Quando fallisce, dovrebbero identificare il livello. Non si tratta solo di igiene scientifica. I dati di errore a strati sono ciò che consente all'architettura di migliorare senza sostituire l'intero sistema o nascondere le regressioni in punteggi aggregati.

Il linguaggio naturale eseguibile rimarrà una capacità limitata. Pragmatica, conoscenza open-world, contesto sociale e istituzioni in evoluzione assicurano che nessun nucleo simbolico finito cogli tutto il significato. L'obiettivo non è quello di rimuovere questo confine, ma renderlo esplicito e utile. Un sistema può offrire forti garanzie per i frammenti strutturali, relazionali, temporali, quantitativi e governati dalle regole, restituendo i risultati condizionali altrove. Il sistema più pericoloso non è quello che ammette i limiti. È uno che converte il linguaggio non modellato in un output simbolico fiducioso e chiama il risultato verificato.

6. Famiglie Architetture per il Linguaggio Naturale Eseguitibile

6.1 Progetti Concorrenti: Rappresentazioni Universali, Portafogli di Formalismi, Compilazione Controllata e Sistemi Orientati alla Query

Le architetture in linguaggio eseguibile possono essere organizzate in base al luogo in cui si collocano l'impegno semantico. Alcuni tentano di tradurre ogni input in una rappresentazione universale. Altri mantengono diverse forme candidate e selezionano un formalismo per compito. Alcuni limitano il linguaggio di input fino a quando la compilazione non è deterministica. Altri conservano il linguaggio aperto, ma formalizzano solo le osservazioni richieste da una domanda. Ogni progetto risolve un problema diverso e un sistema maturo probabilmente li combinerà piuttosto che selezionarne uno esclusivamente.

La rappresentazione intermedia universale è l'architettura più intuitiva. Il testo viene compilato in un ricco grafico semantico o linguaggio logico, e ogni operazione a valle query o trasforma quell'oggetto comune. I vantaggi sono l'unità concettuale, il riutilizzo e la possibilità di controlli di coerenza globale. Una singola rappresentazione può supportare più applicazioni e creare un'interfaccia stabile tra NLP e il ragionamento.

La sua debolezza è l'eccesso semantico. Per servire ogni compito, la rappresentazione deve coprire entità, eventi, discorso, tempo, modalità, quantità, causalità, causelità, norme, incertezza, provenienza e concetti specifici del dominio. O il linguaggio diventa estremamente complesso o semplifica le distinzioni di cui ha bisogno alcune attività. Una rappresentazione universale crea anche un unico punto di fallimento: una parabola globale sbagliata contamina ogni analisi. La pressione per scegliere presto un'identità, un'ambito o un'ontologia può nascondere l'ambiguità.

Un perno semantico è una variante più modesta. Non pretende di essere una forma logica completa. Standardizza la lunghezza della fonte, le menzioni, le ipotesi dell'entità, le osservazioni, i mondi, i valori portanti nel tempo, la modalità, la provenienza, le alternative, la copertura e le lacune. I compilatori specializzati riducono le porzioni selezionate in obiettivi formali. Questo conserva l'infrastruttura comune senza fingere che una semantica di esecuzione si adatti a tutto il significato. LongTextJS, come documentato in nllAgent, è vicino a questa idea: è un mondo di origine indirizzabile e programma di osservazione piuttosto che un linguaggio di dimostrazione del teorema universale [AssistOS-AI 2026].

L'architettura a candidati multipli tratta la formalizzazione come ricerca. Un LLM produce diverse forme logiche, programmi o grafici semantici. I controlli e i vincoli di ontologia eliminano i candidati non validi. L'esecuzione, gli esempi e le controesempi classificano o distinguono i sopravvissuti. Il sistema può restituire una conclusione obbligata quando tutti i candidati sono d'accordo e una conclusione può essere conclusa quando solo alcuni lo fanno. Questa architettura è appropriata quando l'ambiguità è genuina o quando l'affidabilità di formalizzazione è limitata.

Il suo costo è combinatorio. I moduli candidati si moltiplicano attraverso frasi e scelte di identità. Un ingenuo cross-prodotto diventa impossibile. La soluzione è quella di rappresentare la struttura condivisa e le alternative locali in modo compatto, utilizzare domini astratti per

riassumerli e perfezionare solo le scelte rilevanti per i risultati. La fiducia può guidare l'esplorazione, ma i candidati a bassa probabilità non dovrebbero essere rimossi da un'analisi solida a meno che la politica non accetti esplicitamente l'incompletezza probabilistica.

La compilazione del linguaggio controllato adotta l'approccio opposto. L'input viene riscritto in un linguaggio naturale ristretto la cui grammatica e semantica sono deterministiche o altamente limitate. Gli utenti possono autorizzare direttamente nella lingua controllata, o un LLM può proporre una parafrasi controllata per la conferma. Una volta accettata, la compilazione è affidabile. Questa architettura è particolarmente forte per i requisiti, i contratti, le procedure e l'autoring delle regole.

La limitazione è la copertura e l'adozione. I documenti ordinari contengono costruzioni al di fuori del frammento controllato. La scrittura può cambiare significato. Gli utenti possono approvare una parafrasi senza notare una sottile differenza. L'architettura funziona meglio quando la forma controllata viene visualizzata accanto alla fonte, le differenze sono localizzate e i controesempi formali rendono concrete le scelte importanti. Il linguaggio controllato dovrebbe essere un punto di controllo semantico esplicito, non una normalizzazione invisibile.

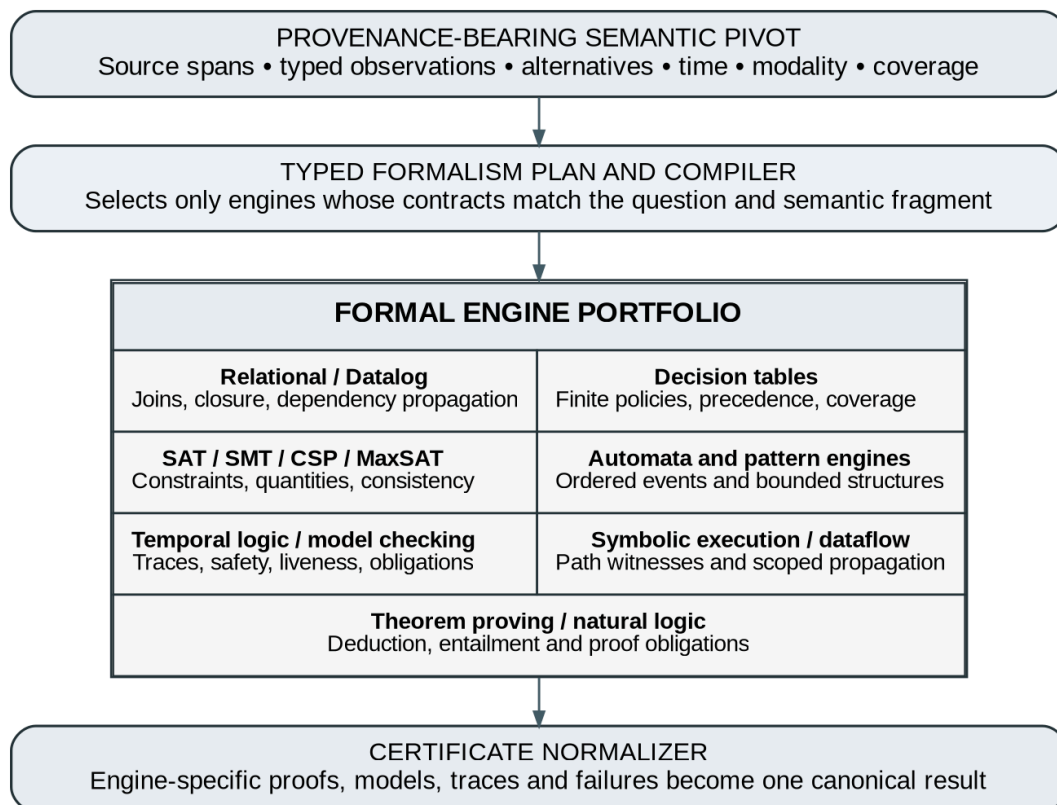
Le architetture query-first evitano la formalizzazione totale. Un circuito o una domanda dichiara di quali osservazioni e relazioni ha bisogno. Il runtime preserva la fonte strutturalmente, recupera le regioni rilevanti e si materializza solo i tipi semantici richiesti. Le query relazionali, le tabelle e le regole decisionali operano sul mondo finito risultante. Questo disegno scala meglio di documenti lunghi e allinea il calcolo con lo scopo.

Una visione dei sistemi di database di database strettamente correlati distingue gli operatori semantici interpretati da quelli compilati. Nel design interpretato, un LLM viene invocato all'interno del ciclo di elaborazione dei dati per ogni riga, record o coppia di candidati. Nel design compilato, LLM viene invocato durante una fase di costruzione separata per tradurre l'operazione semantica in codice locale deterministico che può essere eseguito ripetutamente senza una chiamata modello. Gli esperimenti preliminari hanno riportato sostanziali riduzioni dell'eccesso di latenza e dell'uso del modello, pur mantenendo gran parte della qualità di uscita [Dong and Wang 2026]. La proposta è importante perché raggiunge in modo indipendente la stessa intuizione architettonica sviluppata in questo libro: il modello linguistico è più utile come compilatore o costruttore di piani, mentre il runtime ripetuto dovrebbe essere stabile e ispezionabile. La limitazione è altrettanto istruttiva. Il codice generato può approssimare l'operatore definito dal modello in modo efficiente senza fornire una prova semantica che cattura il concetto previsto dall'utente. La compilazione rimuove la stocalismo per punto; non risolve da sola la fondazione o la fedeltà delle specifiche.

Il rischio è una falsa chiusura. Il recupero può perdere le prove pertinenti e l'analisi della domanda può omettere un concetto necessario. I sistemi di query-first devono tenere traccia della copertura e dei produttori. Se una conclusione negativa richiede la ricerca dell'intero documento per qualsiasi eccezione, alcuni passaggi recuperati sono insufficienti. L'architettura dovrebbe distinguere un risultato di query limitato da una garanzia a livello del mondo chiuso. I contratti di compatibilità e copertura di nllAgent sono un meccanismo importante in questa famiglia.

Un'architettura monoformalista riduce tutti i compiti in una logica, spesso la logica di primo ordine, Datalog o SMT. Semplifica gli obblighi di attuazione e di prova. Per un dominio ristretto, questa può essere la scelta migliore. Logic-LM e LINC ottengono gran parte del loro valore da una lingua di destinazione stabile. Un sistema di programmazione può ragionevolmente

utilizzare un linguaggio di vincolo. Un verificatore di protocollo può utilizzare un calcolo di processo.



La figura 3. Un perno semantico di provenienza può essere compilato in un portafoglio di motori formali specializzati e normalizzato in un unico risultato canonico.

Per le grandi attività in linguaggio naturale, un portafoglio di solutori è più plausibile. La valutazione relazionale gestisce le query dei documenti. Datalog calcola la chiusura monotono. Le tabelle delle decisioni esprimono politiche limitate. SAT e SMT gestiscono vincoli e aritmetici. Il controllo del modello gestisce il comportamento temporale. L'esecuzione simbolica costruisce testimoni. La logica naturale gestisce alcune relazioni di coinvolgimento vicino al linguaggio superficiale. Automi riconoscono i modelli ordinati. Il perno semantico fornisce atomi a terra e un piano di formalismo seleziona uno o più motori.

Il portafoglio crea nuovi obblighi. Il router deve conoscere le capacità espressive e operative di ciascun motore. Un compilatore deve preservare il significato attraverso gli abbassamenti. I risultati di diversi motori hanno bisogno di uno status comune e un modello di provenienza. Se due motori ragionano sui predicati sovrapposti, la coerenza deve essere controllata o lasciata esplicitamente aperta. I criteri di risorse devono impedire a un modello di selezionare un risolutore inutilmente costoso.

Un utile contratto di formalismo indica i tipi di input, gli operatori supportati, la semantica, la copertura richiesta, la classe di garanzia, il certificato atteso, il modello di costo e le limitazioni note. Un motore temporale-logico può richiedere eventi a terra e un sistema di transizione finito. Un motore SMT può supportare funzioni reali aritmetiche intere ma non senza restrizioni. Una tabella decisionale può richiedere condizioni enumerate finite. Un motore naturale-logico può supportare la monotonia ma non la ricorsitazione incrociata arbitraria. Il router può quindi respingere un piano proposto i cui requisiti superano il motore.

La generazione di pro-carico è un'altra famiglia di architettura. Il manufatto primario non è la risposta naturale, ma un risultato canonico più una prova, un testimone, una traccia di esecuzione o una derivazione verificabile. Il modello linguistico genera una spiegazione da questo artefatto. Ogni rivendicazione esplicativa è legata a un elemento di risultato. Un controllo indipendente garantisce che la spiegazione non superi il contenuto formale.

Questo design è prezioso perché la generazione e la verifica hanno diversi punti di forza. Un LLM può scrivere una chiara spiegazione di un vincolo controesterno o insoddisfatto. Un risultato simbolico può garantire che lo scenario sia reale sotto il modello accettato. La spiegazione può ancora essere fuorviante se nasconde ipotesi o incertezza, quindi il risultato canonico deve includere tali dimensioni e il contratto di resa deve richiederne quando materiale.

Le architetture astratti-semriche posizionano un livello di interpretazione astratta prima dell'esecuzione specializzata. Le osservazioni dei candidati popolano e devono essere domini. La propagazione a buon mercato determina quali proprietà sono sicuramente soddisfatte, sicuramente impossibili o potenzialmente violate. Solo le potenziali violazioni sono ridotte in costosi problemi simbolici. Ciò riduce il carico del solvitore e fornisce stati di principio sconosciuti. Crea anche un'architettura incrementale naturale per le modifiche dei documenti.

Le architetture guidate da Counterexample chiudono il cerchio. Una prova fallita o una testimonianza spuria non è solo un errore; identifica quale distinzione semantica conta. Il sistema mappa i fatti del risolutore per reperire, chiede allo LLM o all'utente di perfezionare l'interpretazione pertinente e riproduce l'analisi. Questa architettura è particolarmente promettente perché trasforma il feedback formale in una comprensione mirata del linguaggio piuttosto che in un'auto-riflessione generica.

Le architetture di apprendimento regolano come nuove regole, ontologie e circuiti entrano in produzione. Un estremo consente a un agente LLM di generare codice eseguibile durante ogni esecuzione. Questo massimizza la flessibilità e minimizza la garanzia. L'estremo opposto consente solo regole scritte a mano e formalmente verificate. Un mezzo pratico utilizza agenti di codifica nella messa in scena degli spazi di lavoro, genera parametri di riferimento e mutazioni, limita gli artefatti alle forme dichiarative e richiede una pubblicazione esplicita. Il runtime di produzione legge emissioni immutabili. Questo è il modello di governance implementato in nllAgent e dovrebbe essere generalizzato.

Il seguente confronto riassume il trade-off di base senza affermare che un progetto domina ogni compito.

Tabella 1. Principali famiglie di architettura e loro impegni essenziali.

Architettura di architettura	Forza essenziale e rischio principale
Rappresentazione semantica universale	Riutilizzo e coerenza globale; rischi di una supercomplessità e un impegno semantico prematuro.
Perno semantico più abbassamenti specializzati	Preserva prove comuni e supporta più motori; richiede un'attenta compilazione e contratti tra cross-engine.
For formalizzazioni multiple dei candidati	Rappresenta l'ambiguità e consente le conclusioni di maggio / deve essere; rischia la crescita combinatoria.
Compilazione del linguaggio controllato	Forti semantici e forme ispezionabili dall'utente; sacrifica la copertura e possono introdurre cambiamenti significativi durante la riscrittura.
La prima materializzazione della query-primo	Scale di documenti lunghi e esigenze specifiche delle attività; può confondere il recupero con completezza a meno che la copertura non sia esplicita.

Architettura di architettura	Forza essenziale e rischio principale
Obiettivo unico formale	Semplice base di fiducia e valutazione in un dominio limitato; scarsamente adatto per fenomeni linguistici eterogenei.
Il portafoglio di risabit	Corrisponde al metodo computazionale alla struttura dei problemi; aggiunge obblighi di routing, interoperabilità e normalizzazione dei certificati.
Generazione di pro-portamento	Conclude le risposte in linguaggio naturale ai contenuti verificati; richiede un robusto controllo della realizzazione a livello di reclamo.
Analisi astratta e raffinatezza simbolica	Bilancia e precisione; la solidità rimane condizionata alla copertura dell'interpretazione.
Circuiti Governati esilici	Permette l'evoluzione senza auto-modifiche del run-time; dipende da forti benchmark, test di mutazione e revisione della pubblicazione.

L'architettura generale più credibile è un ibrido tra le opzioni centrali. Utilizza un perno semantico di provenienza, preserva le alternative in cui sono importanti, esegue la materializzazione guidata dalle query, si dirige verso un portafoglio di formalismo e accetta solo risultati proof o supportati da testimoni. Il linguaggio controllato può fungere da punto di controllo interattivo per formalizzazioni ad alto impatto. L'interpretazione astratta controlla la scala, mentre la raffinatezza contro-guide esemplificativa migliora la precisione. L'apprendimento rimane fuori dalla produzione.

Questa conclusione non è solo teorica. Si riflette nella letteratura empirica. PAL, SatLM, Logic-LM, LINC, traduttori temporali-logica, formalizzatori di pianificazione, SymGPT e sistemi di verifica certificati hanno tutti successo restringendo il target formale. Nessun risultato dimostra che una logica universale o un modello end-to-end può sostituire il portafoglio. L'architettura dovrebbe quindi abbracciare l'eterogeneità riducendo al minimo le interfacce attendibili tra i componenti.

6.2 Una Pipeline di Riferimento dal Testo ai Simboli, all'Esecuzione e a Risposte in Linguaggio Naturale Vincolato

Una pipeline di riferimento in lingua eseguibile dovrebbe essere intesa come un toolchain di compilatorio e verifica piuttosto che come un prompt agente. Ogni fase materializza un artefatto immutabile o rivestito di contenuti. Una corsa può essere riattivata senza chiedere al modello di ricostruire il suo ragionamento precedente. Gli errori sono assegnati alle fasi. L'apprendimento può migliorare un produttore o una teoria senza cambiare silenziosamente il significato dei vecchi risultati.

Il primo artefatto è il pacchetto sorgente. Contiene il testo canonico o i dati, il digest di revisione, il linguaggio, la struttura, i canali e le campate stabili. I paragrafi, le intestazioni, le tabelle, i preventivi e i blocchi di codice conservano l'ordine e l'identità. I record esterni hanno chiavi stabili. Il pacchetto sorgente è l'autorità lessicale: stabilisce ciò che era presente, non ciò che significa.

Il secondo artefatto è il contratto di attività. Afferma la domanda, il rilascio delle regole, l'ambito di applicazione della fonte, la garanzia desiderata, le ipotesi consentite, il mondo e la giurisdizione, i bilanci e la politica di output. Il compito determina la richiesta semantica. Una richiesta di trovare acronimi definiti richiede osservazioni diverse da una richiesta per verificare la conformità temporale. Il contratto definisce anche se l'assenza può essere conclusa e quali forme di incertezza richiedono una revisione umana.

Il terzo stadio costruisce un grafico di prova. I produttori deterministici aggiungono osservazioni strutturali. I parser di linguaggio controllato aggiungono osservazioni per i frammenti che riconoscono. I produttori sostenuti da LLM aggiungono eventi candidati, relazioni, identità, modalità o quantità di candidati legati agli schemi. Ogni osservazione registra ancora, produttore, modello o versione dell'algoritmo, stato, alternative e copertura. Il modello non scrive direttamente nel grafo canonico; un materialista ricontrolla, tipi e integrità referenziale.

L'identità e la gestione del mondo avvengono nello stesso livello ma non vengono risolte silenziosamente. Le menzioni sono prove locali. Le ipotesi di entità raccolgono menzioni sotto un mondo e una portata. I candidati all'identità affermano le stesse relazioni diverse o irrisolte con il sostegno. Le interpretazioni reciprocamente esclusive appartengono a mondi separati o alternative protette. Il grafico può quindi rappresentare disaccordo senza corrompere la coerenza globale.

Il quarto stadio calcola uno stato semantico astratto. Le astrazioni specifiche del dominio riassumono le identità possibili, gli ordini temporali, gli stati normativi, le quantità e i livelli di evidenza. Devono conservare i fatti in tutti i mondi ammessi; possono avere fatti in almeno uno. Ignoti registra un gap piuttosto che un'ipotesi probabilistica. Lo stato astratto supporta la propagazione delle regole a buon mercato e identifica quali domande richiedono una maggiore precisione.

Il quinto stadio produce un piano di formalismo. Il pianificatore non è una decisione LLM a forma libera. Si confronta il compito con i contratti di formalismo registrati. Una semplice query di documenti potrebbe rimanere relazionale. Una politica può compilare tabelle decisionali e vincoli. Una procedura può essere compilata in un sistema di transizione. Una violazione dipendente dal percorso può invocare l'esecuzione simbolica. Un'affermazione deduttiva può invocare un testorema. I nomi dei piani richiedevano ingressi, abbassamenti, versioni del motore, limiti di risorse e certificati attesi.

LLM può proporre un piano perché riconosce la struttura dei problemi e il linguaggio di dominio. Un controllo deterministico convalida che il piano utilizza formalismi supportati, che ogni osservazione richiesta ha un produttore compatibile e che non viene persa alcuna distinzione semantica essenziale per il compito. Ad esempio, una regola che coinvolge eccezioni e priorità non può essere compilata in un motore monotono senza una codifica esplicita. Una regola che coinvolge l'identità non può procedere se il dominio dell'identità è irrisolto e il verdetto dipende da esso.

Il sesto stadio esegue la compilazione digitata. Questo compilatore dovrebbe essere piccolo, deterministico e pesantemente testato. Converte gli oggetti semantici in variabili risoltrici, relazioni, transizioni o predicati. Le mappe della fonte collegano ogni elemento formale alle osservazioni e alle campate. I compilatori dovrebbero generare obblighi di prova quando un abbassamento è dannoso o dipende da ipotesi. I compilatori diversi possono essere confrontati attraverso mutazioni semantiche e modelli distintivi.

La settima fase è l'esecuzione formale. Chiusure di calcolo relative e Datalog. Le tabelle decisionali identificano le file corrispondenti e conflittuali. SAT/SMT/CSP risolutori trovano i modelli o dimostrano la fattibilità sotto le loro teorie supportate. Le selezionatrici modello valutano le proprietà temporali. Gli esecutori simbolici esplorano le condizioni del percorso e ritornano testimoni concreti. Gli assistenti di prova o prover del teorema producono derivazioni. L'uscita del motore non è ancora un verdetto rivolto all'utente; è un record di esecuzione che porta un certificato.

L'ottava fase è la verifica del replay. Un verificatore separato controlla il testimone o la prova contro il problema formale legato alla fonte e la teoria pubblicata. Per una scoperta di documenti, conferma che le osservazioni citate esistono, i periodi corrispondono alla revisione della fonte, gli stati soddisfano la regola e la traccia di esecuzione supporta il reclamo. Per un risultato negativo, conferma che la copertura e la completezza sono sufficienti. Il replay è prezioso anche quando il solutore sottostante è attendibile perché convalida la connessione tra l'output del candidato e la politica rilasciata.

Se l'analisi astratta prevedeva una violazione ma l'esecuzione simbolica non può rendersene conto, inizia il perfezionamento semantico. Il sistema estrae il nucleo insoddisfacente, i predicati contrastanti o il fallimento della prova. Le mappe sorgente identificano le specifiche e le scelte semantiche. Lo LLM riceve un compito limitato: riconsiderare questo collegamento di identità, ambito temporale, mappatura delle eccezioni o allineamento predicato. Propone i candidati rivisti; il materialista e il compilatore li riconvalidano; si ripete l'esecuzione. La revisione umana è richiesta quando la distinzione è ad alto impatto o rimane irrisolta.

Questo ciclo dovrebbe essere monotono rispetto alle prove, non solo alla fiducia. Un perfezionamento può dividere un'entità, restringere un intervallo di tempo, aggiungere un'eccezione trascurata o contrassegnare una premessa non supportata. Non dovrebbe aggiornare un'osservazione proposta ai fatti perché il nuovo risultato è conveniente. Ogni cambiamento viene registrato come una nuova revisione semantica. I percorsi precedenti rimangono riproducibili.

Il nono manufatto è un risultato canonico. Contiene la regola o l'identità di query, lo stato accettato, le ipotesi, il mosto e le conclusioni, i candidati respinti, le incognite, la copertura, le prove di origine, i riferimenti alla prova o la testimonianza e i limiti di esecuzione. Può anche

includere un conciso grafico di spiegazione verificabile della macchina. Questo manufatto è autorevole su qualsiasi resa in prosa.

La decima fase realizza il linguaggio naturale. Il renderizzatore riceve solo il risultato canonico e estratti di origine selezionati. Non replica il ragionamento originale. Ogni frase generata è associata a una o più richieste di risultato. Un checker confronta la frase con le richieste consentite, garantisce che le modalità e l'incertezza siano preservate e rifiuta le aggiunte non supportate. L'output può distinguere risultati, ipotesi, interpretazioni e raccomandazioni.

Ad esempio, la risposta corretta potrebbe essere: “Secondo l'interpretazione che entrambi i riferimenti al Consiglio denotano lo stesso corpo, la regola è violata perché l'approvazione precede la revisione della sicurezza registrata. Il documento non stabilisce se i riferimenti denotano lo stesso organismo; pertanto il risultato complessivo è necessario rivedere. Un renderizzatore fluente ma non sicuro potrebbe omettere la condizione e annunciare una violazione. Il risultato canonico impedisce questa semplificazione.

La pipeline supporta anche la generazione, non solo linting. Un utente fornisce un'idea o un obiettivo. I circuiti di pianificazione costruiscono una specifica controllata con testimoni da regola a piano. Un LLM realizza una bozza. I circuiti di controllo analizzano la bozza utilizzando lo stesso percorso di prova e verifica. La revisione continua fino a quando l'audit canonico soddisfa la politica di rilascio o il bilancio è esaurito. Il modello di generazione non diventa mai l'oracolo finale per sapere se il proprio testo soddisfa le regole.

La governance circonda la toolchain. Ontologie, produttori, circuiti, compilatori, contratti di formalismo e renderizzatori hanno versioni e digeri. Gli agenti di apprendimento operano in aree di stadiaggio usa e getta. La pubblicazione richiede parametri di riferimento, mutazioni semantiche, controlli di compatibilità e approvazione esplicita. La produzione legge emissioni immutabili. I profili del modello live sono valutati separatamente perché gli infissi deterministici non stabiliscono la qualità del fornitore. I pacchetti di conoscenza del mondo attuale hanno date e giurisdizioni efficaci.

L'architettura dovrebbe esporre una gerarchia di garanzie. Le garanzie strutturali riguardano intervalli esatti, l'ordine e la validità dello schema. Le garanzie di fondazione riguardano l'allineamento delle prove e lo stato di identità. L'esecuzione garantisce la preoccupazione delle conseguenze di un problema formale. Le garanzie end-to-end li combinano in modo condizionato. L'utente dovrebbe essere in grado di vedere il livello più debole. Un risultato derivato da eventi proposti da modelli e da una transizione meccanicamente verificata non dovrebbe essere etichettato semplicemente come “verificato”; dovrebbe indicare che l'esecuzione è verificata sulle osservazioni proposte.

Un piccolo kernel di fiducia è un obiettivo di progettazione importante. Il kernel include la canonicalizzazione del sorgente, il controllo dello schema e del tipo, i compilatori formali, gli esecutori registrati o le verificatrici, i verificatori di replay e la convalida canonica-risultato. Le istruzioni di LLM, i generatori di candidati, l'euristica di recupero e i modelli di spiegazione rimangono all'esterno. I componenti attendibili possono ancora contenere bug, ma il loro comportamento è stabile, verificabile e revisore. Si tratta di un miglioramento sostanziale rispetto alla fiducia in un modello di fondazione in evoluzione come ragionatore monolitico.

Il gasdotto proposto non è l'unica possibile attuazione, ma coglie gli obblighi rivelati dalla letteratura. Spiega perché un risolutore è necessario ma insufficiente. Dà interpretazione

astratta e ruoli simbolici distinti. Supporta più lingue formali senza abbandonare un livello di prova comune. Permette LLM creatività nella scoperta, riparazione e spiegazione, preservando l'autorità simbolica sulla base e sui risultati accettati. Ancora più importante, trasforma il fallimento in un artefatto chiamato – un tipo non supportato, un'identità non risolta, una copertura incompleta, una perdita di compilazione, una formalizzazione insoddisfatta o una sentenza senza licenza – piuttosto che una vaga allucinazione.

7. LongTextJS, CircuitJS e nllAgent: Un Caso di Studio Critico

Il repository pubblico NaturalLanguageLinterAgent è uno dei pochi tentativi di rendere esplicito il confine tra l'interpretazione del linguaggio e l'esecuzione delle regole in un'implementazione eseguibile. Il progetto non è un ragionatore generale in linguaggio naturale e la sua documentazione non lo presenta come uno. È una libreria Node.js 22+ ESM runtime e run-line con due modalità di prodotto in lingua controllata rispetto allo stesso LongTextJS-CircuitJS. La modalità di audit applica una teoria delle versioni a un documento Markdown esistente ed emette un prodotto di audit canonico. La modalità di specificazione applica le teorie di pianificazione a un'idea di alto livello ed emette un piano di generazione verificato; un modello linguistico opzionale può realizzare quel piano come prosa, ma i circuiti di controllo invariati rimangono l'oracolo di accettazione [AssistOS-AI 2026].

Il repository è utile come caso di studio perché incarna la tesi centrale di questo libro in forma di software: i modelli possono proporre osservazioni semantiche o prosa, ma l'autorità di produzione è assegnata a rilasci immutabili, controlli di compatibilità espliciti, esecuzione a circuito limitato, verificatori e artefatti canonici. Il progetto mantiene anche un registro pubblico di questioni serie. Questa pratica è importante perché una grande superficie della documentazione può altrimenti far sembrare le intenzioni architettoniche come garanzie implementate. La discussione qui sotto distingue ciò che è operativamente stabilito da ciò che rimane un'ipotesi di ricerca.

Nessuna valutazione peer-reviewed indipendente o revisione tecnica di terze parti di nllAgent, LongTextJS o CircuitJS è stata localizzata attraverso ricerche dei nomi del progetto, nome del repository, URL della documentazione e combinazioni con "recensione", "valutazione", "benchmark" e "replica" a partire dal 2 agosto 2026. I risultati della ricerca hanno portato al progetto stesso piuttosto che a una valutazione esterna. La sentenza in questo capitolo è quindi la revisione architettonica indipendente dell'autore, basata sul repository pubblico, sulla documentazione, sugli esempi eseguibili, sul processo di rilascio dichiarato e sull'attuale fascicolo di emissione. L'assenza di una revisione esterna non è né lode né critica; significa che le affermazioni del progetto dovrebbero rimanere restrittive fino a quando non vengono riprodotte in modo indipendente.

7.1 Che cosa Implementa l'Architettura Pubblica e Perché i suoi Confini Contano

Il sistema definisce tre attività produttive che sono deliberatamente separate. L'audit della produzione legge un rilascio immutabile pubblicato, compila un documento in LongTextJS, esegue i circuiti di convalida del rilascio, verifica i risultati dei candidati e persiste un risultato canonico CNL / Audit-1 con un rapporto renderizzato. Le specifiche di produzione compilano un'idea nelle osservazioni LongTextJS, applicano circuiti di pianificazione e produce un manufatto canonico CNL / Plan-1; la realizzazione opzionale può quindi generare Markdown e presentarlo allo stesso macchinario di controllo. L'apprendimento avviene in uno spazio di lavoro di staging usa e getta. Un agente di codifica può generare artefatti e benchmark di authoring candidati, ma i controlli lato host ammettono solo i file in whitelist e un manutentore deve pubblicare e attivare esplicitamente una versione. Ciò rende l'apprendimento analogo allo sviluppo del software governato piuttosto che all'auto-modificazione del tempo di esecuzione.

L'unità di lavoro più importante non è il rapporto finale, ma la directory di esecuzione. Un audit preserva l'input copiato, il programma sorgente, le basi selezionate, i risultati di compatibilità e copertura, la traccia del circuito, i risultati dei verificatori, i risultati, il rapporto leggibile dalla macchina canonica e la vista leggibile dall'uomo. La pianificazione esegue la conservazione dei corrispondenti artefatti del piano. Una corsa che si ferma perché manca la conoscenza registra ancora il fallimento e può creare un problema riutilizzabile. La regola del design secondo cui la conoscenza mancante non viene mai segnalata come conformità è fondamentale. Impedisce un fallimento comune degli agenti dei documenti: tradurre “Non ho trovato prove” in “il documento soddisfa la regola”.

LongTextJS dovrebbe essere inteso come un programma sorgente solo in un senso vincolato. Non assegna comportamenti imperativi a frasi arbitrarie. Compila una revisione della sorgente fissa in un mondo finito e interrogabile di strutture e osservazioni. Le proprietà preziose sono l'identità di origine, intervalli stabili, tipi di osservazione delle versioni, l'identità del produttore, la copertura, le lacune esplicite e la rigiocabilità. Un manufatto sul lato documentale può essere eseguito perché i consumatori registrati possono interrogare e trasformare queste relazioni sotto semantica dichiarata; è più vicino a una rappresentazione intermedia semantica o a un programma di caricamento del database piuttosto che a un codice applicativo senza restrizioni.

Questa distinzione è più che terminologia. Quando un modello traduce una frase, il suo output non dovrebbe diventare autorevole semplicemente perché è serializzato in un linguaggio di programmazione. Il testo di origine rimane un'evidenza; l'osservazione tradotta registra un'interpretazione di tale evidenza, generata da un produttore nominato sotto uno schema e un profilo modello. Il circuito rimane una teoria a parte. Una scoperta diventa pubblicabile solo dopo che un verificatore verifica le condizioni previste dal rilascio. Questi strati impediscono a un parser di diventare silenziosamente un giudice e impediscono a una spiegazione di diventare l'unica registrazione di una deduzione.

L'attuale pacchetto di fondazione illustra l'ambito semantico previsto. I comandi di audit, piano e benchmark ordinari selezionano una base-core delimitata. Il pacchetto riconosce le forme documentate-inglese controllate per lo stato, il tipo, l'esatta precedenza temporale espressa da asserzioni “prima”, aritmetica esatta, quantità e asserzioni di emozioni letterali. Cinque circuiti ri-verificati controllano solo gli invarianti logici, temporali, aritmetici, elementari e di

emozione/tipo. La confezione e il digest selezionati sono persistiti. Cambiare i fatti politici, geografici, sociali, economici e giuridici è intenzionalmente escluso dalla verità di base senza tempo. Questo è un confine diffamabile del modello mondiale. Un sistema che codifica i fatti attuali senza date, giurisdizioni, fonti e politiche di conflitto effettive non può offrire un'esecuzione semantica riproducibile.

Il sistema supporta la materializzazione deterministica e assistita da modelli. La selezione della traduzione può preferire un agente modello di linguaggio configurato AchillesAgentLib, utilizzare l'attuale adattatore di codifica-agente vincolato o funzionare senza un modello per gli agenti deterministici. Ad ogni chiamata modello viene assegnata un'abilità specifica del ruolo piuttosto che l'intero spazio di lavoro dell'agente. Se un'osservazione critica non può essere materializzata all'interno del budget richiesto per lo schema e le risorse, il risultato è incompatibile, incompleto o esaurito. Il modello non controlla la costruzione, la politica di pubblicazione o il significato di un verificatore. Questa è proprio la direzione corretta per un sistema neuro-simbolico federativo: il modello è potente al limite, mentre gli artefatti simbolici e l'accettazione della politica dell'ospite.

La copertura è trattata come un obbligo eseguibile piuttosto che un punteggio di fiducia informale. La compilazione del circuito funziona a ritroso da nodi di portata di uscita per determinare quali osservazioni esterne possono influenzare un risultato. I registri della pubblicazione stipulano contratti che collegano tali richieste ai produttori disponibili. La compatibilità del run-time chiede quindi se il documento concreto abbia effettivamente prodotto osservazioni con il tipo, lo stato, la cardinalità, la struttura, il canale e la copertura. La capacità dichiarata di un produttore non è confusa con la prova che il documento attuale è stato elaborato con successo. Quando una richiesta critica non è soddisfatta, la corsa si ferma a un confine chiamato.

Questo design orientato alla domanda è adatto a documenti lunghi. Evita di chiedere a un modello di produrre un'interpretazione completa di ogni frase prima che qualsiasi domanda sia nota. Un circuito può richiedere le dimensioni semantiche di cui ha bisogno: termini esatti, asserzioni di stato, quantità, relazioni temporali o osservazioni di dominio. Il compilatore può materializzare solo le osservazioni rilevanti per l'output e persistere ciò che era e non è stato coperto. Il vantaggio scientifico non è solo a costi inferiori. Limita gli impegni semantici non necessari e rende visibile la formalizzazione incompleta.

CircuitJS è la rappresentazione dal lato teorico. I moduli di circuito modificabili per agenti sono limitati a un'espressione dichiarativa, normalizzata ai dati semplici e passati attraverso lo stesso compilatore utilizzato per le descrizioni dei circuiti non in codice. Possono invocare solo operatori e verificatori registrati; non possono nascondere il comportamento eseguibile arbitrario all'interno di una regola appresa. Un'applicazione host può installare un'estensione runtime esaminata contenente algoritmi reali, ma gli input e il contesto sono copie di dati semplici congelati, i risultati devono essere dati semplici, l'identità della cache include il digest dell'estensione e le versioni che dipendono dal blocco di estensione che digerisce. L'interfaccia standard della riga di comando non accetta percorsi di estensione eseguibili arbitrari.

Questo limite affronta uno dei rischi pratici più gravi nei sistemi di regole "abbedito all'agente". Se un agente di codifica può generare JavaScript arbitrario in ogni circuito, l'apprendimento di un linter diventa generazione di codice a distanza e il comportamento semantico di una versione diventa difficile da ispezionare. Limitare la creazione ordinaria di circuiti alle operazioni registrate crea un vocabolario operativo finito. Gli algoritmi specializzati

rimangono possibili, ma entrano attraverso un percorso di ingegneria, revisione e versione separata. L'architettura distingue quindi l'evoluzione delle teorie dall'evoluzione del runtime attendibile.

Il limite di verifica è altrettanto importante. L'esecuzione del circuito può creare risultati dei candidati, ma un verificatore controlla se tali candidati entrano nell'audit o nel piano canonico. Negli esempi delimitati, il verificatore può rileggere gli ancoraggi di origine, le osservazioni, rilasciare i metadati e la politica applicata. Non è sufficiente che un nodo di esecuzione emetta un oggetto i cui campi sembrano un avvertimento. Il verificatore stabilisce la proprietà specifica prevista dal prodotto. Questo non dimostra automaticamente la corretta interpretazione semantica, ma impedisce l'uscita accidentale o dannosa del circuito mascherato come una scoperta accettata.

La modalità di specificazione dimostra come la stessa architettura possa supportare la generazione controllata senza concedere l'autorità al modello dello scrittore. I circuiti di pianificazione costruiscono un piano canonico e richiedono testimoni di regole per le regole applicate. La realizzazione facoltativa trasforma il piano in prosa. La prosa candidata passa quindi attraverso la modalità di audit e il modello di realizzazione non giudica la propria conformità. Questo è più forte della generazione limitata solo pronta perché l'oggetto autorevole è il piano e la traccia di audit, non l'adesione auto-riferita del modello.

Anche il sottoinsieme sperimentale query-first è significativo. Canonical LongTextJS è esposto come un mondo di query di sola lettura finito. Le query digitate e i valori delle tabelle di prova-aware vengono abbassati nell'esecuzione ordinaria CircuitJS. I controlli del compilatore hanno supportato relazioni, espressioni, anti-join, criteri di ordinazione e hit; i record di pubblicazione di query sui contratti e sulle mappe di origine; i benchmark confrontano la valutazione di riferimento diretta con l'esecuzione ridotta. L'attuale pubblicazione editoriale di produzione rimane volutamente più semplice, mentre gli aggregati, i modelli ordinati, gli indici nativi e il ragionamento avanzato sono gestiti attraverso operatori diretti o grafici. Questa moderazione è preferibile a rivendicare un linguaggio di query universale maturo prima che la semantica e l'ottimizzazione siano stabili.

Il modello di pubblicazione del repository rafforza la riproducibilità. Gli agenti nominati possiedono materiale di autorità separato, schemi, profili di estrazione, circuiti, suite di benchmark, candidati, rilasci, corse, problemi e feedback. Un agente di codifica può proporre candidati ma non può renderli attivi. La produzione legge un rilascio immutabile. Questa struttura consente di riprodurre quale teoria, fondamento, profilo del produttore, digest di estensione e verificatore ha generato un risultato. Consente inoltre a diversi agenti di codifica o famiglie modello di costruire teorie concorrenti senza condividere silenziosamente un'ontologia o un rilascio attivo.

L'affermazione più forte implementata è quindi architettonica piuttosto che semantica: nllAgent ha un percorso disciplinato dalla fonte immutabile alle osservazioni di applicazione, dalla richiesta di osservazione alla compatibilità, dai circuiti limitati alla pubblicazione controllata dai verracidi e da un prodotto canonico a un rapporto leggibile dall'uomo. Ha degli Stati di stop esplicito. Non ha bisogno di far finta che ogni documento sia ontologicamente compatibile. Tratta i modelli come proposte di ruolo. Si tratta di risultati sostanziali rispetto alle condutture convenzionali LLM-agent, in cui il prompt, l'interpretazione, il ragionamento e la spiegazione spesso esistono solo in un contesto transitorio.

L'architettura, tuttavia, non stabilisce che il linguaggio naturale arbitrario è stato correttamente fondato. Le campate stabili dimostrano che le prove esistono in un luogo; non dimostrano che un attore, un evento, un'eccezione o una modalità siano stati interpretati correttamente. Un'osservazione digitata può ancora riferirsi all'entità sbagliata. Una corsa completa può essere completa solo per il contratto di osservazione limitato, non per ogni significato rilevante che un essere umano potrebbe dedurre. La documentazione e il registro di emissione del progetto sono adeguatamente cauti su questo punto. Il valore dell'implementazione è che queste limitazioni hanno luoghi da rappresentare piuttosto che essere nascosti all'interno di una risposta finale.

7.2 Valutazione Indipendente: Punti di Forza, Problemi Seri Attuali ed Evoluzione Necessaria

La mia valutazione complessiva è positiva sui confini della fiducia e cauta sulle affermazioni semantiche. nllAgent è più di un involucro rapido, più di una pipeline vettoriale e più di una raccolta di schemi JSON. I suoi manufatti dal lato della fonte e dalla teoria, la pubblicazione immutabile, il confine di authoring limitato, la verifica della riproduzione, i controlli di copertura e i prodotti canonici costituiscono un substrato di ricerca credibile. L'architettura è particolarmente ben allineata con il modello federativo identificato nelle recenti recensioni neuro-simboliche: la componente neurale propone o realizza il linguaggio, mentre la componente simbolica conserva la propria identità, semantica e criteri di accettazione [Chatzikyriakidis and Lappin 2026].

L'evidenza attuale del progetto supporta tuttavia la fattibilità limitata, non il linguaggio naturale eseguibile generale. Il fondotinta predefinito riconosce un piccolo frammento controllato. La dimostrazione editoriale si avvale di operatori e verificatori ristretti. La compilation Query-first è sperimentale. La realizzazione controllata è limitata. Nessuna valutazione indipendente ha stabilito la fedeltà di formalizzazione, la risoluzione dell'identità, il trasferimento tra domini, la calibrazione degli errori o la superiorità rispetto alle linee di base più semplici. La giusta strategia di pubblicazione è quindi quella di indicare esattamente quali proprietà sono attuate e di allegare ogni rivendicazione più forte a una dotazione di riferimento e garanzia.

L'attuale registro dei problemi seri identifica sei limitazioni ingegneristiche e semantiche. Il primo è l'ecceduralizzazione del programmatore. Il runtime ha utili meccanismi di dipendenza e caching, ma non rivendica ancora un motore di correzione incrementale completamente generale per circuiti arbitrari, relazioni ricorsive, mondi alternativi e aggiornamenti non monotoni. Questo è importante per i documenti lunghi perché una piccola modifica dovrebbe invalidare solo gli artefatti semantici e i circuiti che dipendono da esso. Un replay completo ingenuo è corretto ma costoso; uno scheduler incrementale non sano è veloce ma pericoloso. La soluzione richiede contratti espliciti di monotonia ed effetto, provenienza della dipendenza, separazione epocale per le retrazioni e confronti di riferimento tra esecuzione incrementale e piena esecuzione.

Il secondo problema è che il supporto di interpretazione non scorre ancora uniformemente attraverso ogni percorso di esecuzione. Un documento può contenere letture, mondi, contesti o condizioni di presenza concorrenti, ma un operatore, una chiave della cache, un'aggregazione o un verificatore può accidentalmente farli collassare. Questo è l'ostacolo centrale per trattare LongTextJS come uno stato semantico astratto. Ogni valore che dipende da un'interpretazione deve portare il suo contesto. I digiuntori devono combinare contesti compatibili. L'aggregazione deve dire se si estende all'interno di un mondo o attraverso alternative. L'identità della cache deve includere la formula del contesto. Gli operatori che non possono preservare le alternative dovrebbero essere dichiarati incompatibili piuttosto che sceglierne in silenzio uno.

Questo problema non è un piccolo gap di funzionalità. La promessa scientifica di base del linguaggio naturale eseguibile è spesso che il sistema può dire ciò che è vero sotto tutte le letture ammissibili, ciò che è possibile sotto almeno uno, e quale risultato dipende da un'interpretazione contestata. Se le alternative vengono perse in un percorso di esecuzione interno, un'interfaccia di maggio/must lucida sarebbe fuorviante. Il progetto dovrebbe quindi rendere la conservazione dell'interpretazione un invariante termine intero e testarlo attraverso mondi finiti generati, proprietà metamorfiche e riproduzioni differenziali.

Il terzo problema è la compatibilità conservativa per il comportamento procedurale opaco. Le estensioni e gli operatori registrati sono necessari perché alcuni algoritmi utili sono imbarazzanti per esprimere in modo dichiarativo. Ma un compilatore non può dedurre precise richieste semantiche, effetti, monotono, utilizzo delle risorse o obblighi di testimonianza da JavaScript opaco. Il rifiuto conservatore è più sicuro dell'accettazione ottimistica, ma l'eccessiva opacità riduce la composabilità e rende meno efficace la materializzazione basata sulla domanda. Ogni primitivo procedurale ha bisogno di un contratto di proprietà del registro che dichiari i tipi di input e output, stato epistemico accettato, requisiti di copertura, possibili effetti, determinismo, monotono, comportamento della cache, classe di costo e testimone leggibile dal verificatore. Il contratto dovrebbe essere più autorevole dei commenti scritti dall'autore di estensione.

Il quarto problema è la pianificazione minima delle query e l'indicizzazione temporale. Un mondo di query finito può essere eseguito correttamente senza un'ottimizzazione sofisticata, ma l'uso della scala documentale richiederà unire la pianificazione, le stime di selettività, gli indici di apertura e entità, gli indici di intervallo temporale e la manutenzione incrementale. Queste ottimizzazioni non sono solo dettagli sulle prestazioni. Un pianificatore che riscrive le query deve preservare la molteplicità di prove, le formule di contesto, l'ordinazione, la semantica anti-unita e la copertura. Gli indici nativi devono rimanere acceleratori usa e getta piuttosto che fonti di verità. I test differenziali dovrebbero confrontare la valutazione ottimizzata e di riferimento rispetto ai mondi generati, in particolare per le query di assenza, la sovrapposizione temporale, le interpretazioni alternative e le prove duplicate.

Il quinto problema è la generazione controllata deliberatamente ristretta. La realizzazione esatta o controllata di risultati e piani limitati è preziosa, ma non equivale ancora a una prosa di lunga durata verificata. La generazione a lungo termine introduce aggregazione, pronomi, ellissi, enfasi retorica, relazioni di discorso e implicazioni create dall'ordine delle frasi. Una realizzazione può contenere solo dichiarazioni atomiche autorizzate e implicare ancora una narrazione causale o normativa non supportata. Il passo successivo dovrebbe essere la realizzazione di prove piuttosto che uno scrittore non vincolato più ampio. Un risultato canonico dovrebbe fornire identificatori di rivendicazione, entità e slot per quantità, modalità, incertezza e relazioni di discorso consentite. Il testo generato dovrebbe essere riparato e confrontato con tali affermazioni. I disallineamenti critici dovrebbero respingere il testo; il materiale stilistico al di fuori del frammento controllato dovrebbe essere etichettato a un livello di garanzia inferiore.

Il sesto problema è il supporto di mutazione specifico per scenario. L'accuratezza ordinaria del parametro di riferimento può nascondere regole che sono inerti, ridondanti o non trattate. Un circuito può passare tutti gli esempi curati anche se un'eccezione viene ignorata perché nessun caso lo attiva. Una valutazione grave dovrebbe generare mutanti semantici dattilomati che cambiano soglie, comparatori, quantificatori, negazione, modalità, legami di identità, ordine temporale, stato di prova, predicati di copertura, ambito di eccezione, priorità delle regole e rotte dei verificatori. Una suite di test "uccide" un mutante quando il risultato canonico atteso cambia nel modo richiesto. I mutanti sopravvissuti rivelano test deboli o clausole semanticamente ridondanti. La mutazione dovrebbe coprire i programmi di origine, le teorie dei circuiti, gli abbassamenti dei compilatori e i vincoli di realizzazione, non solo le stringhe negli esempi.

Questi sei problemi sono più informativi di un'etichetta generica "prototipo" perché identificano dove l'architettura corrente può perdere sia la scala che il significato. Suggestiscono anche un percorso coerente di evoluzione. La prima pietra miliare dovrebbe essere una semantica formale limitata per il mondo dello store semantico e della query. Non ha bisogno di

formalizzare l'inglese senza restrizioni. Dovrebbe definire mondi di origine immutabili, identità di osservazione, stato epistemico, copertura, alternative, transazioni, risultati di query, pubblicazione e ritrattazione. Le invarianti chiave possono essere controllate con modelli rispetto alle istanze finite e testate in termini di proprietà nell'implementazione.

La seconda pietra miliare dovrebbe essere l'esecuzione basata sull'interpretazione. Le condizioni di presenza o i contesti fattorizzati dovrebbero accompagnare osservazioni alternative-dipendenti. Lo scheduler, il motore di query, il registro primitivo, la cache, il verificatore e i prodotti canonici dovrebbero preservare tali contesti o dichiarare incompatibilità. Può, non deve, e non può, e non può, essere calcolato da insiemi espliciti o astrazioni di mondi ammissibili, non dalla fiducia modello. Ciò creerebbe il ponte concreto dall'architettura esistente all'interpretazione astratta.

La terza pietra miliare dovrebbe essere un registro di formalismo piuttosto che uno CircuitJS in espansione indefinitamente. CircuitJS è meglio usato come linguaggio di orchestrazione, disco di prova e pubblicazione. I motori specializzati dovrebbero possedere la semantica specializzata: valutazione relazionale e Datalog per la chiusura e l'adesione; tabelle decisionali per la precedenza politica; SAT, SMT, CSP o MaxSAT per vincoli; automi per modelli ordinati; solutori temporali e pedine per tracce; esecuzione simbolica per i testimoni del percorso; framework di flusso di dati per la propagazione a raggi; prover del teorema o di prova. Ogni adattatore dovrebbe dichiarare un frammento di input digitato, ipotesi di traduzione, certificato di output, correttore di riproduzione e modalità di guasto.

La quarta pietra miliare dovrebbe essere la raffinatezza semantica controverta e guidata. Un'analisi astratta grossolana può identificare una possibile violazione. Un risolutore o un esecutore simbolico cerca quindi una testimonianza coerente. Se il testimone è soddisfacibile e rigiocabile, il sistema lo segnala con la sorgente. Se è spurio, un nucleo insoddisfacente o un insieme di conflitti dovrebbe identificare quale distinzione di identità, temporale, modale, quantità, eccezione o copertura è stata persa. La richiesta di perfezionamento dovrebbe riguardare solo la fonte e il produttore in questione. Il manufatto semantico riparato diventa una nuova versione e il controesempio diventa un caso di regressione. Questo ciclo sarebbe una sintesi genuinamente nuova della proposta semantica basata su LLM, interpretazione astratta ed esecuzione simbolica.

La quinta pietra miliare dovrebbe essere uno strato di fondazione più forte. La sicurezza del tipo e le ancore di origine sono necessarie ma insufficienti. Il runtime ha bisogno di set di identità dei candidati, vincoli espliciti / diversi, validità temporale, controlli di unità e giurisdizione, autorità di origine e politiche di conflitto. I collegamenti top-one generati dal modello non dovrebbero unire in modo distruttivo gli oggetti. Le entità critiche dovrebbero utilizzare identificatori autorevoli o conferme umane. La conoscenza esterna dovrebbe entrare attraverso pacchetti di sola lettura, con pacchetti con contenuti indirizzati con fonti, intervalli efficaci, giurisdizione, scadenza, risoluzione dei conflitti e massimali di garanzia. Il recupero del Web può proporre gli aggiornamenti dei pacchetti, ma non dovrebbe diventare implicita verità senza tempo.

La sesta pietra miliare dovrebbe essere una valutazione indipendente. Gli infissi deterministici stabiliscono la riproducibilità del software, ma non la qualità della traduzione semantica dal vivo. Un profilo di valutazione opt-in dovrebbe individuare il modello, il fornitore, le istruzioni, le versioni degli strumenti, la temperatura e la politica di acquisizione; ripetere i casi per misurare la varianza; il costo record e la latenza; e vietare il fallback silenzioso. Un

valutatore di riferimento implementato separatamente dovrebbe confrontare i risultati delle query e dei circuiti, ove possibile. L'annotazione umana dovrebbe concentrarsi sulla fedeltà fonte-semantiche e sulle ambiguità rilevanti per i risultati. Le linee di base dovrebbero includere il giudizio diretto LLM, LLM plus retrieval, LLM-to-one-formalismo, compilazione del linguaggio controllato e sistemi di pattern deterministici. Il progetto dovrebbe dimostrare quando la sua architettura aggiuntiva produce valore misurabile.

Una valutazione utile può essere riassunta senza gonfiare le affermazioni. La conservazione della fonte, i rilasci immutabili, la persistenza della traccia, la separazione delle osservazioni dei documenti dalla teoria riutilizzabile, la copertura esplicita, l'autoring dei circuiti ristretti, la pubblicazione controllata dai verificatori e l'apprendimento governato sono decisioni di progettazione forti. Rimangono aperte la comprensione semantica generale, l'interpretazione-completa esecuzione, la pianificazione delle query su scala di produzione, la generazione di lunga durata, la mutazione completa e la convalida empirica indipendente. La credibilità dell'architettura deriva in parte dall'ammettere questo confine.

La principale opportunità scientifica non è quella di aggiungere più sintassi DSL. È per dimostrare la solidità semantica condizionale per un frammento limitato ma non banale. Un risultato pubblicabile potrebbe specificare un linguaggio documentale contenente entità, eventi, relazioni temporali, quantità, modalità, eccezioni esplicite e riferimenti alternativi; definire il set di interpretazione ammessa; dimostrare o testare meccanicamente che ogni risultato pubblicato può/deve corretto rispetto a tale insieme; mostrare che il perfezionamento simbolico elimina una classe misurabile di avvertimenti spuria; e verificare che le spiegazioni generate conservino il risultato canonico. Un tale risultato sarebbe più stretto dell'“inglese eseguibile”, ma sostanzialmente più importante.

La principale opportunità pratica è una nuova classe di sistemi di documenti e agenti che sono disposti a smettere. Un normale assistente LLM è ottimizzato per continuare a produrre una risposta. Un runtime simbolicamente radicato può restituire un'ontologia incompatibile, un'identità irrisolta, una copertura insufficiente, un formalismo non supportato, l'esaurimento del budget, il controesempio spurio o la realizzazione senza licenza. Questi risultati possono sembrare meno impressionanti in una dimostrazione, ma sono prerequisiti per un uso affidabile nella scienza, nella regolamentazione, nell'ingegneria e nel processo decisionale istituzionale.

Il mio giudizio indipendente è quindi che nllAgent ha scelto molti dei giusti confini architettonici e dovrebbe resistere alla pressione per ampliare le sue affermazioni più velocemente di quanto non ampli la sua evidenza semantica. È già un prototipo credibile di analisi del linguaggio governate, basate sulla fonte, separate dalla teoria. Non è ancora la prova che il linguaggio naturale senza restrizioni possa essere eseguito con garanzie formali. Il suo percorso migliore è quello di diventare una piattaforma sperimentale per l'affermazione più stretta e più difendibile sviluppata in questo libro: LLMs scopre candidati formali, un kernel simbolico che mette a terra e conseguenza, e l'output in linguaggio naturale rimane subordinato a un risultato canonico rigiocabile.

8. Costruire e Validare un Runtime Neuro-Simbolico

8.1 Strategia di Implementazione: un Sistema di Linguaggio Eseguitibile Minimo ma Scientificamente Onesto

L'architettura proposta in questo libro è ambiziosa solo se è descritta vagamente. Una volta che i suoi obblighi sono separati, il primo sistema attuabile è relativamente modesto. Non ha bisogno di comprendere un linguaggio arbitrario, costruire un'ontologia universale, o dimostrare ogni conclusione. Deve accettare una classe di documenti limitata, preservare prove esatte, materializzare una piccola famiglia di osservazioni digitate, compilare una famiglia ristretta di regole in uno o due motori formali, restituire testimoni o incognite esplicite e impedire che lo strato finale di linguaggio naturale cambi il risultato simbolico. Il valore scientifico deriva dal rendere visibile e misurabile ogni limitazione.

La prima decisione di progettazione è la richiesta di riferimento. Un prototipo di ricerca non dovrebbe iniziare con "il sistema comprende ed esegue il linguaggio naturale". Dovrebbe iniziare con un reclamo come: dati documenti di un genere dichiarato, un'ontologia in versione, una famiglia finita di produttori di osservazione e una serie di regole pubblicate, il runtime determina se ogni regola è soddisfatta, violata, potenzialmente violata, non supportata o al di fuori della copertura semantica del rilascio. Il risultato è rigiocabile e collegato alle campagne di origine. Questa affermazione è abbastanza stretta da testare e abbastanza ampia da supportare la revisione della conformità, l'analisi dei requisiti, l'usura dei documenti scientifici, l'ispezione del protocollo e la pianificazione controllata.

Il livello di origine deve essere deliberatamente noioso. Ogni documento è memorizzato come una revisione immutabile con un digest stabile. Le unità strutturali - testamenti, paragrafi, frasi, elementi di elenco, celle di tabella, didascalie, note a piè di pagina e oggetti incorporati - ricevono identificatori persistenti. La normalizzazione viene registrata piuttosto che applicata silenziosamente. Gli offset di caratteri dovrebbero riferirsi a una rappresentazione di origine canonica, mentre le mappature preservano le posizioni nel file originale. Una dichiarazione in una tabella e una frase visivamente identica in un paragrafo possono svolgere diversi ruoli strutturali; tale distinzione deve sopravvivere all'estrazione. Quando l'OCR è inevitabile, l'output OCR e l'immagine della pagina sono oggetti di prova separati e l'incertezza nella trascrizione non viene commercializzata nel testo ordinario.

Questa disciplina di origine non è un sovraccarico amministrativo. È ciò che rende contestabili le conclusioni simboliche. Una scoperta che cita solo un riassunto generato non può essere controllata in modo indipendente. Una scoperta collegata a testi esatti, celle di tabella e record di trasformazione può essere rivista anche se ogni modello linguistico coinvolto è cambiato. Il pacchetto sorgente dovrebbe pertanto essere considerato parte del record di audit attendibile, sebbene gli estrattori che lo creano rimangano fallibili e richiedano test specifici per il formato.

Il secondo strato è un grafico di prova piuttosto che una singola analisi. Il grafico contiene menzioni, entità, eventi, stati, quantità, espressioni temporali, modalità, relazioni e link di origine. Fondamentalmente, rappresenta anche alternative e status. Un collegamento dell'entità proposto da un modello non è memorizzato come uguaglianza incondizionata. È una relazione di identità candidata con un produttore, un punteggio, intervalli di sostegno, prove contrastanti e

uno stato epistemico come proposto, corroborato, accettato, rifiutato o irrisolto. Una relazione temporale dedotta da "presto dopo" non deve essere normalizzata a una durata esatta arbitraria; dovrebbe mantenere la relazione qualitativa e qualsiasi interpretazione del dominio dichiarato.

Questo livello dovrebbe rimanere finito e legato alle attività. Il sistema non sta costruendo un modello mondiale completo. Si sta materializzando le distinzioni richieste dai circuiti pubblicati e dalla query corrente. La materializzazione di Query-first, già esplorata in nllAgent, è la corretta impostazione predefinita. Un circuito dichiara di aver bisogno di identità dell'attore, relazioni di ordine documentale, modalità deontica e date effettive. Il compilatore deriva da queste richieste prima di un'interpretazione costosa. I produttori generano quindi solo le osservazioni necessarie per decidere la compatibilità ed eseguire la regola. Questo riduce i costi e, soprattutto, limita la superficie semantica su cui possono verificarsi errori.

Il terzo strato è un kernel simbolico di fondazione. "Grounding" è spesso usato vagamente per significare che una risposta cita un passaggio. Qui ha un significato più rigoroso: ogni simbolo ammesso all'esecuzione formale ha un riferimento digitato, provenienza, portata e stato sufficiente per il formalismo di destinazione. Un predicato come approvato (x, y, t) non può essere introdotto solo perché un LLM ha prodotto quei token. Il kernel controlla che x e y si riferiscano ai tipi di entità ammissibili, che t è un valore o un intervallo di tempo supportato, che il produttore di osservazione è autorizzato per questo predicato, che l'intervallo di origine esiste nella revisione dichiarata e che lo stato soddisfa il requisito di garanzia della regola.

I contratti di fondazione dovrebbero distinguere le prove dall'ontologia. Le prove dicono che un arco supporta un'osservazione. L'ontologia dice che tipo di cosa denota l'osservazione e quali relazioni sono ammissibili. Gli assiomi del dominio dicono ciò che deriva da queste relazioni. La conoscenza mondiale fornisce affermazioni di provenienza esterna. Questi strati non devono essere fusi. Un registro delle società può stabilire che due nomi denotano un soggetto giuridico durante un periodo indicato. Un'ontologia di dominio può dichiarare che le persone giuridiche possono firmare accordi. Una teoria politica può dichiarare che una firma di un rappresentante autorizzato è considerata approvazione. Un modello può suggerire tutte e tre le connessioni, ma ogni connessione appartiene a un'autorità diversa e dovrebbe essere accettata o rifiutata separatamente.

L'identità merita il proprio servizio perché è la dipendenza nascosta più comune nel ragionamento a lungo documento. Il servizio deve gestire i cluster di menzione, alias espliciti, identificatori, ruoli organizzativi, validità temporale e mondi selezionati. Dovrebbe consentire diversi candidati irrisolti piuttosto che forzare l'unificazione precoce. In una prima versione, l'identità può essere intenzionalmente conservatrice: gli identificatori espliciti e gli alias dichiarati sono accettati; i collegamenti basati sul ruolo e contestuali rimangono proposti; l'identità non supportata non supportata ferma le regole che lo richiedono. Questa politica conservatrice abbasserà la copertura apparente, ma migliorerà notevolmente il significato dei risultati accettati.

Il tempo ha bisogno di un dominio altrettanto esplicito. Una rappresentazione iniziale pratica può combinare istantanei esatti, intervalli chiusi o aperti, ordine qualitativo e confini sconosciuti. Il runtime deve distinguere l'ordine dei documenti dall'ordine degli eventi e dal tempo di pubblicazione dal tempo effettivo. Frasi temporali come "prima del dispiegamento", "entro trenta giorni", "mentre l'autorizzazione rimane valida" e "dopo qualsiasi cambiamento materiale" inducono strutture diverse. Il compilatore dovrebbe respingere una regola temporale se il grafico

delle prove ha solo osservazioni di ordine documentale quando è richiesta la semantica dell'ora dell'evento. La sostituzione silenziosa tra domini temporali è una classica fonte di conclusioni plausibili ma non valide.

La moda e la forza normativa richiedono un piccolo reticolo piuttosto che un booleano. Come minimo, il sistema dovrebbe distinguere l'affermazione, la relazione, la convinzione, la possibilità, la capacità, l'autorizzazione, l'obbligo, il divieto, la raccomandazione, l'intenzione, la condizione e controfattuale. Dovrebbe inoltre registrare il titolare e il beneficiario di una norma, se del caso. Il fornitore può divulgare i dati e "il fornitore deve divulgare i dati" condivide il vocabolario, ma ha conseguenze di conformità opposte. "Il documento afferma che il fornitore è conforme" è la prova di un rapporto, non necessariamente la prova dell'evento sottostante. Un runtime formale diventa utile proprio rifiutando questi crolli.

Una volta basate le osservazioni, il quarto strato calcola uno stato semantico astratto. Un dominio minimo può rappresentare, per ogni proposta rilevante per la query, sia che sia vera in tutte le interpretazioni ammissibili, vere in alcune, false in tutto, contraddette o irrisolte a causa della copertura mancante. L'implementazione non deve iniziare con una sofisticata libreria reticolare. Può iniziare con esplicite alternative finite e un sisso meccanico / deve riassumere. Man mano che la scala cresce, i domini astratti possono riassumere i candidati dell'identità, intervalli numerici, intervalli temporali, stati modali e set di provenienza senza enumerare ogni mondo.

Lo stato astratto svolge due lavori. In primo luogo, fornisce una propagazione conservativa a livello di documento. Se ogni interpretazione accettata collega una definizione a un termine, tale relazione è un fatto. Se solo uno dei tre incarichi di identità plausibili crea un conflitto di interessi, il conflitto è una violazione che può essere trovata piuttosto che una violazione verificata. In secondo luogo, decide dove è giustificata l'esecuzione più costosa. Un risultato sicuro può terminare in anticipo. Un risultato di violenza si innesca una ricerca di testimoni. Uno sconosciuto causato dalla copertura assente del produttore innesca un problema di compatibilità semantica piuttosto che una chiamata risoltrice.

Il quinto strato è un piano di formalismo. Il piano non è in prosa libera scritta da un agente. È una decisione di compilazione digitata che afferma quale domanda verrà posta su quale motore, quali osservazioni sono gli input, cosa garantisce il ritorno del motore e come il suo risultato mappa al vocabolario canonico. La scelta dovrebbe essere guidata dalla forma semantica piuttosto che dalla moda.

La valutazione relazionale è il primo motore da implementare perché molti utili controlli dei documenti si riducono a join, confronti degli ordini, raggruppamenti e differenze impostate. Regole come "ogni termine tecnico definito viene utilizzato in modo coerente", "ogni affermazione accettata ha almeno una fonte", "nessun identificatore di requisiti è duplicato" o "tutti i rischi elencati hanno un proprietario di mitigazione" non richiedono un dimostratore di teoremi. Un nucleo relazionale è trasparente, efficiente e facile da testare. Datalog estende questo nucleo quando sono richieste la chiusura transitive o le dipendenze ricorsive. Gli esempi includono l'ereditarietà, la propagazione parziale, le catene di dipendenza e la raggiungibilità attraverso grafici di citazione o requisiti.

Le tabelle delle decisioni sono il secondo formalismo utile per le politiche vincolate. Una tabella decisionale rende esplicite le combinazioni e i risultati delle condizioni, espone le righe sovrapposte o mancanti e supporta l'analisi della copertura. È particolarmente appropriato per l'ammissibilità, il routing, la classificazione e le regole politiche la cui complessità deriva da

combinazioni piuttosto che da ricorsione. Un circuito può compilare una politica in linguaggio naturale in una tabella dei candidati, ma la pubblicazione dovrebbe richiedere esempi a livello di riga, rilevamento della sovrapposizione, controlli di completezza e approvazione umana della partizione semantica.

I motori SAT, SMT, CSP e MaxSAT diventano utili quando l'attività contiene vincoli di interazione, aritmetica, programmi, assegnazioni o ottimizzazione. Il compilatore deve scegliere le teorie supportate ed evitare di nascondere la semantica non supportata in stringhe non interpretate. Un risultato SMT può dimostrare infeasibilità sotto vincoli codificati, produrre un modello soddisfacente o esporre un nucleo insoddisfacente. Il nucleo dovrebbe essere mappato fino agli obblighi e alle ipotesi di origine. Questa mappatura è più informativa che chiedere a un LLM di spiegare un errore generico del risolutore; identifica i più piccoli impegni formali che non possono reggere congiuntamente.

La logica temporale e il controllo dei modelli sono appropriati per procedure, flussi di lavoro, protocolli e regole del ciclo di vita. Il grafico delle prove fornisce eventi e proposizioni statali; il compilatore costruisce un sistema di transizione e proprietà temporale; il checker restituisce soddisfazione, una traccia controesempio o un risultato inconcludente sotto i limiti. La parte difficile non è il controllo temporale. Sta decidendo se la fonte definisce un sistema di transizione completo, se le transizioni omesse sono impossibili o semplicemente prive di documenti e se l'equità o le ipotesi ambientali sono giustificate. Il risultato canonico deve quindi esporre il limite del modello.

L'esecuzione simbolica è appropriata quando una regola si riferisce a percorsi fattibili attraverso codice eseguibile, una procedura statica o un artefatto semantico simile a un programma. Il suo output dovrebbe essere un testimone concreto: incarichi, condizioni di ramo, transizioni di stato e collegamenti di origine. Nelle impostazioni di sola documento, l'esecuzione simbolica può esplorare legami alternativi o rami procedurali, ma non deve essere invocata solo perché il sistema contiene simboli. Il termine dovrebbe mantenere il suo significato sensibile al percorso. SymGPT è istruttivo perché la regola del linguaggio naturale viene compilata in una condizione di violazione e quindi testata contro i percorsi contrattuali effettivi piuttosto che contro la prosa più generata [Xia et al. 2026].

La logica naturale, gli automi finiti, l'analisi dei grafici, il calcolo semiringhente e i dimostratori del teorema specializzato appartengono al portafoglio quando le loro ipotesi strutturali si adattano. La logica naturale può gestire in modo efficiente la monotonia e alcuni aspetti lessicali vicino al linguaggio superficiale. Automi sono eccellenti per i modelli locali ordinati. I metodi di grafico e semiringimento possono mantenere più analisi con calcolo condiviso. Gli assistenti di prova sono appropriati quando un piccolo numero di sinistri di alto valore giustifica la formalizzazione manuale e le prove controllate dal kernel. L'architettura dovrebbe rendere l'aggiunta di un motore un'estensione digitata, non un invito a incorporare codice eseguibile arbitrario nei circuiti.

Il sesto strato è la raffinatezza semantica controesterno guidata. La scoperta può essere compilata in un problema di fattibilità. Se il risolutore restituisce un testimone, il runtime lo riproduce contro il grafico delle prove e riporta le scelte di interpretazione esatte richieste. Se il percorso è infeasibile, il motore restituisce un nucleo insoddisfacente, preconditione fallita o un set di conflitti. Il sistema traccia che si limita alla compilazione alle osservazioni di origine e alle mappature dei candidati che lo hanno prodotto. Solo allora viene chiesto un LLM di aiutare.

Il prompt di riparazione dovrebbe essere locale e digitato. Può chiedersi se due menzioni siano state erroneamente unificate, se una condizione fosse intesa come necessaria o sufficiente, se si applica un'eccezione o se una relazione temporale ha l'endpoint sbagliato. Dovrebbe presentare la fonte pertinente, i contratti di ontologia, le alternative attuali e il conflitto simbolico. Lo LLM propone uno o più artefatti semantici rivisti. Tali proposte passano attraverso gli stessi controlli di fondazione e compilazione degli originali. Un essere umano può essere inserito quando l'ambiguità è normativa o scientificamente materiale.

Questo processo è fondamentalmente diverso da un generico auto-riflessione. Al modello non viene detto che la sua risposta era sbagliata e invitata a improvvisare un'altra catena di pensiero. È dato un conflitto clinicamente identificato in un manufatto semantico persistente. La riparazione può essere testata in regressione. Se la riparazione cambia un'ontologia o un produttore, la pubblicazione richiede nuovi casi di benchmark e un diff semantico. Il controesempio diventa una parte permanente del corpus di valutazione.

Il settimo strato è il risultato canonico. Dovrebbe essere un prodotto stabile e indipendente dal solutore che separa il verdetto, le prove, le ipotesi, le interpretazioni, l'esecuzione e le limitazioni. Per una regola di audit, registra la versione della regola, l'ambito applicabile, lo stato di compatibilità, la copertura, le osservazioni accettate, le osservazioni rifiutate, il risultato dell'esecuzione, il riferimento di testimonianza o di prova, il risultato del verificatore e le questioni irrisolte. Per un piano, registra obiettivi, vincoli, passaggi, dipendenze, risorse, alternative e i controlli esatti che il piano soddisfa. Il prodotto canonico è l'autorità per ogni successiva risposta in lingua naturale.

Il percorso di ritorno verso il linguaggio deve essere trattato come compilazione, non come libera generazione. Un render riceve un insieme finito di reclami autorizzati. Ogni pretesa ha modalità, polarità, quantità, riferimenti di entità, incertezza e provenienza. Il renderizzatore può scegliere l'ordine del discorso e il fraseggio esplicativo, ma non può introdurre nuove entità, rafforzare il possibile per deve, convertire le prove mancanti in assenza, sopprimere un'ipotesi materiale o inventare una spiegazione causale. Gli identificatori di richiesta a livello di frase dovrebbero rimanere disponibili per l'audit. Un secondo checker dovrebbe confrontare il testo reso con il risultato canonico, utilizzando test deterministici per nomi, quantità, date, negazione e modalità e, se necessario, un modello separato vincolato a un comportamento testuale piuttosto che a una valutazione a risposta aperta.

La realizzazione di Proof-constrained è particolarmente importante perché gli utenti giudicheranno il sistema dalla sua prosa. Una perfetta traccia simbolica può essere compromessa da un riassunto elegante ma impreciso. L'architettura dovrebbe quindi rendere la verbosità facoltativa ma la fedeltà obbligatoria. In modalità ad alta sicurezza, il sistema può generare un linguaggio semplice e ripetitivo perché ogni frase segue un modello concesso in licenza dal tipo di risultato. In modalità esplicativa, un LLM può migliorare la leggibilità, ma le frasi non supportate vengono respinte e rigenerate. Il manufatto canonico, non la prosa, rimane il record della decisione.

La sicurezza e la governance devono essere integrate fin dall'inizio. I documenti in linguaggio naturale sono input non attendibili e possono contenere iniezioni rapide, esempi contraddittori, formattazione fuorviante o frammenti di codice. I produttori semantici dovrebbero ricevere i dati di origine tramite interfacce digitate piuttosto che un prompt di sistema privilegiato che può essere sovrascritto dal contenuto del documento. Gli artefatti formali generati devono essere analizzati da grammatiche restrittive. I sovventori dovrebbero essere sottoposti a limiti di

risorse. Le estensioni di runtime devono essere riviste, le versioni e bloccate. Gli spazi di lavoro di apprendimento non dovrebbero pubblicare direttamente sulla produzione. Questi confini sono già in parte presenti in nllAgent e dovrebbero essere rafforzati piuttosto che bypassati per comodità [AssistOS-AI 2026].

Un programma di attuazione realistico può procedere in cinque rilasci. Il primo rilascio dovrebbe supportare l'ingestione immutabile di Markdown o DOCX, le campate di sorgente, le osservazioni strutturali, un motore di query relazionale, osservazioni limitate-inglese controllate, stati di copertura, circuiti riutilizzabili, verifica replay e risultati canonici. Dovrebbe risolvere compiti limitati come la completezza della provenienza, la coerenza della definizione, la copertura dei requisiti, le regole di sezione ordinate e semplici distinzioni modali. Non dovrebbe essere rivendicata alcuna identità generale, inferenza temporale o recupero del mondo attuale.

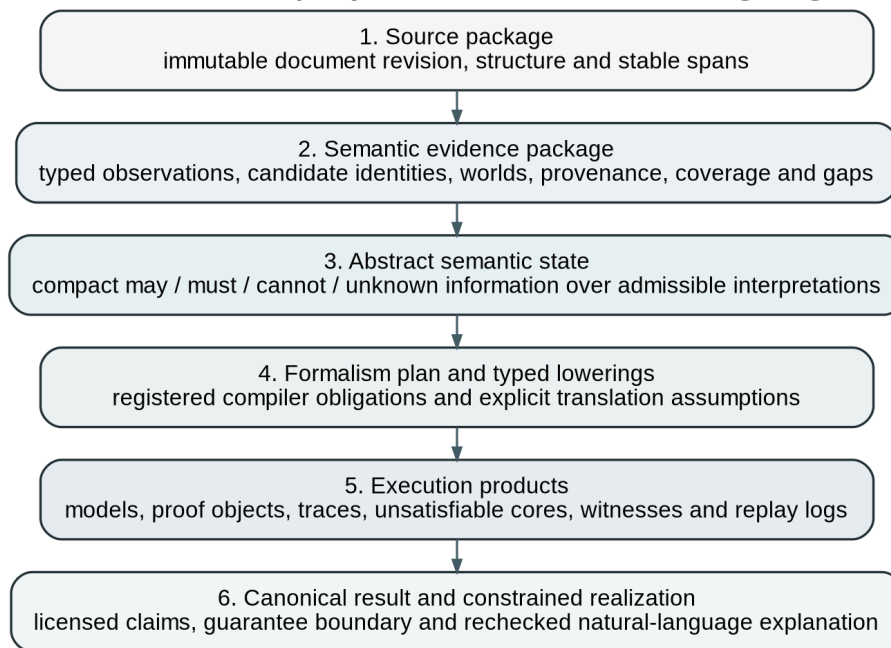
La seconda versione dovrebbe aggiungere identità e tempo digitati. L'identità inizia con identificatori espliciti, alias e ruoli dichiarati da documenti; la semantica temporale inizia con date, intervalli, ordine del documento e un piccolo insieme di modelli di ordine per eventi. I benchmark dovrebbero contenere alias ambigui, ruoli che cambiano, intervalli sovrapposti e collegamenti mancanti deliberatamente. Il sistema dovrebbe fermarsi piuttosto che indovinare quando un circuito richiede un'identità più forte di quella fornita dal rilascio.

La terza release dovrebbe aggiungere tabelle decisionali e esecuzione SMT/CSP, con mappatura insoddisfatta e perfezionamento semantico controesterno guidato. Questo comunicato può affrontare politiche di ammissibilità, programmi, vincoli di assegnazione, coerenza quantitativa e fattibilità della procedura limitata. Ogni risultato del risolutore deve essere riprodotto attraverso un verificatore indipendente o un controllo più piccolo, ove possibile. Il test di mutazione dovrebbe alterare i vincoli e le osservazioni per dimostrare che le suite di riferimento rilevano le regressioni semantiche.

La quarta versione dovrebbe aggiungere un adattatore di controllo temporale/modello e un adattatore di esecuzione simbolico sensibile al percorso per un unico dominio concreto, come le specifiche del flusso di lavoro o le regole smart-contract. Lo scopo non è un'ampia copertura, ma la prova che il registro del formalismo e l'interfaccia di certificazione canonica supportano una semantica davvero diversa. I test di coerenza del formalismo dovrebbero verificare che i fatti condivisi basati mantengano l'identità, il tempo, la modalità e la provenienza attraverso gli abbassamenti.

La quinta release dovrebbe aggiungere pianificazione e realizzazione controllata. Un'idea è stilata in un piano canonico, controllata contro vincoli formali, realizzata in prosa e verificata dallo stesso sistema di regole. La valutazione deve distinguere la validità del piano, la fedeltà di realizzazione, la qualità linguistica e l'efficienza di riparazione. Il piano non diventa affidabile solo perché la prosa è stata generata da un oggetto strutturato; diventa più verificabile perché l'oggetto e i suoi controlli sono persistenti.

Artifact stack for a replayable executable-language runtime



La figura 4. Pila di artefatto per un runtime eseguibile-naturale rigiocabile. Ogni strato è in versione, mappato dalla sorgente e ispezionabile in modo indipendente.

Questa implementazione graduale incarna un importante principio metodologico: l'ambizione semantica dovrebbe crescere solo quando il sistema può rilevare che la sua rappresentazione precedente è inadeguata. Il runtime non dovrebbe nascondere il linguaggio non supportato dietro la fiducia del modello. Dovrebbe esporre le lacune di compatibilità come risultati di prima classe e trasformare lacune ricorrenti in nuovi produttori, ontologia, motore e lavoro di riferimento. Il linguaggio naturale eseguibile avanza attraverso l'espansione governata di un involucro di solidità, non attraverso un salto alla comprensione universale.

8.2 Domande di Ricerca, Benchmark e un Programma di Pubblicazione

L'architettura diventa un contributo scientifico solo quando le sue affermazioni sono confutabili. Una dimostrazione in cui un LLM genera un artefatto formale plausibile e un risolutore restituisce una risposta è insufficiente. Lo stesso modello può generare il benchmark, la traduzione e la spiegazione, creando un ciclo chiuso di errore correlato. Una valutazione grave deve scomporre la gasdotto e testare in modo indipendente ciascun obbligo. Essa deve inoltre comprendere le condizioni di insuccesso alle quali l'approccio proposto dovrebbe essere respinto o ristretto.

La prima domanda di ricerca riguarda la rappresentazione: il mantenimento di diverse interpretazioni semantiche ammissibili migliora l'affidabilità rispetto alla scelta di una traduzione? L'ipotesi corrispondente è che la semantica del set di candidati, combinata con l'analisi di maggio / must, riduce la falsa certezza su input ambigui e poco specificati. Può abbassare il tasso di risposta apparente perché più casi diventano condizionali o sconosciuti. Non è necessariamente un difetto. Le misure di confronto pertinenti se le conclusioni accettate devono essere più spesso corrette e se i risultati possano catturare alternative materialmente plausibili senza travolgere i revisori.

Un punto di riferimento per questa questione dovrebbe contenere testi con insiemi di interpretazione annotata da esperti piuttosto che una forma logica d'oro. Le coppie minime dovrebbero variare la portata del quantificatore, la negazione, l'aggiungendo di eccezione, la coordinazione, la coriferia, i confini temporali, la forza modale e la giurisdizione. Alcuni elementi dovrebbero avere una lettura chiaramente prevista, alcune diverse letture legittime e alcune prove insufficienti per una risposta determinata. I sistemi sono valutati sulla copertura delle letture ammissibili, esclusione di letture inammissibili, calibrazione dello stato di maggio/mus e verdetto a valle. La corrispondenza esatta del grafico è troppo fragile; l'equivalenza semantica dovrebbe essere testata attraverso suite di query o l'attribuzione bidirezionale sotto l'ontologia del benchmark.

La seconda domanda di ricerca riguarda il perfezionamento: i conflitti derivati dai soluttivi sono più utili per riparare la formalizzazione rispetto all'autocritica del modello generico? L'ipotesi è che controesempi, core insoddisfatti e obblighi di prova falliti producano riparazioni più localizzate, stabili ed efficienti dal campione perché identificano la dipendenza responsabile del risultato. Un esperimento controllato dovrebbe confrontare quattro condizioni: riprova diretta, auto-riflessione verbale, feedback dell'esecuzione-errore e perfezionamento semantico controesterno guidato con la mappatura delle fonti. Lo stesso modello, temperatura e budget dovrebbero essere utilizzati ove possibile.

La qualità della riparazione dovrebbe essere misurata non solo dal fatto che l'esempio di destinazione diventi corretto. La riparazione deve conservare i casi non correlati, ridurre la diff semantica ed evitare di introdurre ipotesi non supportate. Le suite di regressione basate su mutazione sono essenziali. Una riparazione che risolve un esempio indebolendo la regola a livello globale non è un successo. L'esperimento dovrebbe registrare il numero di iterazioni, gettoni, interventi umani, mutanti sopravvissuti, osservazioni modificate e se l'artefatto finale fosse rigiocabile in modo indipendente.

La terza questione di ricerca riguarda il contratto di prova. Una separazione dattiloscritta della fonte comprende, le osservazioni, le asserzioni onlogiche, gli assiomi di dominio e le precondizioni di conoscenza del mondo migliorano il trasferimento e l'auditability rispetto a un

grafico della conoscenza piatta o a una catena di pensiero in forma libera? L'ipotesi è che i contratti dattilogrammati riducano l'introduzione della premessa silenziosa e rendano il riutilizzo cross-domain più prevedibile. Il costo probabile è quello di authoring overhead e più stop run.

Il benchmark dovrebbe includere la stessa regola applicata tra i documenti di diversi domini, con variazione controllata nel vocabolario e nell'ontologia. Ad esempio, un requisito generale che ogni rivendicazione accettata abbia prove tracciabili può essere testato su documenti di ricerca, documenti politici, specifiche tecniche e piani. Le osservazioni strutturali possono essere trasferite direttamente; l'entità e le relazioni specifiche del dominio i produttori non possono. La valutazione registra che la compatibilità controlla lo stop, se la ragione di arresto è corretta e quanto nuovo materiale semantico è necessario per riprendere l'esecuzione. Il trasferimento di successo non è semplicemente ottenere una risposta; è riconoscere accuratamente ciò che può e non può essere riutilizzato.

La quarta questione di ricerca riguarda la scelta del formalismo. L'instradamento a un portafoglio di motori specializzati supera un linguaggio intermedio universale con una supervisione semantica uguale? Beiser e Penz mostrano che la scelta del linguaggio intermedio può cambiare complessivamente e l'accuratezza dell'esecuzione in modo sostanziale sui compiti di ragionamento neuro-simbolico [Beiser and Penz 2025]. L'ipotesi qui è più specifica: un router digitato che seleziona il formalismo più semplice adeguato migliorerà il successo della compilazione, la qualità della spiegazione e l'efficienza senza sacrificare la consistenza incrociata.

Un confronto significativo non dovrebbe consentire al sistema di portafoglio informazioni più nascoste. Ogni compito riceve la stessa fonte e l'ontologia. La linea di base del monoformismo potrebbe utilizzare la logica di primo ordine, ASP, SMT o un linguaggio di query di grafico generale. Il sistema di portafoglio può scegliere query relazionali, Datalog, tabelle di decisione, SMT, logica temporale o esecuzione simbolica. Le metriche includono la validità della formalizzazione, l'equivalenza semantica, il successo del risolutore, il runtime, la qualità della testimonianza e il costo di manutenzione. I casi di cross-formalismo dovrebbero richiedere due motori e verificare se i simboli condivisi rimangono coerenti.

L'esperimento dovrebbe anche cercare controprove. Una rappresentazione universale può sovraperformare il portafoglio in domini ristretti perché il routing introduce errori e più compilatori aumentano la base attendibile. La conclusione prevista non è che i portafogli siano sempre superiori. È che la pluralità del formalismo è giustificata solo quando la distribuzione dei compiti contiene veramente diverse strutture computazionali e quando le interfacce preservano la semantica.

La quinta domanda di ricerca riguarda il livello finale del linguaggio. La realizzazione con provezza riduce le affermazioni non supportate o confermate semanticamente rafforzate preservando la leggibilità accettabile? L'ipotesi è che un risultato canonico più la generazione con licenza di rivendicazione inferiore sostanzialmente all'allucinazione e alla deriva di modalità relative alla generazione di risposte dirette o alla generazione di risposte da una traccia di risolutore grezzo. Il costo può essere meno naturale e più rigenerazione.

Il benchmark dovrebbe includere risultati canonici con sottile incertezza, prove contrastanti, condizioni nidificate, quantità e date. I valutatori umani giudicano la fedeltà fattuale, la completezza dei limiti materiali, la leggibilità e l'utilità. Le selezionatrici determinanti misurano l'entità, il numero, la data, la polarità e la conservazione delle modalità. Un set contraddittorio dovrebbe tentare il renderizzatore a produrre una conclusione più forte e più semplice rispetto al

risultato. La metrica centrale non è la preferenza stilistica, ma la velocità con cui la prosa supera l'autorità del prodotto simbolico.

La sesta questione di ricerca riguarda l'architettura federativa stessa. La revisione sistematica del 2026 dello NLP neuro-simbolica riporta guadagni promettenti per i sistemi federativi, ma avverte che le prove sono confuse dalle differenze di attività [Chatzikyriakidis and Lappin 2026]. Un esperimento diretto dovrebbe confrontare una pipeline federativa con un'alternativa iniettiva o end-to-end appresa sugli stessi compiti, dati e scala di modelli. I vantaggi pertinenti possono apparire meno in termini di accuratezza grezza rispetto all'auditability, all'astensione, alla localizzazione degli errori e alla riparazione.

Lo studio dovrebbe quindi riportare diverse dimensioni. La precisione delle attività grezza rimane necessaria. Dovrebbe essere accompagnato da accuratezza semantica-forma, accuratezza sovraccarico-condizionata, calibrazione sconosciuta, riproducibilità tra le versioni del modello, fedeltà della spiegazione, tempo di revisione umana e la frazione di errori attribuibili all'estrazione, alla fondazione, alla compilazione, all'esecuzione o alla realizzazione. Un sistema modulare che corrisponde piuttosto che battere la precisione end-to-end può ancora essere preferibile per lavori ad alta sicurezza se trasforma gli errori silenziosi in artefatti diagnosibili. Al contrario, la modularità non dovrebbe essere difesa come una virtù intrinseca se si limita ad aggiungere la burocrazia senza migliorare le prove.

Una settima domanda di ricerca riguarda l'interpretazione astratta sulle alternative semantiche. È possibile definire un utile dominio astratto che è conservativo rispetto a un insieme di interpretazione di benchmark e più scalabile dell'enumerazione esplicita? I primi esperimenti possono utilizzare insiemi finiti come riferimento concreto. I domini astratti per l'identità, gli intervalli temporali, le modalità, le quantità e la copertura sono implementati separatamente e quindi combinati. La solidità viene verificata meccanicamente contro l'enumerazione su piccoli casi. La precisione è misurata dal tasso di risultati che possono essere eliminati dalla ricerca o dall'affinamento dei testimoni.

Questo programma dovrebbe essere attento alla parola solidità. Se l'annotazione di esperti definisce un insieme di interpretazione finita, l'interprete astratto può essere il suono rispetto a quel set. Non può quindi dimostrare che l'insieme esaurisce il significato umano. Il contributo è una semantica condizionale eseguibile e prove sulla sua utilità. Un successivo documento teorico può definire le funzioni di amministrazione, l'astrazione e la concretizzazione, le condizioni di trasferimento e gli operatori di perfezionamento. Il sistema empirico non dovrebbe aspettare una semantica completa del linguaggio, ma dovrebbe rendere esplicita la natura condizionata delle sue affermazioni.

Un'ottava domanda di ricerca riguarda l'esecuzione simbolica dei programmi semantici. La costruzione di testimoni sensibili al percorso migliora la comprensione dei revisori e la scoperta dei difetti rispetto ai risultati delle regole statiche? In un flusso di lavoro o in un dominio di policy, il runtime può confrontare un semplice valutatore di regole con l'esplorazione simbolica del percorso. Il valutatore dice che una violazione è possibile; l'esecutore restituisce una sequenza concreta di eventi e incarichi. I revisori umani giudicano se il testimone è intelligibile e utile. I difetti della verità e i mutanti seminati determinano se il testimone corrisponde a un caso reale.

L'esplosione del percorso deve essere parte della valutazione piuttosto che essere trattata come un inconveniente di attuazione. Lo studio dovrebbe variare ramificazioni, loop bounds, alternative di identità e scelte temporali. La guida di LLM può dare la priorità ai percorsi, come suggerisce il recente lavoro di esecuzione simbolica guidato da LLM, ma il motore deve

distinguere la copertura euristica dalla completezza formale. Le metriche di copertura dovrebbero riportare gli stati esplorati, gli stati potati, i timeout dei risolutori e il comportamento esterno non supportato. Un testimone è una forte evidenza di esistenza; l'incapacità di trovarne uno non è la prova di assenza a meno che l'esplorazione e il modello non giustificino tale conclusione.

L'architettura di riferimento dovrebbe essere stratificata. Il livello sorgente verifica l'ingestione e l'ancoraggio. Lo strato di osservazione prova l'estrazione e lo stato. Il livello di fondazione testa l'identità, il tempo, la modalità, le quantità e l'autorità. Il livello di formalizzazione testa l'equivalenza semantica e la validità del linguaggio target. Il livello di esecuzione prova le uscite e i certificati del solutore. Il livello di risultato testa la composizione e l'incertezza del verdetto. Il livello di realizzazione verifica la fedeltà delle risposte natural-lingua. Ogni esempio dovrebbe identificare il livello più precoce in cui un guasto iniettato dovrebbe essere rilevato.

I casi di riferimento dovrebbero essere abbastanza piccoli da convalidare manualmente e compattarli in documenti più lunghi. Piccoli esempi forniscono una verità di base affidabile per la logica sottile. Recupero di lunghi compositi, indicizzazione, continuità dell'identità e scalabilità. La stessa regola sottostante può apparire in diverse forme linguistiche e strutture documentali. Le famiglie di parafrasi testano l'invarianza; i set di contrasto cambiano una caratteristica semantica e la sensibilità al test. Il benchmark deve evitare il guasto comune in cui un sistema funziona bene memorizzando i modelli di superficie ignorando la distinzione prevista.

Il test di mutazione dovrebbe essere centrale. Le mutazioni possono cambiare un quantificatore, negare una condizione, spostare un'eccezione, alterare un confine di data, unire le identità, eliminare le prove, cambiare la forza modale, scambiare attore e oggetto, duplicare un requisito o modificare una soglia del circuito. Una suite robusta dovrebbe rilevare il cambiamento previsto e lasciare risultati non correlati stabili. Il punteggio di mutazione non è una misura di qualità completa, ma rivela che i test confermano semplicemente un percorso felice. Per nllAgent, le mutazioni eseguibili affronterebbero direttamente una delle lacune riconosciute del repository [AssistOS-AI 2026].

La valutazione dovrebbe includere pronta iniezione e avversari semantici. I documenti possono contenere istruzioni dirette al modello, intestazioni fuorvianti, testo politico citato, Unicode avversaria, contenuto nascosto o dichiarazioni che assomigliano a dichiarazioni di ontologia. Il parser della fonte deve preservarli come dati. I produttori semantici dovrebbero operare sotto l'autorità di sistema fisso. Il benchmark dovrebbe verificare se il testo del documento può alterare il circuito, sopprimere i risultati o causare chiamate di strumenti privilegiati. I risultati di sicurezza devono essere registrati separatamente dall'accuratezza semantica.

La valutazione del live-modello deve essere sufficientemente riproducibile per sostenere le affermazioni scientifiche. Ogni esecuzione dovrebbe aggiungere il fornitore, l'identificatore del modello, richiedere lo schema, la temperatura, la senza quando supportato, la costruzione del contesto e la politica di ritenzione. Gli input e gli output devono essere acquisiti soggetti a vincoli di privacy. Gli studi ripetuti stimano la varianza. I doppi deterministici rimangono utili per i test del software, ma non possono stabilire la qualità della traduzione. Almeno due famiglie modello dovrebbero essere testate ove possibile per distinguere gli effetti architettonici dal comportamento di un fornitore.

La valutazione umana è necessaria, ma dovrebbe essere progettata in modo restrittivo. Agli esperti non dovrebbe essere chiesto se una risposta “sembra buona”. Dovrebbero giudicare obblighi specifici: se la formalizzazione preserva la fonte, se le identità sono giustificate, se un testimone è fattibile, se la spiegazione indica tutte le ipotesi materiali e se un sconosciuto è adeguatamente restituito. Il disaccordo tra gli inter-raggi è di per sé prova di ambiguità semantica e non dovrebbe essere costretto a un'unica etichetta d'oro senza analisi.

Le metriche quantitative di base dovrebbero includere l'accuratezza della fonte-ancoraggio; precisione di osservazione e richiamo per stato; ontologia e validità del tipo; precisione, richiamo e calibrazione del collegamento dell'identità; accuratezza della relazione temporale e modale; copertura del set candidato; può/devere la solidità rispetto alle interpretazioni annotate; la sintassi di formalizzazione e l'equivalenza semantica; successo del solutore; validità della prova; successo della prova; tasso di garanzia; astensioni appropriate; valutazione; valutazione; ritardo; valutazione; Un singolo punteggio aggregato nasconderebbe il meccanismo e dovrebbe essere secondario.

La falsa assicurazione merita particolare enfasi. Essa misura i casi in cui il sistema presenta una conclusione verificata, anche se la fonte, la fondazione o l'insieme di interpretazione non lo giustificano. Nelle applicazioni ad alta sicurezza, la riduzione della falsa garanzia potrebbe essere più importante che massimizzare le domande a risposta. Il sistema dovrebbe ricevere credito per uno stato preciso fermato quando un LLM diretto inventa con sicurezza un ponte. L'appropriato sconosciuto è un risultato positivo.

Le linee di base dovrebbero includere la risposta diretta LLM, la catena del pensiero o il ragionamento equivalente, l'esecuzione LLM aumentata da strumenti senza artefatti formali persistenti, traduzione monoformalismo, traduzione set candidati senza interpretazione astratta, esecuzione simbolica senza raffinatezza semantica e architettura completa. Per il caso di studio nllAgent, l'attuale rilascio di fondazione delimitato può servire come base implementata, mentre l'identità proposta, l'identità astratta e le estensioni multiformalismo vengono valutate in modo incrementale. Ciò evita di confrontare una visione di ricerca completa solo con sistemi esterni più deboli.

Il programma di pubblicazione può essere organizzato attorno a cinque contributi piuttosto che un'indicazione di grandi dimensioni. Il primo documento dovrebbe essere una posizione e un documento di architettura che definisca il linguaggio naturale eseguibile, il confine di fiducia, il divario di formalizzazione, l'involucro della solidità e il design multi-formalismo. Il suo contributo è la chiarezza concettuale e un'architettura di riferimento, supportata da un'indagine sistematica e da un piccolo dimostratore di lavoro.

Il secondo documento dovrebbe formalizzare il perno semantico e la solidità condizionale della provenienza. Definisce gli oggetti di origine, le osservazioni digitate, le interpretazioni dei candidati, la semantica di maggio/deve e la compatibilità. Dimostra la solidità del calcolo astratto rispetto a un insieme di interpretazione ammissibile e riporta esperimenti su identità, tempo, modalità e copertura. Questo è il nucleo di programmazione-lingua e formal-semantica.

Il terzo documento dovrebbe introdurre un perfezionamento semantico controesterno guidato. Definisce le mappature dai conflitti dei sovrintendenti agli impegni semantici collegati alla fonte e confronta il perfezionamento con l'auto-riflessione e il feedback di esecuzione generico. Il risultato più forte sarebbe una minore falsa garanzia, riparazioni più piccole, una migliore conservazione della regressione e un ridotto sforzo umano. Anche un risultato negativo

sarebbe prezioso se identifica quale feedback del solutore è troppo di basso livello per guidare la riparazione semantica.

Il quarto documento dovrebbe presentare il portafoglio di formalismo e i contratti di motore dattiloscritti. Valuta gli adattatori relazionali, Datalog, Decision-table, SMT, temporali e simbolici-esecuzione su un modello di artefatto comune. Il documento dovrebbe enfatizzare la conservazione semantica e i certificati piuttosto che il numero di integrazioni. Un piccolo set di motori veramente diversi è più persuasivo di molti involucri poco profondi.

Il quinto documento dovrebbe essere il sistema nllAgent e il documento di valutazione. Dovrebbe distinguere chiaramente il rilascio attuato dal più ampio runtime. Può documentare revisioni immutabili delle fonti, LongTextJS, CircuitJS, osservazioni basate sulla domanda, compatibilità, copertura, esecuzione limitata, governance dell'apprendimento e della pubblicazione, verifica della riproduzione, audit e prodotti di piano canonici e processi di pianificazione e il processo di problemi gravi. La valutazione dovrebbe includere la mutazione eseguibile, i record dei modelli dal vivo, i casi di identità, la governance del pacchetto attuale, i parametri di pianificazione e un verificatore indipendente o separatamente, ove possibile.

Un sesto articolo successivo potrebbe studiare la realizzazione con la prova. Confronterebbe la generazione diretta, il rendering dei modelli, il rendering LLM limitato e la verifica post-hoc. Il contributo centrale sarebbe una rappresentazione e una prova che la licenza delle rivendicazioni e la prova che il linguaggio leggibile può essere generato senza modificare modalità, incertezza o contenuto fattuale. Questo lavoro collega l'esecuzione formale alla necessità pratica di una comunicazione umana accessibile.

Il progetto dovrebbe definire criteri di fallimento espliciti. L'approccio del candidato dovrebbe essere riconsiderato se produce così tante alternative che i risultati potrebbero/deve non essere informativi e il perfezionamento non può recuperare precisione. Il portafoglio di formalismo dovrebbe essere ristretto se i bug di routing e cross-formalismo superano i benefici della specializzazione. La formalizzazione assistita da LLM dovrebbe essere limitata al supporto per l'autore se l'equivalenza semantica rimane scarsa sotto documenti realistici. La realizzazione con procedura dovrebbe ricadere sui modelli se la prosa generata dal modello supera ripetutamente la sua licenza. Il ragionamento del mondo attuale dovrebbe rimanere al di fuori del nucleo di fiducia se non è possibile applicare la fonte, il tempo, la giurisdizione e i controlli sui conflitti.

L'onestà scientifica richiede anche il confronto con alternative più semplici. Molti controlli dei documenti possono essere risolti da parsertori deterministici, espressioni regolari, convalida dello schema, query di database o regole create dall'uomo. Un LLM è giustificato quando la variazione del linguaggio, la mappatura dell'online o la proposta semantica riduce materialmente i costi di creazione o aumenta la copertura. Un nucleo simbolico è giustificato quando lo stato esatto, il replay, i testimoni o la materia di verifica indipendente. Un documento di ricerca dovrebbe riferire quando un sistema più semplice funziona altrettanto bene. Il linguaggio naturale eseguibile non è un motivo per formalizzare ogni preferenza editoriale o inserire un risolutore in compiti che hanno bisogno solo di un parser affidabile.

Il probabile risultato pratico non è una macchina che esegue tutto il linguaggio naturale. È una famiglia di compilatori semantici governati per i domini di documenti e agenti, la condivisione di fonti, la fondazione, l'esecuzione e l'infrastruttura di certificazione. Ogni compilatore ha una busta semantica dichiarata. LLMs rende più facile l'autore, l'adattamento e la spiegazione di questi compilatori. I metodi formali rendono i risultati accettati riproducibili e

contestabili. La combinazione è utile proprio perché a nessuno dei due componenti viene chiesto di fornire ciò che non può fornire in modo affidabile da solo.

Il programma di ricerca è quindi più modesto e più consequenziale dello slogan “linguaggio naturale come codice”. Il linguaggio naturale rimane un mezzo aperto, sociale, indipendente dal contesto. Il runtime crea proiezioni eseguibili di esso per le domande dichiarate. Tali proiezioni possono essere abbastanza rigorose da supportare audit, piani, protocolli e controlli scientifici reali, a condizione che il sistema registri la distanza tra origine e oggetto formale invece di fingere che la distanza sia scomparsa.

Conclusione: dalla Forma Probabilistica alla Conseguenza Simbolica

La recente rinascita della AI neuro-simbolica ha prodotto un modello architettonico riconoscibile. Vengono utilizzati grandi modelli linguistici in cui lo spazio di ricerca è linguistico, eterogeneo o difficile da enumerare. I sistemi formali vengono utilizzati in cui lo stato esatto, la coerenza, la ricerca esaustiva esaurita, i testimoni o le prove contano. La domanda interessante non è più se i metodi neurali e simbolici possano essere combinati. E' chiaro che possono farlo. La questione importante è dove si trova l'autorità e cosa è controllato in modo indipendente.

L'indagine mostra che i sistemi più credibili non concedono al modello linguistico l'autorità finale sul risultato. I linguaggi controllati traducono una superficie ristretta in semantica formale. I parser semantici generano query o programmi eseguibili. PAL delega il calcolo a Python. SatLM delega i vincoli a un solutore. Logic-LM e LINC delegano la deduzione ai motori logici. NL2TL e nl2spec traducono i requisiti in specifiche temporali. I sistemi di pianificazione utilizzano modelli formali e strumenti di verifica per mantenere la coerenza long-horizon. SymGPT compila le regole ERC in linguaggio naturale in vincoli e utilizza l'esecuzione simbolica per costruire percorsi contrattuali concreti. SAIL utilizza LLMs per cercare trasformatori astratti ma ammette solo candidati che soddisfano gli obblighi meccanici. Questi sistemi differiscono notevolmente, ma convergono sul principio che la generazione propone e decide un meccanismo indipendente.

Questa convergenza non va sopravvalutata. L'uso del soverbio non verifica la comprensione. Il divario di formalizzazione rimane il problema centrale. Un prover del teorema può derivare una valida conclusione dai predicati che travisano il testo. Un programma può restituire la risposta prevista attraverso un percorso spurio. Un modello SMT può soddisfare i vincoli che omettono il requisito decisivo dell'utente. Un esecutore simbolico può esporre un vero e proprio percorso software per una regola errata. I sistemi attuali più forti riducono l'errore deduttivo e computazionale, trasferendo al contempo il rischio di analisi semantica, fondazione, ontologia e portata.

La risposta appropriata non è quella di abbandonare gli strumenti formali perché la traduzione è imperfetta. È fare della traduzione un artefatto persistente, digitato, recensibile. La fonte deve rimanere disponibile. Ogni osservazione semantica deve identificare il suo produttore, le prove, lo stato e l'ontologia. L'identità, il tempo, la modalità, le quantità e l'autorità devono essere rappresentati piuttosto che dedotti di nuovo durante ogni risposta. Le interpretazioni alternative devono rimanere alternative quando la distinzione influisce sul risultato. Un motore simbolico dovrebbe eseguire solo oggetti a terra che soddisfano contratti espliciti.

L'interpretazione astratta fornisce una teoria utile per il lato globale di questa architettura. Chiede cosa si possa concludere in modo conservativo da un gran insieme di possibili stati. Applicati al linguaggio naturale, gli stati concreti non sono "significazioni" indefiniti ma strutture semantiche delimitate ammissibili sotto una politica dichiarata di grammatica, ontologia, revisione delle fonti e fondamento. Uno stato astratto può quindi rappresentare ciò che deve contenere, può detenere, non può reggere, conflitti o rimanere sconosciuti. La garanzia è subordinata al set di interpretazione, ma la solidità condizionale è comunque significativa quando le condizioni sono esplicite e testate.

L'esecuzione simbolica fornisce il complemento locale. Quando un'analisi astratta riporta una possibile violazione, l'esecuzione simbolica o un altro motore può cercare un'interpretazione e un percorso concreti. Una ricerca di successo restituisce un testimone che un revisore può ispezionare. Una ricerca fallita espone un'astrazione eccessivamente grossolana o una scelta semantica incompatibile. La mappatura di quel fallimento negli intervalli di origine crea un perfezionamento semantico controesterno guidato: il modello o l'uomo rivede il più piccolo impegno formale pertinente e l'analisi viene rigiocata. Questo è un uso più disciplinato del feedback LLM rispetto all'autocritica generica perché il segnale di errore è legato a una dipendenza formale.

Nessuna singola lingua di destinazione è suscettibile di supportare ogni forma utile di prosa eseguibile. La letteratura dimostra già la dipendenza dalla scelta del linguaggio intermedio e i sistemi contemporanei hanno successo utilizzando Python, SQL, logica di primo ordine, logica temporale, PDDL, linguaggi di vincolo o grammatiche specifiche del dominio in base al compito. Un runtime pratico dovrebbe quindi utilizzare un perno semantico di provenienza e un portafoglio di motori formali da ricerca. La valutazione Relational e Datalog gestisce le query e la chiusura dei documenti; le tabelle decisionali gestiscono politiche limitate; SAT/SMT/CSP gestisce vincoli di interazione; controllo del modello gestisce i comportamenti temporali; maniglie simboliche del percorso dei bug; automi che gestiscono modelli ordinati; manipolazioni logiche naturali includenti selezionati; assistenti di prova gestiscono affermazioni formali di alto valore.

La pluralità introduce i propri rischi ingegneristici. Ogni compilatore può distorcere la semantica, ogni motore espande la base attendibile e il routing può guastarsi. Il portafoglio è giustificato solo se le interfacce sono digitate, i simboli condivisi mantengono l'identità e la provenienza, le garanzie del motore sono esplicite e i risultati sono normalizzati in un prodotto canonico che porta certificati. Una logica universale può rimanere preferibile in un dominio ristretto. L'affermazione architettonica non è che la pluralità sia sempre migliore; è che un ampio linguaggio eseguibile richiede diverse strutture computazionali e non dovrebbe mascherarle dietro una vaga DSL.

Il ritorno dai simboli al linguaggio fa parte del confine fiduciario. A un LLM finale non deve essere consentito di trasformare un risultato condizionale in una raccomandazione incondizionata, in una possibile violazione confermata o prove mancanti in prove di assenza. Il risultato canonico dovrebbe definire un insieme finito di rivendicazioni autorizzate con modalità, quantità, entità, ipotesi e provenienza. Il renderizzatore può migliorare la leggibilità, ma dovrebbe rimanere subordinato a tale autorità. La realizzazione di Proof-constrained è il naturale completamento dell'architettura.

L'implementazione pubblica nllAgent è significativa perché incarna già molte di queste discipline. LongTextJS conserva un programma di documenti basati sulla fonte; CircuitJS rappresenta teorie riutilizzabili; la domanda di osservazione è derivata da circuiti; la compatibilità e la copertura sono esplicite; moduli ristretti e estensioni rivisti limitano l'esecuzione nascosta; l'apprendimento avviene nella messa in scena usa e getta; la pubblicazione è manuale e immutabile; la verifica replay controlla i risultati canonici; e la conoscenza mancante non viene trattata come conformità. Queste decisioni rendono il progetto una piattaforma credibile per la ricerca anche se non stabiliscono una comprensione semantica generale.

Gli stessi materiali pubblici identificano l'importante lavoro incompiuto: materializzazione generale dell'identità, contratti primitivi digitati completi, mutazione semantica eseguibile, valutazione indipendente dell'ombra, benchmarking live-model, benchmark di pianificazione e realizzazione e pacchetti di conoscenza attuali governati. Non è stata trovata alcuna valutazione peer-reviewed indipendente di nllAgent per questa edizione. Il giudizio qui offerto è quindi architettonico: l'implementazione ha scelto confini insolitamente solidi, ma le sue più forti affermazioni difendibili rimangono limitate nell'analisi strutturale e controllata del linguaggio. Il suo valore aumenterà se l'ampiezza semantica viene ampliata solo attraverso produttori espliciti, contratti, benchmark e prove.

Il passo successivo proposto non è quello di sostituire LongTextJS o CircuitJS con un linguaggio universale più grande. LongTextJS dovrebbe maturare come mondo semantico finito, interrogabile e di provenienza. CircuitJS dovrebbe rimanere un livello di orchestrazione e di un contratto di prova. I motori registrati specializzati devono eseguire l'esecuzione relazionale, costrittiva, temporale, di flusso di dati, automi o di esecuzione sensibile al percorso. La semantica di maggio/deve dovrebbe diventare operativa. I conteexamples dovrebbero guidare la raffinazione semantica focalizzata. I prodotti canonici CNL dovrebbero controllare l'output finale del linguaggio.

Un rigoroso programma di ricerca può testare ogni parte. La semantica del set di candidati può essere confrontata con sistemi di traduzione singola. La raffinatezza derivata dal solutore può essere confrontata con l'auto-riflessione. I contratti di prova di tipo possono essere testati per il trasferimento e la falsa garanzia. Un portafoglio di formalismo può essere confrontato con un linguaggio intermedio universale. I domini astratti possono essere controllati rispetto ai set di interpretazione espliciti. I testimoni simbolici possono essere valutati per la scoperta dei difetti e l'utilità del recensore. La realizzazione con procedure può essere testata per la fedeltà. Le architetture modulari e end-to-end possono essere confrontate sulle stesse attività piuttosto che su diverse famiglie di benchmark.

La metrica decisiva dovrebbe essere falsa garanzia, non solo di rispondere. Un sistema che ritorna "sconosciuto perché l'identità è irrisolta" può essere più prezioso di uno che produce un verdetto fiducioso fondendo silenziosamente due persone. Un sistema che si ferma perché le prove del mondo attuale sono stantie è più affidabile di uno che recupera un fatto non supportato. Un sistema che espone tre interpretazioni è migliore di una che sceglie il più fluente. L'assorbimento appropriato fa parte della semantica dell'esecuzione.

Il linguaggio naturale eseguibile dovrebbe quindi essere inteso come un rapporto di compilazione governato, non come proprietà intrinseca della prosa. Il testo di origine rimane più ricco di qualsiasi proiezione eseguibile. Il sistema seleziona un compito, materializza la semantica pertinente, preserva l'incertezza, i simboli dei motivi, compila un formalismo appropriato, esegue sotto limiti espliciti, verifica il risultato e restituisce una spiegazione vincolata. Ciò che diventa eseguibile non è un linguaggio nella sua interezza, ma una proiezione semantica dichiarata le cui ipotesi e limiti rimangono visibili.

Questa concezione più ristretta è sufficiente per un notevole impatto pratico. Le rivendicazioni scientifiche possono essere verificate per la provenienza e la coerenza. I requisiti possono essere collegati a test e stati del ciclo di vita. Le politiche possono essere compilate in strutture decisionali. I piani possono essere verificati prima della realizzazione. Le regole del protocollo possono essere mappate per le analisi di stato e di percorso. I documenti lunghi possono essere cullati da teorie riutilizzabili senza chiedere a un modello di ricordare ogni

dettaglio. Gli agenti possono usare il linguaggio naturale come uno strato di authoring e di coordinamento, delegando conseguenze esatte ai tempi simbolici.

Il principio centrale può essere dichiarato senza simbolismo formale: lasciare che lo LLM scopra la forma possibile; lascia che il kernel di fondazione decida cosa sono autorizzati a indicare; lasciare che i motori formali determinino ciò che segue; e lascia che lo LLM finale spieghi solo quali sono le licenze simboliche di registrazione. Questo principio non risolve tutti i problemi del linguaggio. Crea un confine difendibile tra interpretazione e conseguenza, che è il fondamento necessario per il linguaggio naturale per partecipare in modo sicuro ai sistemi eseguibili.

Riferimenti

- Abzianidze, Lasha, Rik van Noord, Hessel Haagsma, and Johan Bos. 2019. "The First Shared Task on Discourse Representation Structure Parsing and Evaluation." Proceedings of the IWCS Shared Task on Semantic Parsing. Association for Computational Linguistics. <https://aclanthology.org/W19-1201/>
- Abzianidze, Lasha. 2017. "LangPro: Natural Language Theorem Prover." Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. Association for Computational Linguistics. <https://aclanthology.org/D17-2022/>
- Angeli, Gabor, and Christopher D. Manning. 2014. "NaturalLI: Natural Logic Inference for Common Sense Reasoning." Proceedings of EMNLP 2014. Association for Computational Linguistics. <https://aclanthology.org/D14-1209/>
- AssistOS-AI. 2026. "NaturalLanguageLinterAgent (nllAgent): LongTextJS–CircuitJS Architecture, Documentation, and Serious Issues." Public repository and generated documentation, inspected 2 August 2026. <https://github.com/assistos-ai/nllAgent>
- Badreddine, Samy, Artur d'Avila Garcez, Luciano Serafini, and Michael Spranger. 2022. "Logic Tensor Networks." Artificial Intelligence 303: 103649. <https://doi.org/10.1016/j.artint.2021.103649>
- Baldoni, Roberto, Emilio Coppa, Daniele Cono D'Elia, Camil Demetrescu, and Irene Finocchi. 2018. "A Survey of Symbolic Execution Techniques." ACM Computing Surveys 51(3): 1–39. <https://doi.org/10.1145/3182657>
- Beg, Arshad, Diarmuid P. O'Donoghue, and Rosemary Monahan. 2025. "A Short Survey on Formalising Software Requirements Using Large Language Models." arXiv:2506.11874. <https://doi.org/10.48550/arXiv.2506.11874>
- Beiser, Alexander, and David Penz. 2025. "Making LLMs Reason? The Intermediate Language Problem in Neurosymbolic Approaches." arXiv:2502.17216. <https://arxiv.org/abs/2502.17216>
- Bos, Johan. 2008. "Wide-Coverage Semantic Analysis with Boxer." Proceedings of the 2008 Conference on Semantics in Text Processing. <https://aclanthology.org/W08-2222/>
- Chatzikyriakidis, Stergios, and Shalom Lappin. 2026. "Neuro-Symbolic NLP: Taxonomy, Assessment, and Directions." Frontiers in Artificial Intelligence 9. <https://doi.org/10.3389/frai.2026.1797587>
- Chen, Lingfeng, Tao Xiao, Masanari Kondo, and Yasutaka Kamei. 2026. "Directed Symbolic Execution for Vulnerability Discovery: An LLM-Guided Approach in KLEE." arXiv:2607.21676. <https://arxiv.org/abs/2607.21676>
- Chen, Yongchao, Rujul Gandhi, Yang Zhang, and Chuchu Fan. 2023. "NL2TL: Transforming Natural Languages to Temporal Logics Using Large Language Models." Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics. <https://aclanthology.org/2023.emnlp-main.985/>
- Clarke, Edmund, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. 2000. "Counterexample-Guided Abstraction Refinement." Computer Aided Verification, LNCS 1855, 154–169. https://doi.org/10.1007/10722167_15
- Cosler, Matthias, Christopher Hahn, Daniel Mendoza, Frederik Schmitt, and Caroline Trippel. 2023. "nl2spec: Interactively Translating Unstructured Natural Language to Temporal Logics with Large Language Models." Computer Aided Verification, LNCS 13964, 383–396. https://doi.org/10.1007/978-3-031-37703-7_18
- Cousot, Patrick, and Radhia Cousot. 1977. "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints." Proceedings of POPL 1977, 238–252. <https://doi.org/10.1145/512950.512973>
- Delvecchio, Giovanni Pio, Lorenzo Molfetta, and Gianluca Moro. 2025. "Neuro-Symbolic Artificial Intelligence: A Task-Directed Survey in the Black-Box Models Era." Proceedings of IJCAI 2025, Survey Track, 10418–10426. <https://doi.org/10.24963/ijcai.2025/1157>
- Dong, Wenkai, and Yifan Wang. 2026. "From Interpretation to Compilation: Compilation-Based Execution of Semantic Operators." arXiv:2607.13407. <https://doi.org/10.48550/arXiv.2607.13407>
- Ferrari, Alessio, and Paola Spoletini. 2025. "Formal Requirements Engineering and Large Language Models: A Two-Way Roadmap." Information and Software Technology 181: 107697. <https://doi.org/10.1016/j.infsof.2025.107697>

- Fuchs, Norbert E., and Rolf Schwitter. 1996. "Attempto Controlled English (ACE)." Proceedings of the First International Workshop on Controlled Language Applications. <https://attempto.ifi.uzh.ch/site/pubs/papers/ace96.pdf>
- Fuchs, Norbert E., Kaarel Kaljurand, and Tobias Kuhn. 2008. "Attempto Controlled English for Knowledge Representation." Reasoning Web, LNCS 5224, 104–124. https://doi.org/10.1007/978-3-540-85658-0_3
- Gao, Luyu, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2022. "PAL: Program-Aided Language Models." arXiv:2211.10435. <https://arxiv.org/abs/2211.10435>
- Goldman, Omer, Veronica Latcinnik, Udi Naveh, Amir Globerson, and Jonathan Berant. 2018. "Weakly Supervised Semantic Parsing with Abstract Examples." Proceedings of ACL 2018. Association for Computational Linguistics. <https://aclanthology.org/P18-1168/>
- Grimaldi, Pasquale. 2026. "pamaldi at SemEval-2026 Task 11: Neuro-Symbolic Syllogistic Reasoning via LLM-Guided Structure Extraction and Deterministic Validation." Proceedings of the 20th International Workshop on Semantic Evaluation. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2026.semeval-1.186>
- Gu, Qiuhan, Avaljot Singh, and Gagandeep Singh. 2026. "SAIL: Sound Abstract Interpreters with LLMs." Proceedings of the ACM on Programming Languages 10 (PLDI): 1534–1559. <https://doi.org/10.1145/3808308>
- Guu, Kelvin, Panupong Pasupat, Evan Liu, and Percy Liang. 2017. "From Language to Programs: Bridging Reinforcement Learning and Maximum Marginal Likelihood." Proceedings of ACL 2017. Association for Computational Linguistics. <https://aclanthology.org/P17-1016/>
- Hamilton, Kyle, Aparna Nayak, Bojan Božić, and Luca Longo. 2024. "Is Neuro-Symbolic AI Meeting Its Promises in Natural Language Processing? A Structured Review." Semantic Web 15(4): 1265–1306. <https://doi.org/10.3233/SW-223228>
- Han, Simeng, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alex Wardle-Solano, Hannah Szabó, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander Fabbri, Wojciech Kryściński, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying, and Dragomir Radev. 2024. "FOLIO: Natural Language Reasoning with First-Order Logic." Proceedings of EMNLP 2024. Association for Computational Linguistics. <https://aclanthology.org/2024.emnlp-main.1229/>
- Hao, Yilun, Yongchao Chen, Yang Zhang, and Chuchu Fan. 2025. "Large Language Models Can Solve Real-World Planning Rigorously with Formal Verification Tools." Proceedings of NAACL 2025, 3434–3483. <https://aclanthology.org/2025.naacl-long.176/>
- Hu, Ruikang, Shaoyu Lin, Yeliang Xiu, and Yongmei Liu. 2025. "LTRAG: Enhancing Autoformalization and Self-refinement for Logical Reasoning with Thought-Guided RAG." Findings of ACL 2025, 2483–2493. <https://aclanthology.org/2025.findings-acl.126/>
- Huang, Pei, Jiayu Li, Yuhao Zhang, Changliu Liu, and Shih-hsi Alex Liu. 2026. "Parameterized Abstract Interpretation for Transformer Verification." Proceedings of AAAI 2026. <https://doi.org/10.1609/aaai.v40i42.40860>
- Jia, Robin, Aditi Raghunathan, Kerem Göksel, and Percy Liang. 2019. "Certified Robustness to Adversarial Word Substitutions." Proceedings of EMNLP-IJCNLP 2019. Association for Computational Linguistics. <https://aclanthology.org/D19-1423/>
- Karthikeyan, T, Om Dehlan, Mausam, and Manish Gupta. 2025. "LRPLAN: A Multi-Agent Collaboration of Large Language and Reasoning Models for Planning with Implicit and Explicit Constraints." Findings of EMNLP 2025, 8280–8310. <https://doi.org/10.18653/v1/2025.findings-emnlp.440>
- Kartáč, Ivan, Kristýna Onderková, Jan Bronec, Zdeněk Kasner, Mateusz Lango, and Ondřej Dušek. 2026. "UFAL-CUNI at SemEval-2026 Task 11: An Efficient Modular Neuro-Symbolic Method for Syllogistic Reasoning." Proceedings of the 20th International Workshop on Semantic Evaluation. Association for Computational Linguistics. <https://aclanthology.org/2026.semeval-1.418/>
- King, James C. 1976. "Symbolic Execution and Program Testing." Communications of the ACM 19(7): 385–394. <https://doi.org/10.1145/360248.360252>
- Kirtania, Shashank, Priyanshu Gupta, and Arjun Radhakrishna. 2024. "LOGIC-LM++: Multi-Step Refinement for Symbolic Formulations." Proceedings of the 2nd Workshop on Natural Language Reasoning and Structured Explanations, 56–63. <https://aclanthology.org/2024.nlrse-1.6/>

- Kuhn, Tobias. 2014. "A Survey and Classification of Controlled Natural Languages." *Computational Linguistics* 40(1): 121–170. https://doi.org/10.1162/COLI_a_00168
- Lalwani, Vivek, et al. 2025. "Autoformalizing Natural Language to First-Order Logic: A Case Study in Logical Fallacy Detection." *Findings of IJCNLP-AAACL 2025*, 132–147. <https://aclanthology.org/2025.findings-ijcnlp.8/>
- Lee, Kang-il, Segwang Kim, and Kyomin Jung. 2023. "Weakly Supervised Semantic Parsing with Execution-based Spurious Program Filtering." *Proceedings of EMNLP 2023*, 6884–6894. <https://aclanthology.org/2023.emnlp-main.425/>
- Li, Yihe, Ruijie Meng, and Gregory J. Duck. 2025. "Large Language Model Powered Symbolic Execution." *Proceedings of the ACM on Programming Languages* 9 (OOPSLA2), Article 385, 3148–3176. <https://doi.org/10.1145/3763163>
- Liang, Chen, Jonathan Berant, Quoc Le, Kenneth D. Forbus, and Ni Lao. 2017. "Neural Symbolic Machines: Learning Semantic Parsers on Freebase with Weak Supervision." *Proceedings of ACL 2017*. Association for Computational Linguistics. <https://aclanthology.org/P17-1003/>
- Liu, Jiangming, Shay B. Cohen, and Mirella Lapata. 2018. "Discourse Representation Structure Parsing." *Proceedings of ACL 2018*. Association for Computational Linguistics. <https://aclanthology.org/P18-1040/>
- Locascio, Nicholas, Karthik Narasimhan, Eduardo DeLeon, Nate Kushman, and Regina Barzilay. 2018. "Neural Generation of Regular Expressions from Natural Language with Minimal Domain Knowledge." *Proceedings of EMNLP 2018*. Association for Computational Linguistics. <https://aclanthology.org/D18-1436/>
- MacCartney, Bill, and Christopher D. Manning. 2007. "Natural Logic for Textual Inference." *Proceedings of the ACL-PASCAL Workshop on Textual Entailment and Paraphrasing*. <https://aclanthology.org/W07-1401/>
- MacCartney, Bill, and Christopher D. Manning. 2009. "An Extended Model of Natural Logic." *Proceedings of IWCS 2009*. <https://aclanthology.org/W09-3714/>
- Manhaeve, Robin, Sebastijan Dumančić, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. 2018. "DeepProbLog: Neural Probabilistic Logic Programming." *Advances in Neural Information Processing Systems* 31. <https://proceedings.neurips.cc/paper/2018/hash/dc5d637ed5e62c36ecb73b654b05ba2a-Abstract.html>
- Mao, Ziyu, Jingyi Wang, Jun Sun, Shengchao Qin, and Jiawen Xiong. 2025. "LLM-Aided Automatic Modeling for Security Protocol Verification." *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, 2025. https://ink.library.smu.edu.sg/sis_research/10297/
- Mitchell, Jacqueline L., Brian Hyeongseok Kim, Chenyu Zhou, and Chao Wang. 2025. "Can LLMs Formally Reason as Abstract Interpreters for Program Analysis?" *arXiv:2503.12686*. <https://arxiv.org/abs/2503.12686>
- Mou, Lili, Zhengdong Lu, Hang Li, and Zhi Jin. 2017. "Coupling Distributed and Symbolic Execution for Natural Language Queries." *Proceedings of ICML 2017*. <https://proceedings.mlr.press/v70/mou17a.html>
- Olausson, Theo X., Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum, and Roger Levy. 2023. "LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers." *Proceedings of EMNLP 2023*. Association for Computational Linguistics. <https://aclanthology.org/2023.emnlp-main.313/>
- Pan, Liangming, Alon Albalak, Xinyi Wang, and William Yang Wang. 2023. "Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning." *Findings of EMNLP 2023*. Association for Computational Linguistics. <https://aclanthology.org/2023.findings-emnlp.248/>
- Pasupat, Panupong, and Percy Liang. 2016. "Inferring Logical Forms from Denotations." *Proceedings of ACL 2016*. Association for Computational Linguistics. <https://aclanthology.org/P16-1004/>
- Pei, Yu, Yongping Du, and Xingnan Jin. 2025. "FoVer: First-Order Logic Verification for Natural Language Reasoning." *Transactions of the Association for Computational Linguistics* 13: 1340–1359. <https://aclanthology.org/2025.tacl-1.61/>
- Tafjord, Oyvind, Bhavana Dalvi Mishra, and Peter Clark. 2021. "ProofWriter: Generating Implications, Proofs, and Abductive Statements over Natural Language." *Findings of ACL-IJCNLP 2021*. Association for Computational Linguistics. <https://aclanthology.org/2021.findings-acl.317/>
- Tantakoun, Nat, Christian Muise, and Yilun Zhu. 2025. "Large Language Models as Planning Formalizers: A Survey." *Findings of ACL 2025*. <https://aclanthology.org/2025.findings-acl.1291/>
- Tziafas, Georgios, and Hamidreza Kasaei. 2026. "Neuro-Symbolic Task Planning and Object Manipulation with Large Language Models." *Autonomous Robots* 50. <https://doi.org/10.1007/s10514-026-10230-6>

- Wang, Bailin, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2018. "Execution-Guided Neural Program Decoding." *Proceedings of the First Workshop on Learned Program Synthesis*. <https://arxiv.org/abs/1807.03100>
- Wang, Boxin, Chejian Xu, Xiangyu Liu, Yu Cheng, and Bo Li. 2023. "Robustness-Aware Word Embedding Improves Certified Robustness to Adversarial Word Substitutions." *Findings of ACL 2023*. Association for Computational Linguistics. <https://aclanthology.org/2023.findings-acl.620/>
- Wang, Jiayimei, Tao Ni, Wei-Bin Lee, and Qingchuan Zhao. 2025. "A Contemporary Survey of Large Language Model Assisted Program Analysis." *Trends in Artificial Intelligence*. <https://arxiv.org/abs/2502.18474>
- Wang, Wenguan, Yi Yang, and Fei Wu. 2025. "Towards Data- and Knowledge-Driven AI: A Survey on Neuro-Symbolic Computing." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47(2): 878–899. <https://doi.org/10.1109/TPAMI.2024.3483273>
- Wang, Yu, You Zhang, Hao Zhang, and Dan Xu. 2026. "YNU-NLP at SemEval-2026 Task 11: A Neuro-Symbolic Approach with Reflexion Mechanism Disentangling Content and Formal Reasoning in Language Models." *Proceedings of the 20th International Workshop on Semantic Evaluation*. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2026.semeval-1.126>
- Weng, Ke, Lun Du, Sirui Li, Wangyue Lu, Haozhe Sun, Hengyu Liu, and Tiancheng Zhang. 2025. "Autoformalization in the Era of Large Language Models: A Survey." *arXiv:2505.23486*. <https://arxiv.org/abs/2505.23486>
- Xia, Shihao, Mengting He, Shuai Shao, Tingting Yu, Yiyi Zhang, Nobuko Yoshida, and Linhai Song. 2026. "SymGPT: Auditing Smart Contracts via Combining Symbolic Execution with Large Language Models." *Proceedings of the ACM on Programming Languages* 10 (OOPSLA1), Article 109. <https://doi.org/10.1145/3798217>
- Yang, Zonglin, et al. 2024. "Autoformalization of Natural Language Reasoning Problems to First-Order Logic." *Proceedings of EMNLP 2024*. Association for Computational Linguistics. <https://aclanthology.org/2024.emnlp-main.1132/>
- Ye, Mao, Chengyue Gong, and Qiang Liu. 2020. "SAFER: A Structure-Free Approach for Certified Robustness to Adversarial Word Substitutions." *Proceedings of ACL 2020*. Association for Computational Linguistics. <https://aclanthology.org/2020.acl-main.317/>
- Ye, Xi, Qiaochu Chen, Isil Dillig, and Greg Durrett. 2023. "SatLM: Satisfiability-Aided Language Models Using Declarative Prompting." *Advances in Neural Information Processing Systems* 36. <https://arxiv.org/abs/2305.09656>
- Zheng, Zihan, Tianle Cui, Chuwen Xie, Jiahui Pan, Qianglong Chen, and Lewei He. 2025. "PlanningArena: A Modular Benchmark for Multidimensional Evaluation of Planning and Tool Learning." *Proceedings of ACL 2025*, 31047–31086. <https://doi.org/10.18653/v1/2025.acl-long.1499>